UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR
DE INGENIEROS DE TELECOMUNICACIÓN



MÁSTER UNIVERSITARIO EN
INGENIERÍA DE TELECOMUNICACIÓN
TRABAJO FIN DE MÁSTER

DISEÑO DE MODELOS DE OPTIMIZACIÓN PARA LA ASIGNACIÓN DE RECURSOS Y GESTIÓN DE EQUIPOS EN PROYECTOS

ANDRÉS ROQUES SÁNCHEZ

2026

MÁSTER UNIVERSITARIO EN INGENIERÍA DE TELECOMUNICACIÓN

TRABAJO FIN DE MÁSTER

Título: Diseño de modelos de optimización para la asignación de recursos y gestión de equipos en proyectos

Autor: D. Andrés Roques Sánchez

Tutor: D. David Carramiñana Jiménez

Ponente: Dª. Ana María Bernardos Barbolla

Departamento: Dpto. de Señales, Sistemas y Radiocomunicaciones (SSR)

MIEMBROS DEL TRIBUNAL

Presidente: D. .....

Vocal: D. .....

Secretario: D. .....

Suplente: D. .....

Los miembros del tribunal arriba nombrados acuerdan otorgar la calificación de: .....

Madrid, a de de 20...

UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR
DE INGENIEROS DE TELECOMUNICACIÓN



MÁSTER UNIVERSITARIO EN INGENIERÍA DE TELECOMUNICACIÓN
TRABAJO FIN DE MÁSTER

DISEÑO DE MODELOS DE OPTIMIZACIÓN PARA LA ASIGNACIÓN DE RECURSOS Y GESTIÓN DE EQUIPOS EN PROYECTOS

ANDRÉS ROQUES SÁNCHEZ

2026

RESUMEN

La asignación eficiente de recursos humanos a proyectos constituye un problema relevante en la gestión de organizaciones que trabajan con varias iniciativas simultáneas. Cada proyecto presenta necesidades específicas de horas, perfiles profesionales, competencias y dedicación económica, mientras que los trabajadores difieren en disponibilidad, coste, especialización y compromisos previos. Esta interacción entre elementos dificulta la formación manual de equipos y justifica el empleo de herramientas cuantitativas de apoyo a la toma de decisiones.

En este contexto, el presente Trabajo Fin de Máster desarrolla un sistema reproducible para la asignación anual de trabajadores a una cartera multiproyecto. El problema se formaliza mediante dos modelos de programación lineal entera mixta. El primero utiliza un enfoque monoobjetivo y minimiza las horas totales asignadas. El segundo, en cambio, aplica programación por objetivos y combina, mediante términos normalizados, las desviaciones de horas, coste e ineficiencia asociada a la compatibilidad de competencias. Como alternativa de menor coste computacional, se implementa además una heurística constructiva multiarranque que combina reglas de cobertura de roles y competencias, un cociente dinámico basado en la capacidad residual y cuatro reinicios con diferentes órdenes de construcción. Los tres métodos comparten las mismas restricciones y sus soluciones se someten a un proceso común de validación.

Para contrastar estos enfoques, la evaluación experimental comprende 1.000 escenarios, distribuidos en diez tipologías empresariales y cien instancias por tipología. Los modelos exactos encuentran una solución óptima en 892 escenarios, mientras que la heurística obtiene 509 soluciones válidas, equivalentes al 50,9 % del total y al 57,06 % de los escenarios factibles de referencia. En cuanto al tiempo de cómputo, la heurística presenta el menor tiempo medio de resolución, con 0,208 segundos por escenario, frente a 0,760 segundos del modelo monoobjetivo y 1,354 segundos del multiobjetivo. Sin embargo, esta rapidez se obtiene a cambio de una menor capacidad de resolución y una pérdida de calidad: en los 509 escenarios resueltos por los tres métodos, la heurística asigna un 17,40 % más de horas que el monoobjetivo. Por su parte, el modelo multiobjetivo reduce un 36,12 % la ineficiencia normalizada de competencias respecto al monoobjetivo, con un incremento horario medio limitado al 1,20 % y un coste total medio prácticamente estable.

Finalmente, el sistema se complementa con un agente basado en un modelo de lenguaje de gran tamaño, configurado como capa de interpretación de resultados individuales. Este agente explica estados, métricas y posibles riesgos sin ejecutar los modelos, modificar las asignaciones ni sustituir el criterio del responsable. En conjunto, los resultados muestran que los métodos exactos ofrecen mayores garantías de factibilidad y calidad, que el enfoque multiobjetivo permite obtener asignaciones técnicamente más equilibradas y que la heurística constituye una alternativa rápida para contextos con suficiente holgura, aunque no reemplaza a los modelos exactos en escenarios restrictivos.

PALABRAS CLAVE

Asignación de recursos humanos, gestión multiproyecto, programación lineal entera mixta (MILP), optimización monoobjetivo, optimización multiobjetivo, programación por objetivos, heurística multiarranque, modelos de lenguaje de gran tamaño (LLM), Pyomo, HiGHS, Microsoft Copilot Studio.

SUMMARY

Efficient allocation of human resources to projects constitutes a significant challenge for organizations managing several simultaneous initiatives. Each project has specific requirements in terms of working hours, professional roles, skills and budget, whereas employees differ in availability, cost, expertise and prior commitments. The interaction among these factors makes manual team formation difficult and justifies the use of quantitative decision-support tools.

Within this context, this master's thesis develops a reproducible system for the annual allocation of employees across a multi-project portfolio. The problem is formalized using two mixed-integer linear programming models. The first adopts a single-objective approach and minimizes the total number of hours assigned. The second applies goal programming and combines normalized measures of deviations in hours and cost with the inefficiency associated with skill compatibility. As a less computationally demanding alternative, a multi-start constructive heuristic is also implemented. It combines role- and skill-coverage rules, a dynamic ratio based on residual capacity, and four restarts using different construction orders. The same constraints are applied to all three methods, and their solutions undergo a common validation process.

To compare these approaches, the experimental evaluation comprises 1,000 scenarios distributed across ten business scenario types, with one hundred replications per type. The exact models find optimal solutions in 892 scenarios, whereas the heuristic produces 509 valid solutions, equivalent to 50.9% of the total and 57.06% of the scenarios found feasible by the exact models. Regarding computational performance, the heuristic records the lowest mean solution time, at 0.208 seconds per scenario, compared with 0.760 seconds for the single-objective model and 1.354 seconds for the multi-objective model. However, this speed comes at the cost of a lower solution rate and reduced solution quality: across the 509 scenarios solved by all three methods, the heuristic assigns 17.40% more hours than the single-objective model. The multi-objective model, in turn, reduces normalized skill inefficiency by 36.12% relative to the single-objective model, with a modest average increase of 1.20% in assigned hours and no evidence of a systematic difference in total cost.

Finally, the system is complemented by a large language model-based agent configured as an interpretation layer for individual results. The agent explains solution statuses, metrics and potential risks without executing the models, modifying assignments or replacing the decision-maker's judgement. Overall, the results show that exact methods provide stronger guarantees of feasibility and solution quality, that the multi-objective approach produces more technically balanced assignments, and that the heuristic represents a fast alternative for contexts with sufficient slack, although it cannot replace exact models in tightly constrained scenarios.

KEYWORDS

Human resource allocation, multi-project management, mixed-integer linear programming (MILP), single-objective optimization, multi-objective optimization, goal programming, multi-start heuristic, large language models (LLM), Pyomo, HiGHS, and Microsoft Copilot Studio.

AGRADECIMIENTOS

A mi madre. Mi santa madre.

ÍNDICE DEL CONTENIDO

RESUMEN 5
SUMMARY 7
LISTA DE SIGLAS Y ACRÓNIMOS 15
1. INTRODUCCIÓN Y OBJETIVOS 17
1.1. Introducción 17
1.2. Objetivos 18
1.3. Medios materiales 19
1.4. Estructura del documento 19
2. ESTADO DEL ARTE 21
2.1. Asignación de recursos humanos 21
2.2. MILP: Programación Lineal Entera Mixta 23
2.3. Optimización monoobjetivo 25
2.4. Optimización multiobjetivo 25
2.5. Programación por objetivos 30
2.6. Heurísticas y metaheurísticas 31
2.7. Agentes LLM para interpretación de resultados 36
3. DESARROLLO DE LA SITUACIÓN EMPRESARIAL EVALUADA 39
3.1. Descripción de la situación empresarial 39
3.2. Modelado del recorrido del problema 40
3.3. Formulación matemática del problema 43
3.4. Diseño de los métodos de resolución 46
3.4.1. Modelo de optimización de un objetivo 46
3.4.2. Modelo heurístico 48
3.4.3. Modelo de optimización multiobjetivo 50
3.4.4. Coherencia y comparabilidad entre los métodos 51
3.5. Metodología de evaluación 52
4. DESARROLLO COMPUTACIONAL DE LOS MÉTODOS DE RESOLUCIÓN 54
4.1. Arquitectura software y componentes 54
4.2. Análisis de herramientas y soluciones de software 55
4.3. Configuración de escenarios 57
4.4. Generador de escenarios 58
4.5. Construcción de los métodos de resolución 59
4.5.1. Construcción del modelo monoobjetivo 60
4.5.2. Construcción del modelo multiobjetivo 62

4.5.3. Construcción de la heurística constructiva multiarranque 63
4.6. Resolución y gestión de estados 70
4.7. Validación y orquestación 72
5. CONFIGURACIÓN DEL SISTEMA AGÉNTICO 75
5.1. Creación y configuración del agente 76
5.1.1. Bloques en la construcción del agente 76
5.1.2. Inputs, outputs y protocolo de interpretación 78
5.1.3. Hallazgos en las interpretaciones 78
5.2. Evaluación del agente 79
6. EVALUACIÓN Y RESULTADOS 84
6.1. Creación de escenarios e instancias 84
6.1.1. Variaciones controladas del escenario 87
6.1.2. Parametrización de los escenarios 88
6.1.3. Configuración de los tipos de escenarios 91
6.2. Metodología de evaluación experimental 92
6.3. Experimento general: 1000 instancias 93
6.3.1. Tasa de éxito y de resolución 94
6.3.2. Diagnóstico de los bloqueos heurísticos 96
6.3.3. Tiempo de resolución 98
6.3.4. Calidad en la solución de los modelos 100
6.4. Uso del sistema agéntico como ayuda a la toma de decisión de los resultados obtenidos 103
6.4.1. Funcionalidades del agente en una situación real 104
7. CONCLUSIONES Y LÍNEAS FUTURAS 107
7.1. Conclusiones 107
7.2. Líneas futuras 108
8. BIBLIOGRAFÍA 110
ANEXO A: ASPECTOS ÉTICOS, ECONÓMICOS, SOCIALES Y AMBIENTALES 114
A.1 Introducción 114
A.2 Descripción de impactos relevantes relacionados con el proyecto 114
A.3 Análisis detallado de alguno de los principales impactos 114
A.4 Conclusiones 115
ANEXO B: DECLARACIÓN SOBRE EL USO DE INTELIGENCIA ARTIFICIAL EN EL TRABAJO 116
ANEXO C: INSTRUCCIONES EN LA CONFIGURACIÓN DEL AGENTE LLM 117

ANEXO D: RESPUESTAS EXACTAS DEL AGENTE 119
ANEXO E: PRESUPUESTO ECONÓMICO..... 124

ÍNDICE DE FIGURAS Y TABLAS

Figura 1. Composición de un problema de asignación de recursos (elaboración propia). 21
Figura 2. Relación entre MILP y optimización (elaboración propia). 26
Figura 3. Representación del frente de Pareto (adaptada sobre [6]) 27
Figura 4. Resolución por programación por objetivos (elaboración propia). 31
Figura 5. Diferenciación entre heurísticas (elaboración propia). 32
Figura 6. Heurística *multi-start* (elaboración propia). 33
Figura 7. Separación conceptual entre generación lingüística, cálculo y ayuda a la decisión (elaboración propia). 36
Figura 8. Utilidad de un agente LLM para el apoyo a la decisión post-resultados (elaboración propia). 38
Figura 9. Flujo computacional. 41
Figura 10. Ciclo experimental completo. 42
Figura 11. Estructura de conjuntos de trabajadores y proyectos. 43
Figura 12. Conjuntos (T-P): A_min y A_exa 44
Figura 13. Orden de reglas aplicadas a la heurística. 49
Figura 14. Anidación de estructuras de la heurística. 64
Figura 15. Diagrama de flujo de la heurística (elaboración propia). 64
Figura 16. Visualización de la factibilidad de la heurística vs. el número de arranques. 66
Figura 17. Diagrama de flujo de los estados de la solución (elaboración propia). 72
Figura 18. Interfaz gráfica de LLM y preguntas de la evaluación. 81
Figura 19. Ejecución de evaluación del agente. 82
Figura 20. Representación de las 7 variantes de escenario (elaboración propia). 87
Figura 21. Matriz de compatibilidad entre *skills*. 91
Figura 22. *Pipeline* del experimento (elaboración propia). 92
Figura 23. Factibilidad de los modelos según el escenario resuelto. 96
Figura 24. Primera causa de los bloqueos heurísticos totales. 97
Figura 25. Bloqueos heurísticos por falta de competencia individual. 98
Figura 26. Tiempo medio por ejecución de los modelos empleados. 99
Figura 27. Tiempo medio por modelo y tipo de escenario. 100
Figura 28. Calidad de la solución en función de su *score* para cada modelo. 102
Figura 29. Diferencia de horas asignadas de la heurística respecto al modelo monoobjetivo. 103
Figura 30. Interfaz de pregunta del agente. 105

Tabla 1. Ejemplo de factor de compatibilidad de skills 44
Tabla 2. Conjuntos del escenario. 45
Tabla 3. Variables del escenario. 45
Tabla 4. Resumen de parámetros del escenario. 46
Tabla 5. Resumen de la estructura del proyecto 55
Tabla 6. Herramientas y tecnologías empleadas en el trabajo. 56
Tabla 7. Modificaciones probadas en cada versión heurística. 68
Tabla 8. Resultados de cada versión heurística. 68
Tabla 9. Funciones por cada fase de la construcción heurística. 69
Tabla 10. Estados posibles de las soluciones. 71
Tabla 11. Archivos generados tras la ejecución. 74
Tabla 12. Reglas en el entendimiento del LLM. 77
Tabla 13. Preguntas empleadas en la evaluación del agente LLM. 80
Tabla 14. Métodos de evaluación empleados. 81
Tabla 15. Descripción de atributos de cada rol. 85
Tabla 16. Lectura empresarial del índice de carga. 86
Tabla 17. Identificación de cada variante propuesta. 89
Tabla 18. Modificaciones cuantificadas de los parámetros de los escenarios. 89

Tabla 19. Auditoría del resultado. 93
Tabla 20. Pruebas unitarias de validación. 93
Tabla 21. Estado de las soluciones según los modelos 94
Tabla 22. Desagregación de la factibilidad de los métodos según el escenario resuelto. 95
Tabla 23. Análisis de tiempo de ejecución [s] de todas las instancias. 99
Tabla 24. Análisis de tiempo de ejecución [s] de las instancias que pasan el "precheck". 99
Tabla 25. Comparación de los resultados de los modelos exactos. 101
Tabla 26. Comparación de los resultados de tres modelos (bajo los resultados de la heurística). 101
Tabla 27. Cantidad de instancias, según su tipo, resueltas por la heurística. 103
Tabla 28. Conversaciones con el Agente LLM. 105

LISTA DE SIGLAS Y ACRÓNIMOS

Técnicos y metodológicos:

IA: Artificial Intelligence / Inteligencia Artificial.

API: Application Programming Interface / Interfaz de Programación de Aplicaciones.

CBC: COIN-OR Branch-and-Cut / Solucionador de ramificación y corte de COIN-OR.

CP: Constraint Programming / Programación por Restricciones.

CP-SAT: Constraint Programming–Satisfiability / Programación por Restricciones–Satisfacibilidad.

CPLEX: IBM ILOG CPLEX Optimization Studio / Entorno de optimización IBM ILOG CPLEX.

CRM: Customer Relationship Management / Gestión de Relaciones con Clientes.

CSV: Comma-Separated Values / Valores Separados por Comas.

ERP: Enterprise Resource Planning / Planificación de Recursos Empresariales.

GLPK: GNU Linear Programming Kit / Kit GNU de Programación Lineal.

HiGHS: High-performance software for linear optimization / Software de alto rendimiento para optimización lineal.

IGD: Inverted Generational Distance / Distancia Generacional Invertida.

JSON: JavaScript Object Notation / Notación de Objetos de JavaScript.

LLM: Large Language Model / Modelo de Lenguaje de Gran Tamaño.

LP: Linear Programming / Programación Lineal.

MCP: Model Context Protocol / Protocolo de Contexto de Modelo.

MILP: Mixed-Integer Linear Programming / Programación Lineal Entera Mixta.

MIP: Mixed-Integer Programming / Programación Entera Mixta.

MSRCPSP: Multi-Skilled Resource-Constrained Project Scheduling Problem / Problema de Programación de Proyectos con Recursos Limitados y Personal Multicompetencia.

OR-Tools: Operations Research Tools / Herramientas de Investigación Operativa.

PuLP: Linear Programming Modeler in Python / Modelador de Programación Lineal en Python.

Pyomo: Python Optimization Modeling Objects / Objetos de Modelado de Optimización en Python.

P95: Percentile 95 / Percentil 95.

QP: Quadratic Programming / Programación Cuadrática.

RAG: Retrieval-Augmented Generation / Generación Aumentada mediante Recuperación.

RAM: Random Access Memory / Memoria de Acceso Aleatorio.

RCPSP: Resource-Constrained Project Scheduling Problem / Problema de Programación de Proyectos con Recursos Limitados.

ReAct: Reasoning and Acting / Razonamiento y Acción.

SAP: Systems, Applications and Products in Data Processing / Sistemas, Aplicaciones y Productos para el Procesamiento de Datos.

SAT: Boolean Satisfiability Problem / Problema de Satisfacibilidad Booleana.

SDK: Software Development Kit / Kit de Desarrollo de Software.

SHA-256: Secure Hash Algorithm 256-bit / Algoritmo de Hash Seguro 256 bits.

SQL: Structured Query Language / Lenguaje de Consulta Estructurado.

TOA: Teamwork Optimization Algorithm / Algoritmo de Optimización de Trabajo en Equipo.

TXT: Text File / Archivo de Texto.

Institucionales, empresariales y estadísticos:

ETSIT: Higher Technical School of Telecommunication Engineers / Escuela Técnica Superior de Ingenieros de Telecomunicación.

INE: Spanish National Statistics Institute / Instituto Nacional de Estadística.

IVA: Value Added Tax / Impuesto sobre el Valor Añadido.

MUIT: Master's Degree in Telecommunication Engineering / Máster Universitario en Ingeniería de Telecomunicación.

PYME: Small and Medium-sized Enterprise / Pequeña y Mediana Empresa.

RR. HH.: Human Resources / Recursos Humanos.

TFM: Master's Thesis / Trabajo Fin de Máster.

TFT: Degree Final Project / Trabajo Fin de Titulación.

TIC: Information and Communication Technologies / Tecnologías de la Información y la Comunicación.

UPM: Technical University of Madrid / Universidad Politécnica de Madrid.

1. INTRODUCCIÓN Y OBJETIVOS

1.1. INTRODUCCIÓN

Una organización orientada a proyectos es un modelo empresarial donde todo el trabajo se organiza, ejecuta y administra en función de proyectos temporales en lugar de departamentos permanentes. En la actualidad es muy habitual adoptar este rumbo de dirección orientado a proyectos debido a su agilidad, velocidad de comercialización (Time-to-Market), experiencia del cliente y mayor ajuste de capacidad y necesidades. De hecho, autores y consultoras globales señalan que estamos viviendo la era de la *"economía de proyectos"*. [1]

Todas las organizaciones que pretendan seguir esta orientación deben transformar capacidades humanas limitadas en equipos capaces de atender simultáneamente una cartera de encargos específicos [2]. La decisión no se reduce a determinar cuántas personas necesita cada proyecto, sino que también exige decidir qué trabajadores participan, cuántas horas aportan, qué función desempeñan, qué competencias reúnen dentro de las requeridas y qué coste implica su dedicación [3]. Cuando varias actividades a realizar requieren de la misma plantilla, una asignación aparentemente razonable para un proyecto puede impedir que otro alcance sus requisitos mínimos [4].

Además, la dificultad de la asignación de estos recursos humanos aumenta debido a que éstos no son homogéneos. Cada trabajador dispone de una capacidad limitada, un coste y un conjunto de habilidades; cada proyecto requiere de necesidades distintas de horas, presupuesto, composición por roles y cobertura de trabajo específico; existen elementos o limitaciones propias de la organización que restringen la capacidad de asignación. A estas condiciones se añaden compromisos contractuales o decisiones previas que obligan a seguir unas ciertas pautas en la toma de decisión [2].

La capacidad agregada no basta para anticipar si la cobertura de una cartera de proyectos es viable o no. Una empresa puede disponer, en conjunto, de más horas asignables que las requeridas y, sin embargo, carecer de un perfil específico que cumpla una labor en el momento necesario. También puede concentrar la capacidad en trabajadores que ya han alcanzado su límite de asignaciones o que no poseen competencias suficientemente compatibles. La factibilidad surge de la interacción entre todas las restricciones, por lo que no puede evaluarse mediante un único cociente entre demanda y disponibilidad [3].

Siendo, por ende, un problema combinatorio, también supone una dificultad agregada su escalabilidad. A medida que crece el número de elementos involucrados en la toma de decisión (trabajadores, proyectos, restricciones, etc.), aumenta rápidamente el conjunto de asignaciones posibles. La planificación manual puede apoyarse en experiencia y reglas sencillas, pero resulta difícil garantizar que todas las condiciones se cumplen, comparar de forma sistemática varias alternativas o justificar que no existe una solución mejor. La literatura sobre personal multicompetencia y programación multiproyecto recurre por ello a modelos de programación matemática, métodos de descomposición y procedimientos aproximados [4].

A esta dificultad de escala se le suma otra de naturaleza distinta, y es que no existe necesariamente un único criterio de decisión. La organización puede preferir diferentes objetivos, como minimizar las horas totales o el coste, maximizar la productividad o conservar margen de capacidad para incidencias [5]. Estos criterios pueden y suelen entrar en conflicto. Una solución de coste reducido puede presentar una peor adecuación técnica, mientras que una asignación muy especializada puede consumir perfiles escasos y disminuir la flexibilidad de la cartera [6].

Frente a estas dos dificultades, la programación lineal entera mixta (MILP) ofrece un marco adecuado para representar conjuntamente variables de decisión continuos y binarias. Sus restricciones permiten traducir reglas de negocio a relaciones verificables para que un programa pueda alcanzar una solución óptima y, cuando la encuentra, garantiza que es la mejor posible [7]. Sin embargo, la componente discreta puede elevar el tiempo de cálculo. Por esta razón, resulta relevante comparar los modelos exactos con otros modelos de aproximación subóptima como heurísticas constructivas, que prioricen rapidez y trazabilidad, aunque no garantice encontrar una solución ni demostrar su optimalidad [8].

La utilidad empresarial del resultado ya sea mediante MILP o heurística, no termina cuando el algoritmo devuelve una solución numérica . El responsable de planificación necesita saber qué restricciones condicionan el resultado, dónde aparecen cuellos de botella y qué compromisos existen entre los parámetros. Las salidas de un algoritmo de resolución o *solver* son precisas, pero suelen expresarse mediante variables y métricas propias que requieren de conocimiento especializado. Para analizar y comprender esta solución, es interesante emplear modelos de Inteligencia Artificial (IA) basados en texto, denominados LLM (*Large Language Model* o Modelos de Lenguaje Extenso) [9]. Los LLM y los agentes programables abren una vía para convertir esa evidencia estructurada en explicaciones y consultas en lenguaje natural, siempre que el cálculo permanezca en herramientas deterministas y la decisión final continúe bajo supervisión humana [10].

Este TFM aborda la problemática mediante un sistema reproducible de apoyo a la decisión. El trabajo formaliza una situación empresarial sintética pero interpretable, desarrolla dos modelos MILP de distinta naturaleza: uno monoobjetivo y otro multiobjetivo basado en programación por objetivos, diseña una heurística constructiva y los compara sobre diferentes escenarios de negocio. Finalmente, plantea un agente LLM orientado a interpretar los resultados de una ejecución concreta. El propósito no es automatizar de forma autónoma la gestión de personas, sino ofrecer al decisor asignaciones verificables, métricas comparables y explicaciones trazables.

La relevancia académica reside en conectar cuatro niveles que a menudo se estudian por separado: la representación del problema empresarial, la resolución matemática, la aproximación heurística y la interpretación de resultados. La relevancia práctica consiste en proponer unos métodos de resolución de problemas explorando herramientas de optimización, analizar qué método conviene emplear según la situación del escenario y qué información necesita un responsable para convertir una solución matemática en una decisión organizativa.

1.2. OBJETIVOS

El objetivo general del presente trabajo es diseñar, implementar y evaluar un sistema de apoyo a la toma de decisión para la asignación de recursos humanos heterogéneos a una cartera de proyectos, comparando métodos exactos y heurísticos e incorporando una capa basada en LLM para la interpretación trazable de los resultados. Para lograr este objetivo, se establecen los siguientes subobjetivos específicos:

1. Caracterizar y formalizar el problema de asignación multiproyecto mediante conjuntos, parámetros, variables y restricciones que representen disponibilidad, horas, costes, roles, competencias, límites de participación y dedicaciones contractuales.

2. Evaluar técnicas de programación lineal entera mixta para resolver el problema de asignación de personal a proyectos y obtener una referencia exacta respecto de la factibilidad y la minimización de horas asignadas.

3. Analizar la programación por objetivos como mecanismo para integrar en una única solución de compromiso las desviaciones normalizadas de horas, coste e ineficiencia asociada a la compatibilidad de competencias.

4. Diseñar una heurística constructiva multiarranque veloz, estable y trazable, y determinar su capacidad para encontrar soluciones factibles de calidad en un tiempo inferior al de los modelos exactos.

5. Construir un marco experimental común que permita comparar tasa de éxito, tiempo de cálculo, distancia a la optimalidad, la robustez ante cambios y estabilidad de los métodos sobre escenarios e instancias con variaciones controladas de demanda, composición y rigidez.

6. Diseñar y evaluar un agente basado en LLM para interpretar el resultado de un escenario, explicar métricas y restricciones relevantes y generar recomendaciones para el decisor sin sustituir el cálculo del *solver* ni la validación humana.

1.3. MEDIOS MATERIALES

Equipo informático:

- Lenovo IdeaPad S340-15IWL (modelo 81N1).
- Procesador Intel Core i5-8265U @ 1,60 GHz.
- 4 núcleos y 8 procesadores lógicos.
- Memoria RAM: 8 GB (velocidad 2400MHz).
- Sistema operativo de 64 bits, Windows 26200.9168
- Python 3.10.11
- Pyomo 6.10.1.devo
- Visual Studio Code 1.135.0
- Microsoft Copilot Studio en el entorno de Universidad Politécnica de Madrid.

1.4. ESTRUCTURA DEL DOCUMENTO

La memoria está construida para iniciar desde una necesidad empresarial como es la de gestionar recursos humanos de una manera eficiente, avanza hacia su formalización matemática y termina evaluando tanto la resolución como la interpretación de los resultados a través de una metodología definida. Esta secuencia permite leer el trabajo con distintos niveles de profundidad sin perder la relación entre las reglas del negocio, los modelos y la evidencia experimental.

En primer lugar, los capítulos 2 y 3 constituyen el bloque conceptual y de diseño del trabajo. El trabajo comienza en el Capítulo 2 proporcionando la base conceptual necesaria para comprender el problema y los métodos: asignación de recursos humanos, MILP, optimización monoobjetivo y multiobjetivo, programación por objetivos, heurísticas y agentes LLM. Sobre esta base teórica, en el Capítulo 3 se traslada la literatura al problema concreto abordado, describiendo la situación empresarial, su modelado computacional y formulación matemática, así como el diseño de los métodos de resolución empleados.

A continuación, el Capítulo 4 traslada este diseño a una implementación reproducible, explicando la arquitectura del software, las herramientas utilizadas, la configuración y generación de escenarios, la construcción y resolución de los métodos, la gestión de estados, la validación independiente y la orquestación de los experimentos.

Una vez cerrado el procesamiento determinista, el Capítulo 5 desarrolla el componente agéntico encargado de interpretar la solución de un escenario sin modificar los valores calculados ni sustituir la validación matemática. Se describe su configuración, las reglas utilizadas para interpretar las salidas y el proceso seguido para evaluar su comportamiento.

Por su parte, el Capítulo 6 presenta la evaluación experimental. En él se definen las variaciones y tipologías de escenarios utilizadas para probar los métodos y se analizan sus resultados en términos de factibilidad, tiempo de resolución y calidad de las soluciones. Asimismo, se estudian las causas de los bloqueos heurísticos y se analiza el empleo del agente como herramienta de apoyo a la toma de decisiones.

Finalmente, el Capítulo 7 reúne las principales conclusiones obtenidas y plantea las líneas futuras del trabajo, mientras que el Capítulo 8 recoge la bibliografía empleada. Los anexos completan la memoria con el análisis de impactos éticos, económicos, sociales y ambientales, las instrucciones empleadas en la configuración del agente LLM, sus respuestas completas durante la evaluación y el presupuesto económico del proyecto. De esta forma, el cuerpo principal conserva el argumento técnico, mientras que la información complementaria permanece accesible para los lectores interesados en la implementación y sus implicaciones.

Atendiendo al interés específico del lector, los aspectos relacionados con la gestión y la caracterización empresarial del problema se desarrollan principalmente en la Sección 1.1, la Sección 3.1 y la Sección 6.4, donde se presenta respectivamente la problemática general, su aplicación al escenario empresarial y el uso de los resultados como apoyo a la toma de decisiones. Los fundamentos y métodos de optimización se concentran en las Secciones 2.2 a 2.6, mientras que su adaptación al problema concreto y su formulación se desarrollan en las Secciones 3.2 a 3.5. La implementación computacional, la construcción de los métodos, su resolución y validación se recogen en las Secciones 4.1 a 4.7, y su comportamiento experimental se analiza principalmente en la Sección 6.3. Por último, la integración de inteligencia artificial como apoyo a la interpretación se introduce en la Sección 2.7, se desarrolla en el Capítulo 5 y se evalúa en su aplicación práctica en la Sección 6.4.

2. ESTADO DEL ARTE

2.1. ASIGNACIÓN DE RECURSOS HUMANOS

La asignación de recursos en gestión de proyectos es un problema bien conocido, que estudia cómo distribuir un conjunto limitado de recursos entre actividades que compiten por ellos. Los elementos clave que formalizan el problema son los recursos y las actividades, una función objetivo, la meta de buscar un valor óptimo y unas restricciones que siguen las condiciones particulares del problema entre recursos y tareas o algún atributo específico de ellos. Por ejemplo: un problema de asignación de recursos consiste en distribuir elementos disponibles, como personal o maquinaria, a distintas actividades o proyectos con el fin de minimizar costes o tiempos, o maximizar beneficios. En la Figura 1 vemos gráficamente la composición de un problema de asignación de recursos habitual [11].



Figura 1. Composición de un problema de asignación de recursos (elaboración propia).

En el problema de asignación clásico, cada agente se vincula con una única tarea y cada tarea recibe un único agente, de forma que se minimiza un coste total o se maximiza un beneficio. Cuando los agentes disponen de capacidad para atender varias tareas, aparecen requisitos adicionales o la composición del grupo influye en el resultado, el problema se aproxima a la asignación generalizada, la formación de equipos y la planificación de recursos [11].

La planificación de recursos humanos introduce características que no están presentes en la formulación de problemas de asignación clásicos. Las personas están sujetas a unas disponibilidades, costes, roles, competencias y preferencias diferentes; los proyectos a su vez pueden exigir cantidades mínimas de trabajo, perfiles concretos, conocimientos técnicos o límites de tamaño de equipo. También pueden existir incompatibilidades, dedicaciones previamente comprometidas y restricciones sobre el número de actividades simultáneas. Como consecuencia, la decisión deja de ser una correspondencia de carácter uno a uno y pasa a combinar varias asignaciones, selección de participantes y composición del equipo [3].

En los problemas industriales reales se incorpora una mayor cantidad de elementos a gestionar en el problema. Aparecen recursos con habilidades diferentes, restricciones complejas, objetivos de planificación, etc. [12] Esto promueve el empleo tanto de MILP como

de algoritmos matemáticos para abordar instancias grandes [12]. Bajo un contexto de proyectos de tecnologías de la información, la planificación centralizada y la consideración de la eficiencia de asignación de las cualidades técnicas de los trabajadores pueden aportar soluciones factibles y de menor coste frente a reglas simples [2].

Por ello, es relevante conocer las dimensiones de dificultad de esta clase de problemas. Debemos tener en cuenta que la capacidad es compartida, los trabajadores no son objetos ni recursos homogéneos, pueden producirse acoplamientos (ya que "compiten" por los mismos recursos) o los objetivos pueden no estar alineados unos con otros. De hecho, en este último punto es lógico pensar que los objetivos varíen según el transcurso del proyecto o que sea una combinación entre varias metas [4].

Es normal encontrar representadas las propiedades más objetivas de los recursos, requeridas por las actividades, mediante matrices binarias, escalas ordinales o grados de compatibilidad. Una matriz binaria indica si una persona posee o no una habilidad; una escala ordinal distingue niveles de dominio; y una matriz de compatibilidad permite expresar sustituciones parciales entre conocimientos relacionados. Las preferencias añaden una dimensión distinta, pues miden el grado de satisfacción del participante o del equipo. En este tipo de problemas es frecuente combinar restricciones duras de elegibilidad con criterios blandos de afinidad o satisfacción [13].

A día de hoy, existen numerosos problemas de asignación en el ámbito organizativo cuya resolución se basa en este tipo de formulación. La asignación automatizada de estudiantes a proyectos finales [14] o la asignación de personal constituyen un ejemplo de formación de equipos basada en competencias, intereses y requisitos. Las variables enteras determinan la pertenencia a cada grupo y las restricciones garantizan tamaños y condiciones de composición válidos. Una vez obtenido un resultado válido, puede evaluarse su grado de adecuación o calidad de la solución mediante indicadores agregados (satisfacción media, preferencias cumplidas…), su comparación con cotas o soluciones de referencia, la distribución y equidad de resultados entre equipos o su estabilidad ante variaciones en los datos de entrada [15].

La asignación estudiada se encuentra en la intersección de tres familias: asignación de recursos, formación de equipos y planificación multiproyecto. El problema clásico de asignación vincula agentes y tareas de forma discreta; la formación de equipos incorpora composición y competencias; y la planificación multiproyecto introduce capacidad compartida entre iniciativas simultáneas. El caso desarrollado añade una particularidad: la decisión principal no es únicamente binaria, porque cada vínculo trabajador–proyecto lleva asociada una cantidad de horas y una distribución entre competencias.

Una tipología de problema estrechamente relacionada con la asignación de recursos humanos es el *Multi-Skilled Resource-Constrained Project Scheduling Problem* (MSRCPSP), extensión del problema clásico de programación de proyectos con recursos limitados en la que los recursos disponen de diferentes competencias [3]. Su formulación incorpora, además de las restricciones habituales de disponibilidad y precedencia entre actividades, la necesidad de garantizar que los recursos asignados posean las competencias requeridas para ejecutar cada tarea. Esto introduce una dimensión adicional de heterogeneidad, ya que los trabajadores no son necesariamente intercambiables y su capacidad para cubrir una determinada necesidad depende de su perfil competencial.

Dentro de esta familia de problemas, la representación de las competencias puede incorporar diferentes niveles de dominio y distintos grados de flexibilidad en la asignación de los recursos. De este modo, la planificación no depende únicamente de la cantidad de capacidad disponible, sino también de la correspondencia entre las competencias requeridas por las actividades y las disponibles en la plantilla. Esta consideración convierte la asignación de personal en una decisión estrechamente vinculada con la programación temporal del proyecto [3].

La complejidad aumenta cuando los recursos se comparten entre varios proyectos. En este contexto, la planificación deja de realizarse de forma independiente para cada proyecto, ya que las decisiones adoptadas en uno de ellos condicionan la capacidad disponible para los restantes. Los problemas de programación multiproyecto con recursos compartidos analizan precisamente esta interacción, incorporando simultáneamente restricciones de capacidad, competencia por recursos y relaciones temporales dentro de una cartera de proyectos [4].

2.2. MILP: PROGRAMACIÓN LINEAL ENTERA MIXTA

MILP (*Mixed Integer Linear Programming*) o Programación Lineal Entera Mixta, es un marco de formulación matemática que integra programación lineal con variables enteras, permitiendo modelar métodos de resolución de problemas complejos, incluyendo variables de decisión continuas y discretas. Su uso está muy extendido en diversos campos como la optimización energética, la logística de transporte o distribución, la producción o la gestión de grupos o recursos humanos [7].

El propósito principal y principal ventaja del método es la resolución óptima de una función objetivo teniendo en cuenta ciertas restricciones definidas, obteniendo así el mejor resultado posible mediante un método supervisado y fiable, que emplea ecuaciones lineales donde algunas variables son números enteros y otras pueden tener decimales.

La programación lineal optimiza una función lineal sometida a un conjunto de restricciones también lineales. Su forma general puede expresarse como:

$$\min\{c^T x\} \qquad \text{sujeto a} \qquad A\,x \leq b, \quad x \geq 0,$$

donde x es el vector de variables de decisión, c es el vector de coeficientes de la función objetivo, A es la matriz de coeficientes de las restricciones y b es el vector de recursos o límites disponibles.

Cuando los datos pueden expresarse mediante variables que adoptan valores enteros, se obtiene un modelo de programación lineal entera. Las variables continuas, en cambio, representan magnitudes divisibles, como tiempo, producción o presupuesto, algo habitual en este tipo de asignaciones. En el caso de que ambos tipos de variables convivan en un mismo problema, se obtiene el modelo que ya hemos definido como MILP. Además, las variables binarias, restringidas a los valores cero y uno, permiten expresar decisiones lógicas, como seleccionar un recurso, activar una relación o abrir una instalación.

La elección de MILP se justifica por la naturaleza híbrida de las decisiones usuales en el problema a tratar. En el caso de una asignación de RRHH, se emplean recursos de valor entero y decisiones binarias, además de que existen varias relaciones lineales entre recursos y tareas. Esta combinación permite representar el problema sin recurrir a aproximaciones no lineales y aprovechar algoritmos exactos que proporcionan una solución factible, una cota y cuando termina con estado óptimo, una certeza de que no existe una solución mejor para la función definida.

Los modelos MILP son adecuados para problemas de asignación en los que deben decidirse simultáneamente el volumen de dedicación y relaciones de pertenencia. En ellos, traducimos cada una de las reglas empresariales en una familia de restricciones indexada por los parámetros de cada problema, por ejemplo: trabajadores, proyectos, roles o competencias. Estas restricciones pueden representar capacidad, cobertura de demanda, número máximo de asignaciones, composición por categorías y relaciones de enlace entre una variable continua y otra binaria. Su principal ventaja es la precisión de la formulación: una solución factible

satisface todas las restricciones incluidas, y una solución óptima certificada no puede ser mejorada dentro del modelo definido [7].

La formulación requiere expresar cada regla de negocio o requisito con exactitud para interpretarla matemáticamente antes de la resolución. Por ejemplo, *"una persona puede participar como máximo en R proyectos"* no queda como una instrucción informal, sino como una suma de variables binarias acotada. Del mismo modo, *"el rol k debe realizar una fracción mínima de horas de un proyecto"* se convierte en una desigualdad que puede validarse numéricamente. De esta manera, permite verificar, con casos de prueba sencillos, si cada restricción representa correctamente el requisito de negocio antes de resolver el modelo completo.

La dificultad computacional procede principalmente de la componente discreta. Si se permite temporalmente que las variables enteras o binarias tomen cualquier valor real (lo que se conoce como relajar las condiciones de integralidad) se obtiene la relajación lineal, cuya solución proporciona una cota para el problema entero. Los algoritmos exactos explotan estas cotas para descartar regiones del espacio de búsqueda sin enumerar todas las combinaciones. Entre los mecanismos más habituales se encuentran combinaciones de [7]:

- Simplex y métodos de punto interior, empleados para resolver problemas lineales y relajaciones continuas.
- *Branch and Bound*, que divide el problema en subproblemas, calcula cotas y elimina nodos incapaces de mejorar la mejor solución conocida.
- *Branch and Cut*, que combina la ramificación con desigualdades válidas destinadas a reforzar la relajación lineal.
- *Presolve* y propagación, que simplifican restricciones, fijan variables y detectan incompatibilidades antes o durante la búsqueda.
- Heurísticas internas, que intentan obtener soluciones factibles tempranas y mejorar la cota incumbente.

La resolución exacta aporta garantías de factibilidad y optimalidad, pero su coste puede crecer rápidamente con el número de variables binarias y la interacción entre restricciones. La comparación entre formulaciones MILP y procedimientos heurísticos muestra una relación habitual entre calidad y tiempo: los métodos exactos proporcionan una referencia mientras que los procedimientos aproximados pueden alcanzar soluciones de buena calidad con menor esfuerzo computacional [8].

A estas alternativas se le suma la programación por restricciones (*Constraint Programming* o CP) y los métodos basados en satisfacción booleana, conocidos como SAT, (del inglés *Satisfiability*), que constituyen otra familia de resolución. CP-SAT combina ambos enfoques trabajando con variables enteras, restricciones lógicas y mecanismos de propagación, aprendizaje de cláusulas y búsqueda combinatoria. Resulta especialmente adecuado cuando predominan decisiones discretas, reglas condicionales y restricciones de composición. Si se quisiera emplear estos métodos, hay que tener en cuenta que antes de usar este enfoque, las magnitudes continuas deben transformarse a una escala entera [16].

Aparte de esto, la literatura recoge también métodos exactos diseñados para explotar la estructura específica de los problemas de asignación y planificación multicompetencia. Entre ellos se encuentran enfoques de descomposición que separan distintas decisiones del problema y combinan MILP con programación por restricciones mediante esquemas de Benders [17]. También se han desarrollado procedimientos de *branch-and-price*, que integran ramificación y generación de columnas para resolver formulaciones de gran tamaño sin representar explícitamente todas las alternativas posibles [18]. Estas estrategias permiten

descomponer la resolución y mejorar la eficiencia computacional, aunque requieren algoritmos adaptados a la estructura particular de cada problema.

2.3. OPTIMIZACIÓN MONOOBJETIVO

Como explica el anterior capítulo, la formulación de un problema mediante MILP determina las variables de decisión y las condiciones que debe cumplir cualquier solución admisible. Sin embargo, las restricciones únicamente delimitan el conjunto factible y no establecen, por sí mismas, cuál de sus soluciones debe seleccionarse. Para realizar esta elección es necesario incorporar una función objetivo que permita ordenar las alternativas de acuerdo con un criterio determinado [7].

Sea X el conjunto factible definido por las restricciones del modelo MILP y sea $f(x)$ una función escalar que representa el criterio de decisión. Un problema de optimización monoobjetivo puede expresarse como:

$$F^{on}{}_{objetivo} = \min_{x \in X} f(x)$$

Donde hallaremos una solución $x_m \in X$ óptima si se satisface:

$$f(x_m) \leq f(x), \qquad \forall x \in X.$$

Por tanto, una solución factible es aquella que cumple todas las restricciones, mientras que una solución óptima es una solución factible que alcanza el mejor valor de la función objetivo. Esta distinción resulta fundamental: la factibilidad depende de las restricciones del modelo, mientras que la optimalidad depende del criterio utilizado para comparar las soluciones factibles.

La optimización monoobjetivo proporciona una interpretación directa sobre el requisito único y principal que debe optimizarse. Por ejemplo, si se pretende minimizar el coste de asignación, el modelo identifica la alternativa de menor coste entre todas las que cumplen las restricciones. No obstante, esta solución solo puede considerarse óptima en relación con el criterio económico. No necesariamente será la mejor desde el punto de vista de las demás variantes del problema. Además, por su naturaleza, el modelo monoobjetivo resulta especialmente útil como referencia. Su resolución permite determinar el mejor resultado alcanzable cuando se considera aisladamente un criterio y proporciona un punto de comparación para valorar posteriormente el coste de oportunidad asociado a una solución de compromiso [6].

Dentro de los enfoques exactos de asignación multiproyecto, también se han planteado formulaciones de programación entera mixta que integran simultáneamente decisiones de planificación y dotación de recursos. Estas formulaciones permiten considerar recursos internos y externos, múltiples competencias y diferencias de eficiencia entre trabajadores, utilizando una función objetivo única para ordenar las alternativas factibles. En este contexto, la minimización del coste laboral constituye uno de los criterios empleados para comparar políticas centralizadas de asignación frente a reglas más simples de planificación [2].

2.4. OPTIMIZACIÓN MULTIOBJETIVO

Habiendo definido la optimización de un único objetivo, nos encontramos ahora un inconveniente: los problemas de decisión reales suelen estar determinados por varios criterios que deben considerarse simultáneamente [5]. El coste, el tiempo, la calidad, la adecuación

técnica, la continuidad o el equilibrio de cargas son factores que pueden mejorar o empeorar al mismo tiempo en la resolución del problema. Además, estos criterios suelen encontrarse en conflicto: una reducción del coste suele implicar una menor adecuación de los perfiles asignados, una mejora del equilibrio puede requerir soluciones más costosas, etc. [6] Surge así la necesidad de ampliar la formulación monoobjetivo.

De esta forma se establece la relación conceptual, representada en la Figura 2, donde la formulación MILP define las variables de decisión y las restricciones estructurales del problema, delimitando el conjunto de soluciones factibles. Sobre este conjunto, la optimización monoobjetivo emplea una única función objetivo para seleccionar la mejor alternativa conforme a un criterio específico. Cuando la decisión requiere considerar simultáneamente varios criterios potencialmente conflictivos, esta formulación se amplía hacia la optimización multiobjetivo [5]. Dentro de este tipo de escenarios hay muchas estrategias dispnibles que permiten formalizar una solución, como la ponderación de objetivos, la optimización lexicográfica o la generación de soluciones eficientes y que sirven de apoyo al decisor.



Figura 2. Relación entre MILP y optimización (elaboración propia).

La optimización multiobjetivo representa esta situación mediante un vector de funciones definido sobre un mismo conjunto factible [5] de la siguiente forma: Sea $x \epsilon X$, el vector de variables de decisión y X el conjunto de soluciones que satisfacen todas las restricciones del problema. Bajo el convenio de maximización o minimización, el problema multiobjetivo puede expresarse como [6]:

$$\min_{x \epsilon X} f(x) = \min_{x \epsilon X} \left(f_1(x), \dots, f_q(x)\right) \quad o \quad \max_{x \epsilon X} f(x) = \max_{x \epsilon X} \left(f_1(x), \dots, f_q(x)\right)$$

donde q es el número de objetivos y $f_n(x)$ representa el valor alcanzado por la solución x en el criterio n. El conjunto X constituye el espacio factible de decisión, mientras que su imagen o solución $f(X)$ define el espacio factible de objetivos.

En casi todos los casos, es inviable encontrar una solución factible que alcance simultáneamente el valor óptimo individual de todos los objetivos. Por esta razón, la optimalidad multiobjetivo se define mediante el concepto de dominancia. Dadas dos soluciones factibles $x_1, x_2 \epsilon X$, se dice que x_1 domina a x_2 en el caso de que:

$$f_n(x_1) \leq f_n(x_2), \qquad \forall n = 1, \dots, q,$$

y existe al menos un objetivo j para el que:

$$f_j(x_1) < f_j(x_2),$$

De esta manera aparece el concepto de solución Pareto-óptima u óptimo de Pareto. [19] Un óptimo de Pareto es un punto de equilibrio donde ya no es posible mejorar la situación de una persona o un objetivo sin empeorar la de otra [6]. Por ende, cualquier solución del tipo $x_m \epsilon X$ se considerará Pareto-óptima cuando no exista otra solución factible que mejore al menos un objetivo sin empeorar alguno de los restantes. El conjunto de estas soluciones forma el frente de Pareto, como podemos ver representado en la Figura 3 y permite observar los compromisos existentes entre criterios. La optimización multiobjetivo no conduce necesariamente a una única respuesta, por lo que es necesario incorporar preferencias antes, durante o después de la resolución.

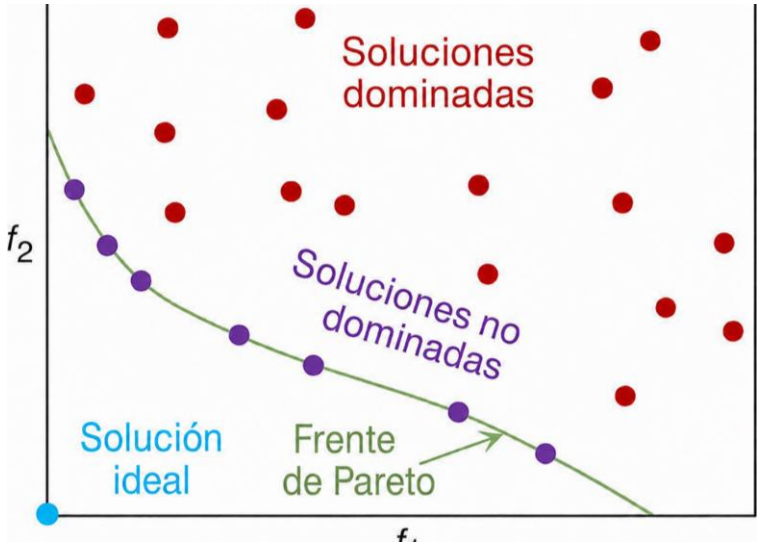


Figura 3. Representación del frente de Pareto (adaptada sobre [6])

Es importante explicar la diferencia entre el conjunto de Pareto y el frente de Pareto. El primero está formado por las soluciones eficientes expresadas en el espacio de decisión:

$$X^P = \{x \epsilon X, \quad la\ solución\ x\ es\ Pareto\text{-}óptima\},$$

mientras que el frente de Pareto corresponde a la imagen de dichas soluciones en el espacio de objetivos:

$$F^P = \{f(x): x \epsilon X^P\}$$

El frente de Pareto es relevante ya que nos permite observar los compromisos existentes entre los distintos criterios. Dos soluciones pertenecientes al frente pueden ser matemáticamente incomparables; por ejemplo, una puede representar un menor coste, mientras que la otra proporciona un mejor ajuste de capacidades o una distribución más equilibrada. Para seleccionar una solución definitiva entre ellas se requiere incorporar información adicional sobre las preferencias del decisor de una manera supervisada o subjetiva.

Otros conceptos útiles para caracterizar el espacio objetivo son el punto ideal y el punto nadir. Para un problema de minimización, el punto ideal contiene el mejor valor alcanzable individualmente por cada objetivo:

$$z_k^{\text{ideal}} = \min_{x \in X} f_k(x),\ k = 1, \dots, q.$$

Este vector representa una referencia teórica, pero normalmente no corresponde a una solución factible, ya que los óptimos individuales de los distintos criterios pueden obtenerse mediante decisiones diferentes. El punto nadir reúne, por su parte, los peores valores de cada objetivo dentro del conjunto Pareto-óptimo:

$$z_k^{\text{nadir}} = \max_{x \in X^P} f_k(x),\ k = 1, \dots, q.$$

El punto nadir suele ser más difícil de calcular porque exige conocer previamente el conjunto Pareto-óptimo, por ello en aplicaciones prácticas suele optarse por no calcularlo, si no estimarlo mediante soluciones extremas o valores de referencia. Ambos puntos permiten caracterizar el rango relevante de los objetivos y pueden utilizarse para su normalización [5].

La optimización multiobjetivo no conduce necesariamente a una única respuesta o resultado óptimo. Esto varía en función del momento en el que se incorporan las preferencias del decisor. De esta manera los métodos de resolución pueden clasificarse en métodos sin información de preferencias, métodos *a priori*, métodos interactivos y métodos *a posteriori* [5].

Los métodos *a priori* incorporan las preferencias antes de resolver el problema. La suma ponderada transforma el vector de objetivos en una única función escalar [20]:

$$\min_{x \in X} F(x) = \sum_{k=1}^{q} w_k \tilde{f}_k(x),$$

donde $\tilde{f}_k(x)$es el objetivo normalizado y los pesos satisfacen habitualmente (aunque no es necesario, dependiendo del problema) la siguiente condición:

$$w_k \geq 0,\ \sum_{k=1}^{q} w_k = 1.$$

Los pesos w_k representan el criterio que quiere implementar el modelo o, dicho de otra forma, la importancia relativa que se le quiere asignar a la optimización de cada elemento de la función. Cuando todos son estrictamente positivos, una solución óptima de la suma ponderada es Pareto-óptima.

No obstante, la suma ponderada presenta limitaciones: diferentes combinaciones de pesos pueden conducir a una misma solución y, cuando el frente de Pareto es no convexo, pueden existir soluciones eficientes que no se obtengan mediante ninguna combinación de pesos. Esta cuestión es especialmente relevante en problemas discretos o de programación entera, como los problemas de asignación de recursos [6].

Llevando esta información a un problema real, cabe tener en cuenta que el sentido matemático puro del frente de Pareto y las soluciones óptimas no siempre es el buscado. Depende de la persona encargada gestionar unos pesos que sigan la lógica interna para buscar una solución factible que atienda a los requisitos del problema.

Ahondando en esta idea, la solución resultante debe interpretarse como una solución de compromiso asociada a una configuración concreta de aspiraciones y pesos, no como la representación completa del frente de Pareto. Además, una solución óptima para la función de logro no tiene por qué ser necesariamente la única solución eficiente respecto de los objetivos originales. Por ello, resulta recomendable estudiar cómo varía la asignación ante diferentes pesos y niveles de aspiración, así como verificar que las soluciones seleccionadas no estén dominadas por otras alternativas factibles.

Un tipo común de modelo a priori es el método secuencial, el cual emplea optimización secuencial. Este establece un orden jerárquico y estricto de prioridad entre los objetivos a cumplir [21]. Primero se optimiza el criterio de mayor prioridad y, posteriormente, se optimizan los siguientes criterios sin permitir que se deteriore el valor alcanzado por los anteriores. Es decir, primero minimiza (o maximiza) f_1, después f_2 imponiendo que f_1 conserve su valor óptimo, después f_3 manteniendo los resultados óptimos de f_1 y f_2, y así sucesivamente. Este enfoque resulta apropiado cuando existe una jerarquía sólida entre los criterios, aunque puede producir soluciones poco equilibradas si las prioridades no se establecen de manera coherente.

Por su lado, los métodos interactivos incorporan las preferencias de decisión durante el propio proceso de optimización. Introduce un cambio mayor en el proceso, ya que pasa de ser automático de principio a fin, a necesitar cierta retroalimentación del decisor. En cada iteración del método se presenta al decisor una o varias soluciones y este expresa qué criterios desea mejorar o qué niveles considera aceptables. Con ello, el procedimiento utiliza dicha información para generar nuevas alternativas. De esta manera, el decisor aprende siguiendo el proceso cuáles son los compromisos realmente alcanzables.

Los métodos *a posteriori*, por su parte, generan el frente de Pareto completo (o una aproximación representativa) antes de solicitar las preferencias. La decisión se adopta tras observar las alternativas no dominadas y los intercambios existentes entre los criterios. Estos métodos aportan mayor información, pero pueden exigir un esfuerzo computacional considerable cuando el número de objetivos o el tamaño del problema es elevado [6].

Finalmente, la agregación de objetivos heterogéneos en la mayoría de los casos exige controlar sus unidades y escalas numéricas. Si se intenta sumar magnitudes directamente expresadas en unidades diferentes (por ejemplo: horas con millones de euros), el objetivo con valores numéricos mayores puede dominar la función agregada aunque no sea el más importante para el decisor. Una normalización basada en los puntos ideal y nadir puede formularse como:

$$\tilde{f}_k(x) = \frac{f_k(x) - z_k^{\text{ideal}}}{z_k^{\text{nadir}} - z_k^{\text{ideal}}},$$

siempre que el denominador sea distinto de cero. También pueden utilizarse rangos conocidos, niveles de aspiración o valores de referencia representativos del problema:

$$\tilde{f}_k(x) = \frac{f_k(x)}{R_k},\ R_k > 0.$$

La normalización hace comparables los términos, pero no elimina la necesidad de justificar los pesos. Estos deben interpretarse como una representación de las preferencias del decisor sobre objetivos previamente escalados y no como simples factores de conversión de unidades [20]. Por ello, su elección debe complementarse con un análisis de sensibilidad que permita comprobar si pequeñas modificaciones de los pesos producen cambios sustanciales en la solución obtenida [21].

En la literatura de asignación de personal aparecen criterios como el coste, la duración, la tardanza, la estabilidad, el uso de competencias y el equilibrio de cargas. Los trabajos se diferencian tanto por los criterios seleccionados como por la forma de tratarlos: agregación ponderada, prioridades lexicográficas, metas o búsqueda de alternativas no dominadas. Por ello, la comparación de resultados exige identificar previamente qué política de decisión representa cada modelo [22].

En el modelo desarrollado de este trabajo, se adopta una estrategia *a priori* basada en programación por objetivos (*goal programming*): para cada criterio se define una meta u objetivo deseado y se introduce una variable de desviación que mide el exceso respecto a dicha meta [23]. El modelo pondera y minimiza estas desviaciones, obteniendo así una única solución de compromiso para cada combinación de pesos, aunque no construye por sí mismo el frente de Pareto completo. En el caso de emplear pesos iguales, la multiplicación de toda la función objetivo por una constante común, por ejemplo 1 o 1/n cuando existen n términos, no modifica la solución óptima, únicamente cambia el valor numérico de la función agregada [21].

2.5. PROGRAMACIÓN POR OBJETIVOS

La programación por objetivos (*goal programming*), también denominada programación por metas constituye una técnica estrechamente relacionada con el enfoque de optimización multiobjetivo explicado en el capítulo anterior. Mientras que esta última estudia el comportamiento simultáneo de varias funciones objetivo y los compromisos que conforman el frente de Pareto, la programación por objetivos incorpora las preferencias del decisor, habitualmente con un enfoque *a priori,* mediante la definición de unas metas que responden a qué resultados se consideran deseables y qué tipos de incumplimiento deben penalizarse [21].

Su fundamento responde a una lógica de satisfacción o *satisficing* [24]. Esta idea propone en lugar de optimizar cada criterio de manera aislada, buscar una solución factible que alcance, o se aproxime en la mayor medida posible, a un conjunto de metas previamente definidas. Esta característica resulta especialmente útil cuando los criterios son conflictivos y no existe una solución capaz de alcanzar simultáneamente sus respectivos valores óptimos, como suele ser habitual [23].

La programación por objetivos se apoya en niveles de aspiración definidos para cada criterio. Para cada criterio $q \in Q$, existe una meta g_q y siendo $x \in X$ el vector de variables de decisión, donde X representa el conjunto definido por las restricciones estructurales del problema, define una función $f_q(x)$. La correspondiente ecuación de meta se formula mediante una o dos variables de desviación no negativas [21]:

$$f_q(x) + d_q^- - d_q^+ = g_q, \qquad d_q^-, d_q^+ \geq 0.f$$

La desviación negativa d_q^- mide la cantidad que falta para alcanzar la meta y la desviación positiva mide el exceso. La función objetivo penaliza únicamente las desviaciones consideradas indeseables. En una meta de beneficio mínimo puede penalizarse el defecto; en un límite máximo de coste puede penalizarse el exceso; y en una meta de igualdad pueden penalizarse ambas desviaciones. Es por ello por lo que el significado de cada desviación debe definirse antes de construir la función objetivo. Penalizar simultáneamente defecto y exceso equivale a buscar proximidad absoluta a la meta; penalizar solo una de ellas establece una preferencia unidireccional [23].

Esta implementación es metodológicamente útil. Si una condición debe cumplirse siempre (como no superar la disponibilidad anual o ajustarse a un presupuesto) debe permanecer como restricción sin desviaciones. Pero si por el contrario la organización acepta un defecto o exceso cuantificable respecto de una referencia (utilizar más horas a cambio de mejorar la adecuación técnica o gastar más dinero para mejorar la calidad del resultado final) puede incorporarse mediante una desviación penalizada. Es importante tratar cada restricción con lógica dependiendo su naturalidad. Convertir todas las preferencias en restricciones rígidas aumentaría la probabilidad de inviabilidad; tratarlas todas como objetivos permitiría incumplimientos inaceptables.

Siguiendo el camino descrito que sigue la resolución de un problema por objetivos, podemos representarlo de manera gráfica en la siguiente Figura 4 [21]:

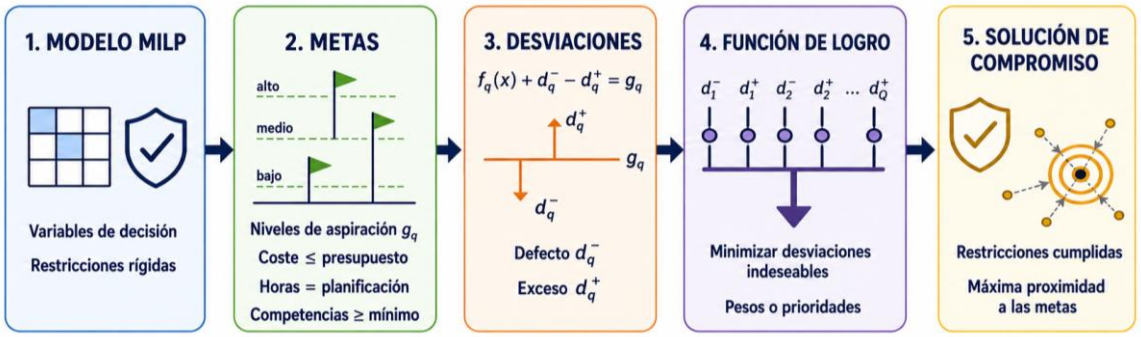


Figura 4. Resolución por programación por objetivos (elaboración propia).

La programación por objetivos ponderada permite establecer compensaciones entre las desviaciones de las diferentes metas. Esta normalización es especialmente importante cuando las desviaciones se expresan en unidades diferentes o presentan órdenes de magnitud alejados [21]. En este tipo de soluciones, un aumento de la desviación en una meta es aceptado cuando permite reducir suficientemente la desviación de otra. Esta propiedad resulta adecuada cuando la organización admite intercambios entre los criterios, pero puede ser inapropiada cuando alguna meta posee una prioridad absoluta. Su formulación general puede expresarse como:

$$min\, A\,(x, d^-, d^+) = \sum_{q=1}^{Q} \left(w_q^- \frac{d_q^-}{R_q} + w_q^+ \frac{d_q^+}{R_q} \right),$$

En problemas de gestión de recursos, la optimización multiobjetivo es un apoyo al decisor y un aspecto configurable muy relevante en el modelado de soluciones. La asignación de recursos exige valorar dimensiones que no pueden reducirse sin pérdida de información a una única unidad física. Los elementos que buscan optimizarse en el problema pueden tener consecuencias distintas. La optimización por objetivos no elimina esa pluralidad decisiva, sino que la hace explícita y ofrece mecanismos para explorar compromisos.

2.6. HEURÍSTICAS Y METAHEURÍSTICAS

Como hemos visto anteriormente, la formulación MILP define las variables de decisión, las restricciones delimitan el conjunto factible del problema y la función objetivo, resuelta por un método de optimización de uno o varios objetivos, genera una solución del problema. Sin embargo, el problema puede abordarse también mediante métodos aproximados o subóptimos, llamados métodos heurísticos. Esta distinción no modifica necesariamente la formulación matemática, sino el procedimiento utilizado para explorar el espacio de soluciones y las garantías que pueden ofrecerse sobre el resultado obtenido [8].

En el ámbito de la planificación de proyectos con recursos multicompetencia se han planteado formulaciones multiobjetivo que incorporan simultáneamente distintos criterios y recurren a métodos evolutivos para explorar los compromisos existentes entre ellos [25]. En contextos multiproyecto también se han considerado de manera diferenciada criterios temporales, como la tardanza y la duración ponderada de los proyectos, combinando formulaciones MILP con métodos metaheurísticos para abordar instancias de mayor tamaño [12]. Estos trabajos muestran que la definición de los criterios de decisión condiciona tanto la solución seleccionada como el procedimiento utilizado para obtenerla y evaluarla.

Una heurística puede encontrar una solución factible con un valor de función objetivo cercano al óptimo, e incluso coincidir con el óptimo global, pero, por sí sola, no calcula una cota que le permita demostrarlo. Esta solución se construye mediante reglas adaptadas a la estructura del problema, sin garantizar que sean óptimas. Por tanto, la expresión "solución heurística" no indica que su valor encontrado esté cercano o alejado del óptimo, sino que esa proximidad no puede demostrarse formalmente.

Existe una cantidad innumerable de tipos de heurísticas y, al mismo tiempo, no existe una única clasificación universal de las heurísticas, porque un mismo algoritmo puede pertenecer simultáneamente a varias categorías. No obstante, una forma común de dividirlas es en constructivas o de mejora según su forma de encontrar una solución, Figura 5.

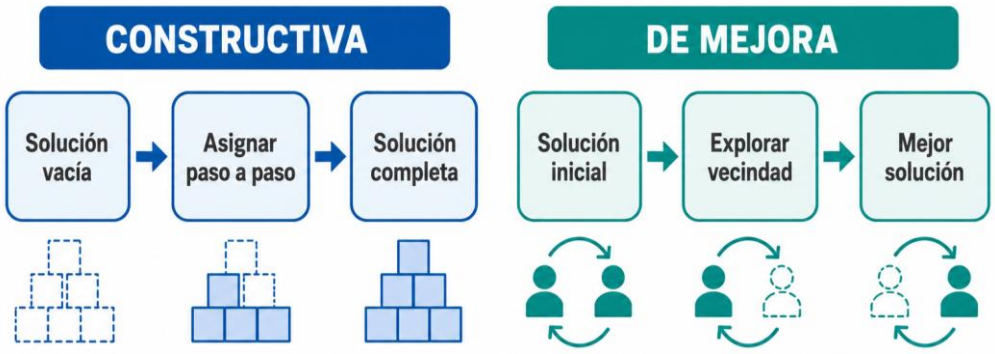


Figura 5. Diferenciación entre heurísticas (elaboración propia).

Una heurística constructiva genera la solución de forma progresiva a partir de una solución parcial inicialmente vacía. Se generan una serie de reglas ordenadas que sirven como etapas del algoritmo y en cada una de ellas se selecciona una decisión y se incorpora siempre que resulte compatible con las restricciones consideradas. El proceso continúa hasta obtener una solución completa o hasta alcanzar una situación en la que no sea posible realizar nuevas asignaciones válidas [26].

Las heurísticas constructivas utilizan frecuentemente criterios *greedy* o voraces. Estos criterios seleccionan en cada paso la alternativa que parece más favorable de acuerdo con la información disponible en ese momento, sin analizar exhaustivamente todas las consecuencias futuras. Este comportamiento *greedy* permite reducir considerablemente el tiempo de cálculo y facilita la interpretación del procedimiento. Sin embargo, una decisión localmente favorable puede comprometer asignaciones posteriores. Por tanto, las decisiones adoptadas al comienzo de la construcción pueden condicionar la calidad final o conducir a un bloqueo antes de completar una solución factible.

Este riesgo de bloqueo en la ejecución puede reducirse mediante reglas de prioridad que atiendan primero las decisiones más difíciles de satisfacer, aunque no eliminarse por completo. También pueden reservarse recursos críticos o introducir mecanismos de reparación cuando una decisión anterior impida completar la solución, aunque este remedio afectará al tiempo de ejecución del algoritmo.

En los problemas de planificación de proyectos con recursos multicompetencia se han desarrollado heurísticas basadas en reglas de prioridad, en las que la solución se construye de forma progresiva mediante una secuencia ordenada de decisiones locales. Estos métodos resultan especialmente útiles cuando se busca reducir el tiempo de cálculo, incorporar reglas específicas del problema y mantener una lógica de decisión interpretable. Como contrapartida, su comportamiento puede depender del orden de aplicación de las reglas y, al tratarse de procedimientos heurísticos, no ofrecen por sí mismos garantías de optimalidad ni de obtención de una solución factible en todas las instancias [26].

Por su parte, las heurísticas de mejora, también conocidas como búsqueda heurística o local, parten de una solución completa y exploran soluciones próximas mediante transformaciones locales. Sea x una solución factible, su vecindad $N(x)$ es el conjunto de soluciones que pueden obtenerse aplicando uno o varios movimientos previamente definidos. En el caso específico de un problema de asignación de RRHH, estos movimientos pueden consistir, por ejemplo, en reasignar horas de un trabajador, sustituir un trabajador por otro, intercambiar asignaciones, etc.

Una búsqueda local reemplaza la solución actual por una solución vecina cuando el movimiento satisface el criterio de aceptación. Bajo el convenio de minimización, pueden encontrarse óptimos locales respecto de la vecindad N (que no tiene por qué coincidir con el óptimo global del MILP) si una solución x^L cumple:

$$f(x^L) \leq f(y), \qquad \forall y \epsilon N(x^L).$$

Las heurísticas constructivas y las de mejora pueden combinarse. La primera obtiene rápidamente una solución inicial, mientras que la segunda intenta corregir decisiones tempranas y mejorar su calidad. También puede incorporarse aleatoriedad en la ordenación de los recursos o en la resolución de empates, para buscar otras soluciones siguiendo caminos diferentes. La ejecución repetida desde diferentes configuraciones iniciales recibe el nombre de estrategia multiarranque o *multi-start*, Figura 6. Cada reinicio, o intento de búsqueda, puede conducir a una región diferente del espacio de búsqueda, aumentando la probabilidad de encontrar una solución factible o de mejor calidad, aunque sin proporcionar una garantía de optimalidad. [27]



Figura 6. Heurística *multi-start* (elaboración propia).

La factibilidad debe comprobarse con respecto a las restricciones originales del MILP. Existen tres mecanismos habituales para tratar las restricciones en una heurística: generar únicamente movimientos factibles, reparar las soluciones que presentan incumplimientos o introducir penalizaciones en la función de evaluación. En este último caso, una baja penalización no convierte automáticamente una solución en factible. Antes de aceptar el resultado, debe verificarse explícitamente el cumplimiento de todas las restricciones rígidas [8].

En consecuencia, el fracaso de una ejecución heurística no demuestra que el modelo MILP no sea factible. Únicamente indica que el que procedimiento no ha encontrado una solución válida con las reglas, el orden de construcción y el presupuesto computacional empleados. La inviabilidad del problema solo puede afirmarse cuando se dispone de una demostración proporcionada por un método exacto o de un análisis matemático equivalente.

Más allá de las heurísticas específicas, existen estrategias generales de nivel superior que organizan y orientan el proceso de búsqueda con el propósito de explorar de forma más amplia el espacio de soluciones. A estos métodos se los conoce como metaheurísticos. Lo que los diferencia de una heurística estándar, es que éstos introducen mecanismos generales para recorrer el espacio de búsqueda, escapar de óptimos locales y decidir qué soluciones deben conservarse o explorarse.

Entre las metaheurísticas más conocidas se encuentran los algoritmos genéticos, la búsqueda tabú, el recocido simulado (más conocido como *simulated annealing*) y los métodos de optimización por enjambre. Aunque pueden obtener soluciones de elevada calidad en problemas complejos gracias a la combinación de intensificación y diversificación, tampoco garantizan por sí mismas la identificación del óptimo global [28].

La intensificación explota regiones prometedoras y examina con mayor profundidad el entorno de las mejores soluciones conocidas. La diversificación por su lado dirige la búsqueda hacia regiones poco exploradas para reducir el riesgo de convergencia prematura. Es importante encontrar un comportamiento equilibrado entre ambos, ya que un exceso de intensificación puede provocar que el algoritmo quede atrapado alrededor de un óptimo local, mientras que una diversificación excesiva puede impedir que se refine suficientemente ninguna región.

Las metaheurísticas pueden clasificarse comúnmente en métodos basados en trayectorias y métodos basados en poblaciones. Los primeros modifican principalmente una solución actual y generan una trayectoria a través del espacio de búsqueda, mientras que los segundos mantienen simultáneamente un conjunto de soluciones e intercambian información entre ellas. Con independencia de esta clasificación, todas las técnicas empleadas en la metaheurística requieren definir una representación de las soluciones, operadores compatibles con el problema, mecanismos de tratamiento de restricciones y criterios de parada. Además, su utilidad debe demostrarse mediante una formulación reproducible y una evaluación experimental adecuada.

En la literatura sobre el *Resource-Constrained Project Scheduling Problem* (RCPSP) se desarrollan enfoques híbridos que combinan distintas estrategias de resolución con el objetivo de aprovechar sus ventajas complementarias. Estas propuestas pueden integrar reglas de prioridad, procedimientos de búsqueda local y componentes exactos, buscando un equilibrio entre la intensificación en regiones prometedoras del espacio de soluciones y la diversificación hacia alternativas diferentes. Frente a una construcción heurística aislada, estos métodos pueden mejorar la calidad de las soluciones, aunque a costa de una mayor complejidad de diseño, parametrización y ajuste [29].

Otro ejemplo relevante de empleo de metaheurística es el TOA o *Teamwork Optimization Algorithm*, que constituye un modelo poblacional inspirado en la cooperación entre los integrantes de un equipo. Cada miembro de la población representa una solución candidata y su actualización se basa en la influencia del mejor integrante y en las interacciones con otros miembros [28].

Más allá de su desempeño en problemas monoobjetivo, la relación entre metaheurísticas y optimización multiobjetivo depende del tratamiento aplicado a los distintos criterios. Una primera posibilidad consiste en escalar los objetivos mediante una suma ponderada, una función de logro u otra técnica [20]. En este caso, la heurística o metaheurística minimiza una única función agregada [30]:

$$F(x) = \sum_{q=1}^{Q} w_q \, \tilde{f}_q(x).$$

Cada combinación de pesos conduce a una búsqueda orientada hacia una solución de compromiso concreta. Aunque el procedimiento sea computacionalmente monoobjetivo, representa las preferencias establecidas sobre varios criterios.

Una segunda posibilidad consiste en utilizar directamente relaciones de dominancia. Las metaheurísticas multiobjetivo basadas en poblaciones intentan conservar un conjunto diverso de soluciones no dominadas y aproximar diferentes regiones del frente de Pareto. Para ello pueden incorporar archivos externos, ordenación por dominancia, estimadores de densidad y mecanismos específicos de preservación de diversidad.

En este contexto, el resultado no es necesariamente una única solución, sino una aproximación del frente de Pareto. Dicha aproximación no equivale al frente exacto: contiene las soluciones no dominadas encontradas por el algoritmo, pero podrían existir otras soluciones factibles que las dominasen y que no hayan sido descubiertas.

Los algoritmos meméticos constituyen una forma de hibridación entre la búsqueda basada en poblaciones y los procedimientos de mejora local. La población favorece la exploración global, mientras que la búsqueda local refina individualmente determinadas soluciones. En problemas de *workflow scheduling*, esta hibridación puede incorporar restricciones de ejecución y prioridad, operadores específicos de reasignación y archivos destinados a conservar soluciones no dominadas [31].

La calidad de una aproximación heurística multiobjetivo puede evaluarse mediante indicadores como el hipervolumen y la distancia generacional invertida (IGD) [30]. El hipervolumen mide la región del espacio objetivo dominada por el conjunto de soluciones obtenido y delimitada por un punto de referencia. Bajo una configuración común, un valor mayor indica una mejor combinación de convergencia y cobertura. A su vez, la IGD calcula la distancia media desde un conjunto representativo del frente de referencia hasta la solución aproximada. Al contrario que el hipervolumen, un valor menor indica una mayor proximidad y cobertura.

Ambos indicadores dependen de la normalización de los objetivos y de los conjuntos o puntos de referencia utilizados. Además, solo resultan apropiados cuando el método genera un conjunto de soluciones destinadas a aproximar el frente. Si el modelo emplea una suma ponderada y devuelve una única solución, el hipervolumen y el IGD no constituyen, por sí solos, las métricas más adecuadas para evaluar esa ejecución.

Por último, las heurísticas también pueden combinarse con técnicas de programación matemática. Estas combinaciones reciben habitualmente el nombre de métodos híbridos o matheurísticas cuando utilizan explícitamente el modelo matemático o componentes de un solucionador dentro del procedimiento aproximado [32].

Una solución heurística factible puede proporcionarse como solución inicial (también llamado usualmente *warm start*) de un solucionador MILP. En un problema de optimización, esta solución establece una cota inicial y puede permitir que el algoritmo exacto descarte antes los nodos cuya cota ya sea peor que el incumbente. Dependiendo de si el algoritmo busca minimizar o maximizar un resultado, esta cota será superior o inferior respectivamente.

En sentido contrario, la información obtenida mediante programación matemática puede orientar el diseño heurístico. Una relajación lineal puede señalar qué variables presentan valores prometedores y un solucionador puede explorar de forma exacta una vecindad

restringida alrededor de una solución heurística. También pueden fijarse temporalmente algunas variables y resolver el subproblema restante mediante MILP.

Aun así, la hibridación no transmite automáticamente las garantías del método exacto a la heurística. Si el proceso finaliza después de obtener una solución aproximada, no existe certificación de optimalidad al igual que cualquier otra heurística. Esta solo se conserva cuando el solucionador exacto continúa explorando el problema hasta cerrar la diferencia dentro de la tolerancia establecida. Por tanto, los métodos híbridos permiten combinar rapidez en el procesamiento y capacidad de certificación, pero la garantía final depende del criterio de terminación aplicado [32].

2.7. AGENTES LLM PARA INTERPRETACIÓN DE RESULTADOS

Los modelos de lenguaje de gran tamaño (*Large Language Models*) más conocidos como LLM constituyen una familia de modelos generativos entrenados sobre grandes colecciones de texto y código, capaces de procesar lenguaje natural, producir representaciones estructuradas y generar código [33], [34]. En investigación operativa pueden actuar como capa lingüística tanto en la formulación de problemas como en la interpretación de resultados [35]. Esta utilización conduce a una arquitectura híbrida (Figura 7) en la que las operaciones lingüísticas se separan de las deterministas: el modelo matemático o la heurística realizan el cálculo, mientras que el LLM interpreta la evidencia obtenida y la presenta en lenguaje natural [9]. En este trabajo se adopta únicamente esta segunda función, manteniendo la formulación y la resolución fuera del componente lingüístico y conservando la decisión final bajo supervisión humana.

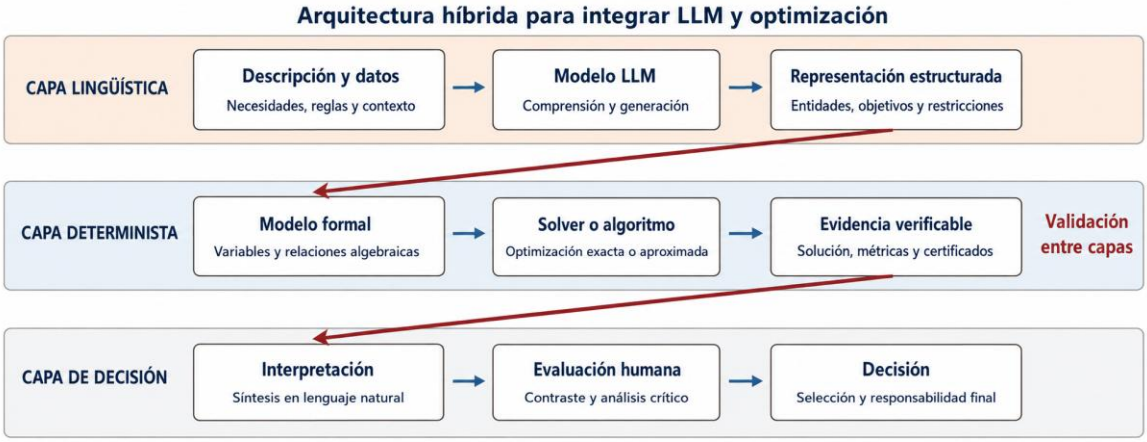


Figura 7. Separación conceptual entre generación lingüística, cálculo y ayuda a la decisión (elaboración propia).

Los LLM contemporáneos se apoyan mayoritariamente en "arquitecturas Transformer" [33], que permiten modelar relaciones entre elementos de una secuencia y aprovechar grandes volúmenes de datos durante la contextualización del agente. Una propiedad especialmente relevante para su uso aplicado es el aprendizaje en contexto: instrucciones, definiciones y ejemplos proporcionados en la consulta pueden orientar la respuesta sin modificar los parámetros internos del modelo [34]. Esta capacidad sustenta la ingeniería de *prompts*, utilizada para fijar terminología, formatos de salida y reglas de actuación, aunque la naturaleza probabilística de la generación impide garantizar formalmente la corrección de la respuesta [36].

Debido a esta dependencia del contexto, el conocimiento empleado por un LLM es paramétrico, es decir, se adquiere en la fase de preentrenamiento o se proporciona en el momento de la consulta.[37] La generación aumentada mediante recuperación (*Retrieval-Augmented Generation*, RAG) complementa este contexto con información localizada en fuentes externas, permitiendo que la respuesta se apoye en documentación actualizable y controlada [38]. En aplicaciones técnicas, esta estrategia facilita limitar la interpretación a datos, formulaciones y resultados previamente validados [39].

Una vez validado el modelo, el LLM puede actuar como interfaz entre la evidencia técnica y el usuario explicando variables, restricciones, métricas y diferencias entre alternativas [9]. Para poder emplear esta función de manera rigurosa, se le exige al agente LLM, a la hora de interpretar resultados, distinguir si se ha realizado un cambio en el proceso de resolución del problema o en los datos iniciales del mismo. El LLM puede identificar el cambio, preparar los nuevos parámetros o activar el procedimiento correspondiente, pero no debe presentar como hecho una solución que el algoritmo no haya producido. Esta distinción mantiene la trazabilidad entre hipótesis, instancia, ejecución y conclusión [40].

Del mismo modo, la explicación de casos de infactibilidad constituye otro caso relevante. Un procedimiento exacto puede detectar incompatibilidades mediante certificados, restricciones conflictivas o subconjuntos irreducibles. A partir de esta evidencia, el LLM puede relacionar los elementos matemáticos con conceptos de negocio y expresarlos en un lenguaje más accesible. La validez del diagnóstico sigue procediendo del componente matemático, mientras que el modelo lingüístico aporta capacidad de interpretación y comunicación [37].

Dado que la interpretación lingüística puede alejarse de la evidencia que la sustenta, resulta necesario que el apoyo a la decisión sea auditable, es decir, las salidas deben conservar una relación explícita con la fuente. Los formatos estructurados, los identificadores de escenario, las unidades y la separación entre hechos reducen el riesgo de que una síntesis narrativa oculte incertidumbres. El uso de recuperación RAG sobre un marco documental controlado y previamente validado puede reforzar este vínculo, aunque la presencia de una fuente no garantiza que el modelo la interprete correctamente [39].

Sobre estas capacidades puede construirse un sistema agéntico, en el que bajo un LLM que genere una respuesta condicionada por el contexto recibido, un agente añade un bucle de control que mantiene el estado de la tarea, selecciona acciones, utiliza herramientas, observa sus resultados y decide el siguiente paso. Este patrón de alternancia entre razonamiento y actuación permite que la información obtenida externamente modifique o complete el desarrollo de la tarea [41].

En el ámbito del apoyo a la decisión, se han propuesto arquitecturas híbridas en las que el modelo lingüístico no sustituye al componente de optimización, sino que actúa como interfaz para interpretar preferencias expresadas en lenguaje natural y trasladarlas a un modelo formal. En aplicaciones de planificación de reuniones, este enfoque se ha utilizado para obtener de manera iterativa preferencias contextuales del usuario y combinarlas con programación por restricciones, de modo que el sistema pueda generar alternativas compatibles con dichas preferencias y refinarlas a medida que avanza la interacción [42].

Una línea más amplia plantea el uso de los LLM como asistentes durante distintas fases del proceso de optimización. El concepto de *Decision Optimization CoPilot* propone que el componente lingüístico acompañe al decisor desde la descripción inicial del problema en lenguaje natural hasta su formalización como modelo de optimización, incluyendo la identificación de variables, restricciones y objetivos, así como el apoyo durante la resolución y la interpretación posterior de los resultados. Este planteamiento no presenta al LLM como sustituto del optimizador, sino como una capa de interacción entre el usuario y las herramientas matemáticas de decisión [9].

En cuanto a la implementación de agentes conversacionales mediante Microsoft Copilot Studio, se han estudiado arquitecturas orientadas a la automatización de procesos empresariales y al cumplimiento normativo [43]. Estos trabajos consideran aspectos como la descomposición de tareas, el mantenimiento del contexto, la adaptación al rol del usuario y la transparencia de las respuestas, junto con mecanismos de evaluación orientados a la precisión, la eficiencia y la confianza en la interacción entre usuario y agente. También se ha abordado la gestión de estos agentes más allá de su creación inicial, considerando su ciclo de vida como un proceso continuo de diseño, despliegue, supervisión y actualización [44]. En este contexto, Copilot Studio se utiliza como entorno de orquestación del agente y *Dataverse* como soporte para la gestión estructurada de datos, incorporando además aspectos de monitorización, gobierno, documentación y revisión del comportamiento del sistema a lo largo del tiempo.

Por tanto, el término apoyo a la decisión es más preciso que considerar al LLM como decisor. La responsabilidad final permanece en la persona, que debe valorar aspectos no representados por el modelo, como preferencias de los trabajadores, continuidad con clientes, riesgos organizativos o información confidencial. El agente acerca la salida técnica a lectura empresarial, pero su respuesta solo es fiable si se encuentra vinculada a una fuente estructurada, a validaciones deterministas y a reglas explícitas de actuación. En la Figura 8 siguiente vemos la utilización de un sistema agéntico como apoyo a la toma de decisiones:



Figura 8. Utilidad de un agente LLM para el apoyo a la decisión post-resultados (elaboración propia).

3. DESARROLLO DE LA SITUACIÓN EMPRESARIAL EVALUADA

3.1. DESCRIPCIÓN DE LA SITUACIÓN EMPRESARIAL

El problema abordado en este trabajo tiene como base un problema de asignación de recursos del tipo MSRCPSP (definido en el subcapítulo 2.1) y tiene relación con estas líneas de investigación la existencia de una plantilla heterogénea y limitada que debe distribuirse entre varios proyectos, así como la necesidad de considerar las competencias de los trabajadores. Con el fin de abordar el problema en estudio, se ha diseñado un escenario empresarial plausible, es decir, una configuración concreta de trabajadores, proyectos, roles y competencias que representa una situación posible de una empresa de servicios profesionales. El trabajo en el que se centra el proyecto es la asignación de los trabajadores de su plantilla y la labor que debe ejecutar cada uno para completar unos determinados proyectos en un marco temporal acotado.

Sin embargo, se sitúa en un nivel de decisión diferente al del MSRCPSP ya que no se programan actividades individuales ni se establecen precedencias, fechas de inicio o finalización. La decisión se plantea de forma agregada sobre un horizonte anual, determinando la participación de los trabajadores en los proyectos, las horas aportadas y su contribución a los requisitos de competencias, roles y capacidad. En este caso, se opta por un horizonte temporal de un año que se representa de forma agregada en todo el escenario, es decir, se entiende que todos los proyectos deben ejecutarse dentro de ese periodo, la disponibilidad de cada trabajador también se considera anual en su conjunto y, coherentemente, las horas de trabajo asignadas también se reparten anualmente.

La organización debe distribuir la capacidad de trabajo disponible entre contratos que presentan volúmenes de trabajo, necesidades técnicas, costes laborales imputados y exigencias funcionales diferentes. Para ello, se definen grupos funcionales a los que llamaremos roles, que definen la categoría a la que pertenece cada trabajador. Sin embargo, los trabajadores de un mismo rol no son necesariamente intercambiables, ya que cada persona tiene características individuales como sus obligaciones con determinados proyectos o sus determinadas competencias (o *skills*) propias.

En este contexto, el objetivo es determinar qué trabajadores asignar a cada proyecto y durante cuánto tiempo, teniendo en cuenta sus competencias para asegurar una asignación eficiente. Asimismo, esta asignación debe satisfacer la necesidad de esfuerzo de los proyectos, las cuotas mínimas de determinados roles, los requisitos de competencias y las dedicaciones previamente comprometidas que comprometan a un trabajador con un proyecto, ya sea como un mínimo de horas o como una cantidad exacta a cumplir. Al mismo tiempo, ningún trabajador puede superar su disponibilidad anual ni intervenir en más proyectos de los permitidos por las reglas organizativas.

No toda asignación que cumple estas condiciones resulta igualmente conveniente para la organización. Entre las múltiples asignaciones factibles, interesa identificar aquellas que mejor concilian criterios como el coste total, la calidad de la cobertura de competencias o el equilibrio de la carga entre trabajadores. Para ello, es necesario definir una meritocracia que permita valorar y comparar distintas alternativas factibles, de forma que la decisión no se limite a encontrar una asignación válida, sino la más conveniente según los criterios establecidos por la organización. Así aparece la función objetivo, una forma de expresar cuantitativamente los puntos a valorar en la solución y cuya formulación concreta (y el

tratamiento de los distintos criterios en conflicto), se desarrolla más adelante en el subcapítulo 3.3.

En este escenario, a priori la dificultad se traduce en situaciones concretas: perfiles con competencias no equivalentes, roles que actúan como cuello de botella, y compromisos previos que reducen el margen de maniobra para el resto de la cartera. Por ello, el problema exige considerar de forma conjunta la capacidad, el coste, la composición de los equipos y la adecuación técnica de las asignaciones.

La salida útil para el decisor no se limita a una matriz de horas y asignaciones. Además de identificar si se ha obtenido una asignación válida, el sistema debe mostrar el valor de las métricas relevantes como el coste total o la cobertura de competencias, la distribución de carga por trabajador y proyecto, los desajustes de competencias, las restricciones activas y las tensiones que explican el resultado. Esta necesidad justifica que la solución técnica combine métodos de optimización, validación automática e interpretación empresarial mediante un sistema agéntico basado en un modelo de lenguaje.

3.2. MODELADO DEL RECORRIDO DEL PROBLEMA

El diseño comienza por definir una relación explícita entre las decisiones (qué trabajadores participan en cada proyecto, cuántas horas dedican y cómo se cubren las competencias exigidas) y los elementos computacionales necesarios para responderlas. A cada elemento involucrado en estas decisiones se le asigna una variable: la participación de un trabajador requiere una variable binaria, las horas asignadas una variable continua (por trabajador y proyecto) y la cobertura de competencias necesita una desagregación adicional por competencia. Otros aspectos del problema a tratar como disponibilidades, costes, roles y requisitos se mantienen como parámetros porque describen la instancia y no son modificados por el algoritmo.

A continuación, se clasifica cada regla según la función que ha de satisfacer. Las restricciones de cobertura obligan a satisfacer la demanda de los proyectos; las de capacidad limitan el uso de la plantilla; las de composición garantizan cuotas de rol y competencias; las de participación enlazan decisiones binarias y continuas; y las de compromiso fijan dedicaciones mínimas o exactas. Esta clasificación evita incorporar una misma condición de dos formas incompatibles y permite comprobar posteriormente cada familia por separado.

Los tres métodos se diseñan alrededor de un mismo núcleo de factibilidad: una asignación aceptable debe cumplir los mismos requisitos independientemente del modelo de resolución empleado, ya que esta definición no depende del objetivo elegido ni del procedimiento de búsqueda. Bajo este núcleo, las variables de participación actúan como enlace entre las decisiones binarias y continuas del modelo mediante restricciones que acotan las horas asignadas en función de dicha participación, habilitando o impidiendo el resto de las restricciones. A partir de este enlace se aplican el límite de proyectos y los compromisos mínimos o exactos. La coherencia entre horas, participación y competencias se comprueba también en la validación posterior.

La cobertura técnica se calcula a partir de las horas destinadas a cada competencia y del factor de compatibilidad. De esta forma, el modelo diferencia entre volumen de trabajo total y contribución efectiva. El cumplimiento de la cobertura real exigido evita que se atribuya a una competencia una contribución superior a su necesidad, mientras que la relación entre las horas dedicadas a competencias específicas y las horas totales impide contabilizar dos veces una misma dedicación.

Tras definir todas las estructuras empleadas, la construcción se realiza de forma incremental. Primero se cargan los conjuntos y parámetros de la instancia y se comprueba que sus dimensiones y estructuras de datos sean coherentes entre sí. Después se crean las variables y las restricciones de factibilidad. Sobre este núcleo se incorpora la función objetivo monoobjetivo y multiobjetivo junto a las variables de desviación que esta requiere. La heurística reutiliza la misma información, manteniendo el estado de la disponibilidad y cobertura pendiente en cada iteración de la construcción. Esta secuencia facilita localizar el punto de fallo del problema. Además, la separación entre construcción, resolución y extracción evita tratar como resultado válido una estructura incompleta o un estado del *solver* que no contiene valores utilizables.

Por último, se unifica y explica el convenio de estados y unidades. Las asignaciones y disponibilidades se mantienen en horas anuales, el coste en euros anuales y los factores de compatibilidad como valores adimensionales. Los métodos informan de si existe solución, del estado de terminación y del tiempo registrado. Esta normalización es necesaria para consolidar posteriormente las ejecuciones sin realizar conversiones específicas para cada algoritmo.

Una vez construidos los modelos de resolución, se integran en un flujo que concatena varias etapas: un generador construye la instancia del problema, es decir, un escenario concreto con datos específicos de trabajadores, proyectos y restricciones, a partir de una configuración preestablecida. Esta instancia se entrega a los tres métodos de resolución que, tras ejecutar su intento de resolución, devuelven soluciones que se someten posteriormente a un validador común y se transforman en una estructura homogénea de resultados. Finalmente, una solución seleccionada puede ser interpretada por el agente LLM como apoyo a la toma de decisión.

El flujo completo se resume en la Figura 9 siguiente. La configuración solo interviene en la generación; una vez escrito, cada escenario es independiente. Los tres resolutores consumen exactamente el mismo JSON y aplican las mismas comprobaciones comunes antes y después de construir una solución.

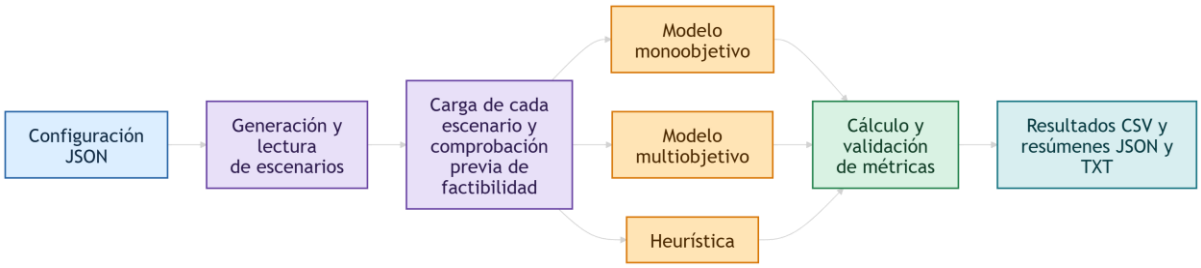


Figura 9. Flujo computacional.

Este proceso se diseña de forma unidireccional: la configuración produce una instancia, la instancia alimenta a los métodos, los métodos producen candidatos, el validador clasifica esos candidatos, y la consolidación calcula las métricas comparables. A esto, se le agrega un agente LLM entrenado para el experimento que se sitúa al final y recibe únicamente información producida en las fases anteriores, lo que impide que una explicación lingüística modifique retrospectivamente los datos, el estado o el valor de las variables de decisión.

Cada etapa genera un elemento (una configuración, un fichero de escenario, un resultado, un registro de validación, un informe…) identificable, cuya conservación permite replicar una ejecución concreta y localizar en qué etapa aparece una discrepancia entre lo esperado y lo obtenido. La reproducibilidad, definida como la capacidad de repetir una ejecución y obtener los mismos resultados a partir de la misma configuración, se plantea así como una propiedad íntegra del proceso, y sienta la base sobre la que se apoya la evaluación comparativa descrita

en el apartado 3.5. Estas etapas generan un ciclo de experimento que podemos visualizar de manera esquemática en la siguiente Figura 10:



Figura 10. Ciclo experimental completo.

1. Definición del experimento. Se fijan la composición general de la plantilla, los rangos de los proyectos, las competencias, los compromisos de asignación y las tipologías que se desean estudiar. Esta fase establece qué elementos permanecerán comunes y qué parámetros se modificarán de manera controlada.

2. Generación de la instancia. El generador materializa los parámetros en un archivo JSON autogenerado, mediante un proceso pseudoaleatorio controlado (por ejemplo, con una semilla fija) que instancia los valores concretos dentro de los rangos definidos en la fase anterior. Una vez generado el archivo, la instancia de cada escenario deja de depender de dicho proceso y se convierte en una entrada fija que puede suministrarse a los tres métodos.

3. Comprobación previa. Antes de resolver las instancias mediante los modelos, se revisan incompatibilidades generadas en el proceso de su construcción, por ejemplo, que la demanda total de un proyecto supere la disponibilidad conjunta de los trabajadores. El objetivo es distinguir una incongruencia estructural evidente de un fallo posterior del algoritmo, registrando la causa del error mediante un estado específico.

4. Resolución. La misma instancia se procesa con el modelo monoobjetivo, el multiobjetivo y la heurística. Cada método conserva su propia lógica, pero utiliza los mismos conjuntos, parámetros y reglas de factibilidad para que las diferencias observadas se deban al procedimiento y no a cambios en el problema.

5. Validación independiente. La asignación devuelta se vuelve a comprobar fuera de la lógica que la produjo. Se recalculan coberturas, disponibilidad, participación y compromisos, y se descarta cualquier candidato que no supere estas verificaciones.

6. Cálculo y consolidación de métricas. Las salidas válidas se transforman a un esquema común. Se registran el estado, el tiempo, las horas, el coste y las desviaciones pertinentes, preservando los valores ausentes cuando una métrica no puede calcularse por no existir solución.

7. Interpretación para el decisor. Una solución seleccionada se entrega al agente junto con su contexto y su validación. El agente organiza la evidencia en un diagnóstico, pero cualquier

modificación sugerida debe convertirse en un nuevo escenario y recorrer de nuevo las etapas anteriores.

3.3. FORMULACIÓN MATEMÁTICA DEL PROBLEMA

Una vez descrito el escenario empresarial, es necesario traducir sus requisitos a una formulación matemática que represente con precisión las decisiones de asignación. Para ello, se definen los conjuntos, parámetros y variables del problema, así como la función objetivo y las restricciones asociadas a la capacidad, la demanda, los roles y las competencias.

Se trata de configurar un sistema de asignación de T trabajadores para una cantidad de P proyectos en una compañía, de una manera en la que se cubran las necesidades de trabajo de cada proyecto, respetando la disponibilidad de los trabajadores y el coste económico mínimo necesario. Ambos conjuntos se representan en la posterior Figura 11.

Para este modelo, cada trabajador se considerará parte de un subgrupo diferenciado por su rol, el cual presenta unas características similares entre sus integrantes. Así, los trabajadores se considerarán o bien técnicos, directivos o financieros.

Cada trabajador **t** ∈ (1, ..., T) tiene las siguientes características:

r_t [rol] = rol ϵ K al que pertenece el trabajador t. K={TEC, DIR, FIN}
s_t [skills] = habilidades técnicas que posee el trabajador t.
h_t [horas] = horas anuales de trabajo disponibles del trabajador t.
c_t [euros/hora] = coste económico que implica el trabajo/hora del trabajador t.

Cada proyecto **p** ∈ (1, ..., P) tiene las siguientes características:

S_p [skills] = habilidades técnicas específicas que requiere el proyecto p.
H_p [horas] = horas mínimas de trabajo total que requiere el proyecto p.
E_p [euros] = coste mínimo que requiere el proyecto p.
$H_{p,s}$ [horas] = horas mínimas de cada habilidad técnica ϵ S_p que requiere p.

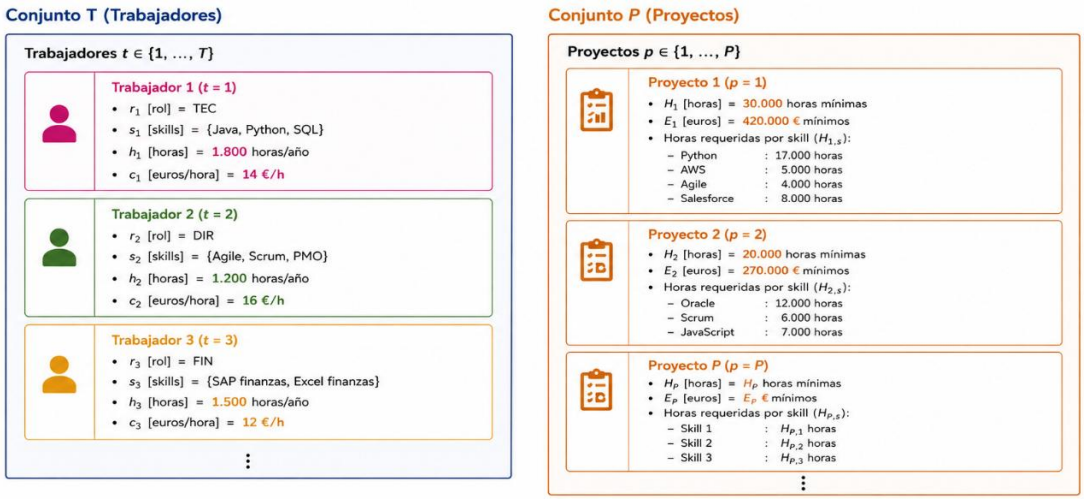


Figura 11. Estructura de conjuntos de trabajadores y proyectos.

Por políticas internas de la empresa, contamos con varias restricciones que mantengan una cohesión de trabajo y una planificación de equipos inteligente:

1. Se requiere que un porcentaje mínimo de las horas de trabajo del proyecto p sea realizado por trabajadores t pertenecientes al rol $r_t = k \in K$ para mantener una cohesión del trabajo total, $\omega_{k,p}$.
2. Existe un número máximo de implicaciones en proyectos para todos los trabajadores de la plantilla, R.
3. A ciertos trabajadores de la empresa elegidos pertenecientes a los subconjuntos (A_{min} y A_{exa}), se les exigirá cumplir una cierta cantidad mínima ($\theta_{t,p}^{min}$) o exacta ($\theta_{t,p}^{exa}$) de horas relativas a su disponibilidad en implicaciones a proyectos específicos, representado en la Figura 12. Para ello, elegimos unos porcentajes de disponibilidad exigible que variarán en partes de 25%, es decir: $\theta_{t,p}^{min}$ =[0.25, 0.50, 0.75] y $\theta_{t,p}^{exa}$ = [0.25, 0.50, 0.75, 1.00].

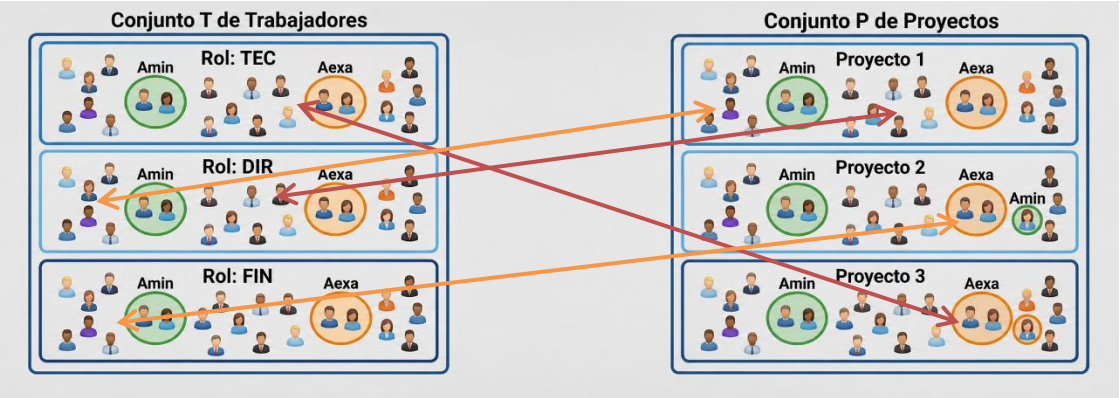


Figura 12. Conjuntos (T-P): A_min y A_exa

Para resolver las necesidades propias de cada proyecto, se cuantifican un número de horas necesarias por habilidades técnicas o skills que deben cumplir los trabajadores responsables de estos proyectos. Para evaluar si una competencias requerida "*i*" puede ser cubierta por un trabajador que posea la skill "*j*", realizaremos una matriz de compatibilidades, donde le daremos a cada par de skills un factor de semejanza o compatibilidad ($\mu_{i,j}$) cuyo valor estará comprendido entre 0 y 1. Un ejemplo se visualiza en la Tabla 1, donde vemos los factores de compatibilidad en una matriz simplificada.

Tabla 1. Ejemplo de factor de compatibilidad de skills

	SAP	EXCEL	SALESFORCE	...
SAP	1	0.2	0.6	
EXCEL	0.2	1	0.4	
SALESFORCE	0.6	0.4	1	
...				

De esta manera, las horas necesarias para el proyecto 1 de SAP, $H_{1,SAP}$, se considerará que pueden ser cubiertas por $H_{1,SAP}/\mu_{Skill_x,SAP}$ horas de la skill x, siempre y cuando ambas competencias tengan una compatibilidad no nula. Dándole otra visión, el factor de compatibilidad entre skills nos asegura que un trabajador preparado en la skill "i", puede realizar la skill j con una productividad horaria del $\mu_{i,j}$ %.

En resumen: el sistema recibe una plantilla y una cartera de proyectos y devuelve una matriz de horas $Y_{t,p}$. Para cada pareja trabajador-proyecto también indica si existe participación y qué parte de las horas se dedica a cada skill. Los datos y parámetros definidos

y empleados en la construcción del problema se resumen en estas dos tablas siguientes, tanto los conjuntos de elementos como los parámetros empleados.

Para definir y diferenciar correctamente los elementos del escenario, realizamos tres tablas que representen 1.Los conjuntos del escenario: definen los elementos que forman el problema, como trabajadores, proyectos, roles o competencias (Tabla 2). 2. Las variables: representan las decisiones que debe tomar el modelo, como qué trabajador asignar, a qué proyecto y cuántas horas. (Tabla 3) y 3. Los parámetros: son datos conocidos y fijos del escenario que condicionan las decisiones, como disponibilidades, costes, demandas o compatibilidades (Tabla 4).

Tabla 2. Conjuntos del escenario.

Símbolo	Definición
T	Conjunto de trabajadores.
P	Conjunto de proyectos.
K	Conjunto de roles: {TEC, DIR, FIN}
S	Catálogo global de skills
S_t	Skills poseídas por el trabajador t
S_p	Skills requeridas por el proyecto p
T_k	Trabajadores cuyo rol es k
A^{min}	Parejas trabajador-proyecto con dedicación mínima obligatoria
A^{exa}	Parejas trabajador-proyecto con dedicación exacta obligatoria

Tabla 3. Variables del escenario.

Variable	Tipo	Unidad	Descripción
$Y_{t,p}$	Decisión	h	Horas asignadas al trabajador t en el proyecto p
$Z_{t,p}$	Decisión	—	Indica si el trabajador t participa en el proyecto p
$X_{t,p,s}$	Decisión	h	Horas de t en p destinadas a la competencia s
H_p^{real}	Auxiliar	h	Horas totales asignadas al proyecto p
C_p^{real}	Auxiliar	€	Coste laboral total imputado al proyecto p
$S_{p,s}^{efec}$	Auxiliar	h	Cobertura efectiva de la competencia s en el proyecto p. Mide horas reales * factor de productividad

Las decisiones de asignación se representan mediante tres familias de variables. La variable continua $Y_{t,p}$ determina el número total de horas que un trabajador le dedica un proyecto. $X_{t,p,s}$ desagrega una parte de esa dedicación para indicar las horas destinadas específicamente a cubrir una competencia únicamente. En tercer lugar, la variable binaria $Z_{t,p}$ identifica si existe participación del trabajador t en el proyecto p. En conjunto, estas variables permiten representar tanto la composición de los equipos como el volumen de trabajo asignado y la distribución de las horas entre las competencias requeridas.

Además de estas variables de decisión, se introducen tres variables auxiliares que permiten expresar de manera más compacta las principales magnitudes obtenidas a partir de la asignación. En concreto:

El número total de horas asignadas a un proyecto p: $H_p^{real} = \sum_t Y_{t,p}\ ,\ \forall p \in P$

El coste laboral total imputado a un proyecto p: $C_p^{real} = \sum_t c_t \cdot Y_{t,p}\ ,\ \forall p \in P$

Las horas de cobertura efectiva alcanzadas para la competencia s: $S_{p,s}^{efec} = \sum_t q_{t,s} \cdot X_{t,p,s}\ ,$ $\forall p, s \in PS$.

Tabla 4. Resumen de parámetros del escenario.

Parámetro	Unidad	Descripción
h_t	h/año	Disponibilidad máxima del trabajador t.
c_t	€/h	Coste horario del trabajador t.
r_t	—	Rol del trabajador t.
H_p	h	Horas mínimas del proyecto p.
E_p	€	Coste laboral mínimo que debe materializarse en p.
$H_{p,s}$	h efectivas	Requisito mínimo de la skill s en el proyecto p.
$\omega_{k,p}$	[0,1]	Fracción mínima de H_p que debe cubrir el rol k.
R	proyectos	Máximo de proyectos por trabajador en una instancia
$\theta_{t,p}^{min}$	[0,1]	Fracción mínima de h_t que debe asignarse a p.
$\theta_{t,p}^{exa}$	[0,1]	Fracción exacta de h_t que debe asignarse a p.
$\mu_{i,j}$	[0,1]	Compatibilidad de la skill poseída i con la requerida j.
$q_{t,s}$	[0,1]	Compatibilidad máxima entre las competencias del trabajador t con la skill requerida s. $\max_{j \in S_t}\{\mu_{j,s}\}$
h^{min}	h	Dedicación mínima de horas asociada a una participación ($Z_{t,p} = 1$), se fija en 1 h.
λ, μ, ρ	—	Pesos de desviaciones de horas, coste y skills.

3.4. DISEÑO DE LOS MÉTODOS DE RESOLUCIÓN

3.4.1. MODELO DE OPTIMIZACIÓN DE UN OBJETIVO

Este primer enfoque consiste en formular el problema como un modelo de programación lineal entera mixta, ya que combina variables continuas de horas asignadas con variables binarias de participación trabajador-proyecto. Se busca la optimización de una variable a elección para visualizar el mínimo absoluto posible para un problema y adoptarlo como referencia para la comparativa de resultados obtenidos.

VARIABLES DE DECISIÓN

$Y_{t,p} \geq 0$, número de horas asignadas al trabajador t para el proyecto p.

$X_{t,p,s} \geq 0$, número de horas dedicadas a la skill s por el trabajador t para el proyecto p.

$Z_{t,p}$ { 1, si el trabajador T participa en el proyecto P.
0, si no.

FUNCIÓN OBJETIVO

En nuestro caso, decidimos emplear un modelo de optimización en el que queramos minimizar el número total de horas reales asignadas, garantizando la cobertura efectiva:

$$\boldsymbol{min\ F} = \min\left\{\sum_{t=1}^{T}\sum_{p=1}^{P} Y_{t,p}\right\}$$

RESTRICCIONES

Cobertura mínima de horas por proyecto: $\sum_t Y_{t,p} \geq H_p\ ,\ \forall p \in P$
$H_p^{real} \geq H_p\ ,\ \forall p \in P$

Cada proyecto debe recibir al menos las horas mínimas requeridas.

Cobertura mínima de costes por proyecto: $\sum_t Y_{t,p} \cdot c_t \geq E_p,\ \forall p \in P$
$C_p^{real} \geq E_p\ ,\ \forall p \in P$

El coste asignado a cada proyecto debe alcanzar su mínimo coste laboral asociado.

Cobertura min de skills por proyecto: $\sum_t X_{t,p,s} \cdot\ q_{t,s} = H_{p,s},\ \forall p \in P,\ \forall s \in S_p$
$S_{p,s}^{efec} = H_{p,s}, \forall p \in P,\ \forall s \in S_p$

La suma de las contribuciones efectivas debe igualar el requisito de cada skill.

Horas máximas de cada trabajador: $\sum_p Y_{t,p} \leq h_t\ ,\ \forall t \in T$

Ningún trabajador puede superar su disponibilidad horaria de trabajo anual.

Participación mínima de rol por proyecto: $\sum_{t,r_t=k} Y_{t,p} \geq H_p \cdot \omega_{k,p}, \forall k \in K\ , \forall p \in P$

Ha de cumplirse que unas horas mínimas de cada proyecto sea realizada por trabajadores de un rol determinado.

Proyectos máximos de cada trabajador: $\sum_p Z_{t,p} \leq R\ ,\ \forall t \in T$

Cada trabajador puede participar como máximo en R proyectos distintos.

Restricciones de horas mínimas: $Y_{t,p} \geq \theta_{t,p}^{min} \cdot h_t, \forall\ t,p \in A_{min}$

El subconjunto de trabajador-proyecto elegido debe cumplir la dedicación mínima horaria que se les exige.

Restricciones de horas exactas: $Y_{t,p} = \theta_{t,p}^{exa} \cdot h_t, \forall\ t,p \in A_{exa}$

El subconjunto de trabajador-proyecto elegido debe cumplir la dedicación exacta horaria que se les exige.

Restricciones de relación: $h^{min} \cdot\ Z_{t,p} \leq Y_{t,p} \leq Z_{t,p} \cdot h_t\ , \forall t \in T, \forall p \in P$

La cota superior impide asignar horas cuando $Z_{t,p} = 0$; la cota inferior garantiza al menos h^{min} horas cuando $Z_{t,p} = 1$.

$$\sum_{s \epsilon S_p} X_{t,p,s} \leq Y_{t,p}, \quad \forall t \epsilon T, \forall p \epsilon P$$

La suma horaria que un trabajador dedica a cubrir las diferentes habilidades requeridas por un proyecto no puede superar el total de horas que se le asignan a dicho proyecto.

Restricciones de dominio:

$$X_{t,p,s} \geq 0, \forall t \epsilon T, \forall p \epsilon P, \forall s \epsilon S_p$$
$$Y_{t,p} \geq 0, \forall t \epsilon T, \forall p \epsilon P$$
$$Z_{t,p} = \{0,1\}, \forall t \epsilon T, \forall p \epsilon P$$

3.4.2. MODELO HEURÍSTICO

La heurística se diseña como un procedimiento constructivo alineado con las mismas reglas de los modelos matemáticos. No intenta resolver el problema de manera óptima sino construir una asignación factible desde cero manteniendo el estado y valor de las variables pendiente de trabajadores y proyectos en todo momento durante su ejecución. Esta elección favorece una ejecución rápida y permite explicar por qué se selecciona cada candidato.

En cada paso se filtran los trabajadores que aún pueden participar sin superar su disponibilidad ni el límite de proyectos. Sobre ese conjunto se aplican criterios vinculados a la necesidad pendiente: compatibilidad técnica para cubrir skills, pertenencia al rol cuando existe una cuota y relación entre coste y horas para completar los requisitos generales. La actualización inmediata del estado evita reutilizar una capacidad que ya ha sido asignada.

La heurística prioriza la construcción de una solución válida rápidamente frente a la mejora posterior. No incorpora una fase exhaustiva de retroceso ni una búsqueda local que deshaga grandes bloques de asignaciones. Como consecuencia, una decisión temprana puede impedir completar una instancia que sí es factible. Esta limitación es deliberada y constituye una de las diferencias que la validación experimental debe medir.

Con esto, podemos resumir afirmando que este modelo heurístico tiene como finalidad obtener soluciones factibles subóptimas en un tiempo reducido, especialmente para escenarios de mayor tamaño en los que el modelo MILP pueda requerir tiempos de resolución elevados. Su contrapunto es que no garantiza una solución óptima global ni el encontrar una solución a un problema donde sí es viable encontrarla pero puede construir asignaciones razonables respetando las principales restricciones del problema. Se basa en una construcción secuencial de la solución, teniendo en cuenta las reglas que se aplican en el modelo.

REGLAS APLICADAS

Las reglas aplicadas para la generación de soluciones rápidas siguen el siguiente orden:

a) Partiendo de la plantilla total de trabajadores, se asignan primero las restricciones exactas y mínimas, garantizando las dedicaciones obligatorias (conjuntos A_{min} y A_{exa}).
b) Actualizar la disponibilidad horaria restante y el número de proyectos en los que participa cada trabajador.
c) Cubrir las necesidades mínimas de rol y competencias por proyecto.

d) Cubrir las horas restantes de cada proyecto en función a c_t y el ratio coste pendiente hasta alcanzar el mínimo/horas restantes por cumplir de cada proyecto.
e) Hacer un segundo barrido, comprobando la cobertura económica de cada proyecto, asignar trabajadores de forma secuencial.

CONSTRUCCIÓN SECUENCIAL Y CONDICIONES DE TERMINACIÓN

El modelo heurístico parte de una solución vacía:

$Y_{t,p} = 0, \forall t \in T,\ \forall p \in P$
$Z_{t,p} = 0, \forall t \in T,\ \forall p \in P$

También se inicializa la disponibilidad restante de cada trabajador:
$h_{t_res} = h_t, \forall t \in T.$

Para esta situación, aplica todas las reglas ordenadamente para intentar construir una solución viable en mucho menor tiempo. Además, para reducir la dependencia de un único orden de construcción, nos aprovechamos de la naturaleza *multi-start* de la heurística, en la que se realizan varias ejecuciones de búsqueda o reinicios (lo explicaremos de manera más detallada en el subapartado 4.5.3). Los intentos pueden alternan la prioridad definida de las reglas, , como vemos en la siguiente Figura 13, mientras que la aleatoriedad se limita a los empates entre candidatos equivalentes según el criterio aplicado. De esta manera, se introduce diversidad sin convertir el procedimiento en una búsqueda completamente aleatoria [27].

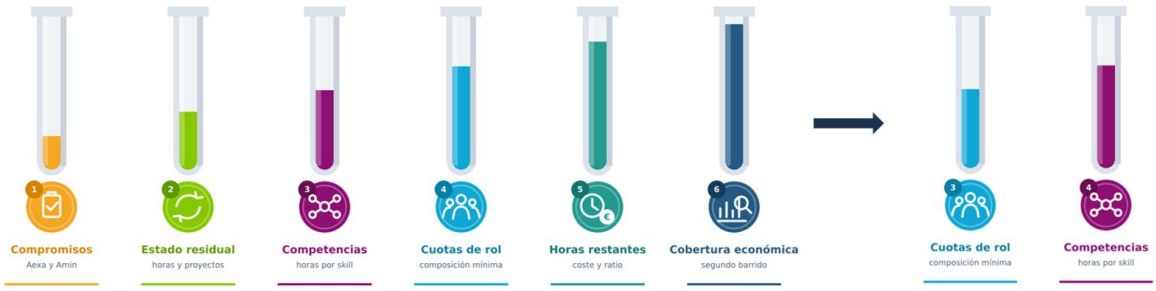


Figura 13. Orden de reglas aplicadas a la heurística.

Cada reinicio parte de una solución vacía y reconstruye todos los valores y estados de manera independiente. Si un intento completa las reglas, el candidato se somete a una validación (descrito en el subcapítulo 3.2 como 5ª etapa de la ejecución). Si no puede cubrir una necesidad con los trabajadores disponibles, el intento finaliza y se inicia el siguiente. El procedimiento no reutiliza parcialmente un intento fallido, ya que hacerlo podría conservar decisiones que originaron el bloqueo.

Cuando varios reinicios producen soluciones válidas, se conserva la mejor de acuerdo con el criterio de calidad implementado (de nuevo, se detalla en el subapartado 4.5.3) y se mantienen sus métricas para su evaluación posterior. Este criterio calcula, con las soluciones halladas de la heurística, las desviaciones normalizadas en horas, coste e ineficiencia asociada a las competencias (de manera parecida a la función objetivo multiobjetivo solo que se usa como puntuación, no como objetivo). Cuando ninguno completa el escenario, el estado se registra como fallo heurístico. Este estado no se convierte en inviabilidad: únicamente indica que la estrategia constructiva configurada no encontró una asignación en los intentos disponibles.

3.4.3. MODELO DE OPTIMIZACIÓN MULTIOBJETIVO

El segundo enfoque amplía el modelo anterior mediante programación por objetivos. En este caso no solo se busca obtener una solución factible, sino también que las horas asignadas, el coste salarial y la adecuación de habilidades técnicas se acerquen lo máximo posible a los valores objetivo de cada proyecto.

VARIABLES DE DECISIÓN

$X_{t,p,s} \geq 0$, número de horas dedicadas a la skill s por el trabajador t para el proyecto p.

$Y_{t,p} \geq 0$, número de horas asignadas al trabajador t para el proyecto p.

$$Z_{t,p} \begin{cases} 1, & \text{si el trabajador T participa en el proyecto P.} \\ 0, & \text{si no.} \end{cases}$$

$\alpha_{Hp} \geq 0$, desviación de horas asignadas frente a las mínimas requeridas por proyecto.

$\alpha_{Ep} \geq 0$, desviación de euros gastados frente al coste laboral mínimo requerido por los salarios implicados en el proyecto.

$\alpha_{Sp,s} \geq 0$, desviación de horas frente a las óptimas productivas, asociadas a la incompatibilidad entre las skills de los trabajadores asignados y la skill requerida.

FUNCIÓN OBJETIVO

En este caso, se propone un modelo de optimización multiobjetivo empleando el método de programación por objetivos. La finalidad consiste en minimizar conjuntamente las desviaciones de: 1) horas excedidas del mínimo requerido, 2) coste excedido del mínimo requerido y 3) horas de productividad perdidas a la incompatibilidad entre las skills de los trabajadores asignados y las requeridas por los proyectos.

Para hacer comparables estas desviaciones, expresadas en unidades y escalas diferentes, se normaliza cada una respecto a la magnitud total de referencia de la cartera de proyectos:

$$min\left\{\frac{\sum_P \alpha_{Hp}}{\sum_P H_p} + \frac{\sum_P \alpha_{Ep}}{\sum_P E_p} + \frac{\sum_P \sum_{s\epsilon S_p} \alpha_{Sp,s}}{\sum_P \sum_{s\epsilon S_p} H_{p,s}}\right\}$$

Esta formulación mide el exceso total de cada requisito de proyecto respecto a su correspondiente objetivo total. De este modo, todas las unidades de desviación reciben la misma penalización independientemente del proyecto en el que se produzcan. Así, por ejemplo, una hora o un euro adicionales conserva el mismo valor para la organización con independencia del tamaño del proyecto. Debe distinguirse esta formulación de la normalización individual por proyecto:

$$NO\ ADECUADA: min\left\{\sum_P \frac{\alpha_{Hp}}{H_p} + \sum_P \frac{\alpha_{Ep}}{E_p} + \sum_P \sum_{s\epsilon S_p} \frac{\alpha_{Sp,s}}{H_{p,s}}\right\}$$

Aunque esta segunda formulación es matemáticamente válida, no resulta adecuada para la lógica empresarial adoptada, ya que calcula una desviación relativa diferente para cada proyecto. En consecuencia, una misma unidad de exceso recibe una penalización mayor en los proyectos pequeños que en los proyectos grandes. La normalización agregada evita este comportamiento y representa la gestión de todos los proyectos como una única cartera.

La función objetivo puede expresarse de manera más legible y compacta mediante los coeficientes efectivos de normalización λ, μ, ρ a las desviaciones. Estos coeficientes convierten las tres desviaciones en magnitudes adimensionales y asignan la misma importancia a los tres criterios. Por tanto, el valor de la función objetivo representa la suma de las desviaciones relativas agregadas de horas, coste y skills. Así resulta finalmente la función objetivo:

$$\boldsymbol{min\ F} = min\left\{ \lambda \sum_{p\in P} \alpha_{Hp} + \mu \sum_{p\in P} \alpha_{Ep} + \rho \sum_{p\in P} \sum_{s\in S_p} \alpha_{Sp,s} \right\}$$

$$\lambda = \frac{1}{\sum_{p\epsilon P} H_p},\ \ \mu = \frac{1}{\sum_{p\epsilon P} E_p},\ \ \rho = \frac{1}{\sum_{p\epsilon P} \sum_{s\epsilon S_p} H_{p,s}},$$

RESTRICCIONES

Cobertura mínima de horas por proyecto: $H_p^{real} - \alpha_{Hp} = H_p\ ,\ \forall p \epsilon P$

Cobertura mínima de costes por proyecto: $C_p^{real} - \alpha_{Ep} = E_p,\ \forall p \epsilon P$

Cobertura min de skills por proyecto: $S_{p,s}^{efec} = H_{p,s},\ \forall p \epsilon P,\ \forall s \epsilon S_p$

Definición de la ineficiencia de *skill*: $\alpha_{Sp,s} = \sum_t X_{t,p,s}(1 - q_{t,s})\ ,\ \forall p \epsilon P,\ \forall s \epsilon S_p$

Horas máximas de cada trabajador: $\sum_p Y_{t,p} \leq h_t\ ,\ \forall t \epsilon T$

Participación mínima de rol por proyecto: $\sum_{t,r_t=k} Y_{t,p} \geq H_p \cdot \omega_{k,p}, \forall k \epsilon K\ , \forall p \epsilon P$

Proyectos máximos de cada trabajador: $\sum_p Z_{t,p} \leq R\ ,\ \forall t \epsilon T$

Horas mínimas de trabajador-proyecto: $Y_{t,p} \geq \theta_{t,p}^{min} \cdot h_t\ \forall\ t, p \epsilon A_{min}$

Horas exactas de trabajador-proyecto: $Y_{t,p} = \theta_{t,p}^{exa} \cdot h_t\ \forall\ t, p \epsilon A_{exa}$

Relación entre variables: $h^{min} \cdot Z_{t,p} \leq Y_{t,p} \leq Z_{t,p} \cdot h_t\ , \forall t \in T, \forall p \epsilon P$

$\sum_{s\epsilon S_p} X_{t,p,s} \leq Y_{t,p}\ ,\ \forall t \epsilon T, \forall p \epsilon P$

Restricciones de dominio: $X_{t,p,s} \geq 0, \forall t \epsilon T, \forall p \epsilon P, \forall s \epsilon S_p$

$Y_{t,p} \geq 0, \forall t \epsilon T, \forall p \epsilon P$

$Z_{t,p} = \{0,1\}, \forall t \epsilon T, \forall p \epsilon P$

$\alpha_{Ep}\ ,\ \alpha_{Hp}\ , \alpha_{Sp,s} \geq 0,\ \forall p \epsilon P, \forall s \epsilon S_p$

3.4.4. COHERENCIA Y COMPARABILIDAD ENTRE LOS MÉTODOS

La comparabilidad se apoya en cuatro controles: identidad de la instancia, igualdad de las reglas de factibilidad, validación común y esquema de salida homogéneo. Cada resultado

conserva el identificador del escenario y del método, por tanto las comparaciones se realizan entre ejecuciones que comparten exactamente los mismos datos y no entre promedios de carteras diferentes.

Las funciones objetivo son claras y no se confunden: las horas del modelo monoobjetivo proporcionan la referencia exacta de ese criterio mientras que el modelo multiobjetivo representa un equilibrio entre tres desviaciones. Por su lado, la solución que produce la heurística constructiva puede evaluarse con ambas familias de métricas. Cuando se calcula una diferencia, se especifica siempre qué valor se toma como referencia.

Los estados sin solución se conservan en la base experimental y sus métricas no se sustituyen por ceros. Un cero sería un valor numérico válido y alteraría medias y desviaciones. Mantener la ausencia permite separar el rendimiento de factibilidad de la calidad condicionada a haber obtenido una asignación.

Finalmente, el validador actúa como frontera común. El *solver* puede informar de un estado matemático y la heurística de un estado procedimental, pero únicamente las asignaciones que superan las comprobaciones independientes se incorporan como soluciones válidas. Este criterio reduce la dependencia de la implementación interna de cada método.

3.5. METODOLOGÍA DE EVALUACIÓN

Como hemos visto anteriormente, la resolución se aborda mediante tres enfoques. El primero es un modelo MILP monoobjetivo que minimiza las horas totales asignadas y se utiliza como referencia exacta para la optimización de las horas asignadas porque, al considerar un único criterio, permite determinar sin ambigüedad el mínimo absoluto alcanzable, sirviendo de punto de comparación antes de introducir los criterios adicionales que sí compiten entre sí en el modelo multiobjetivo. El segundo, como se acaba de mencionar, es un modelo MILP multiobjetivo, formulado mediante programación por objetivos, que busca evaluar una política distinta: minimizar los perjuicios que puede sufrir respecto a los objetivos de la empresa, penalizando de forma normalizada las desviaciones de horas, coste y adecuación de competencias. El tercero es una heurística constructiva que aplica secuencialmente las reglas del problema y realiza varios reinicios con órdenes de construcción y desempates diferentes. Dicha heurística reproduce las mismas obligaciones mediante reglas constructivas y proporciona una alternativa de menor coste computacional.

Cabe destacar que los tres enfoques no son versiones intercambiables de un mismo objetivo, si no diferentes instrumentos útiles para estudiar el compromiso entre calidad de la asignación, capacidad de resolución y tiempo de cómputo y, con ello, aplicarlo según la situación más se adecúe. Esta comparativa, como explicamos en apartados anteriores, se realiza desde una situación generada igual para todos los métodos, por lo que las diferencias únicamente se limitan a la función objetivo, las variables de desviación y el procedimiento de búsqueda. De este modo, cuando dos métodos devuelven soluciones distintas puede analizarse si la diferencia procede de la política de optimización o de la capacidad del algoritmo para encontrar una asignación, sin introducir condiciones empresariales diferentes.

La evaluación se realiza sobre un lote de instancias de escenarios variados que simulan distintas situaciones empresariales, generadas modificando parámetros como la demanda, la necesidad de roles concretos o la rigidez contractual. Todo esto se explicará en detalle en el subcapítulo 6.1. Todas las instancias se procesan con el mismo flujo y, tras completar la resolución y la validación, se comparan mediante un conjunto común de métricas: la factibilidad de la solución obtenida, el tiempo de ejecución, la diferencia respecto a la mejor

cota conocida, la robustez ante situaciones de negocio severas y la estabilidad de los resultados entre instancias de un mismo escenario.

Por último, cabe mencionar que la evaluación cuantitativa descrita en este apartado permanece independiente de la capa de interpretación lingüística: el agente LLM no debe emplearse para comparar métodos, modificar una solución ni sustituir a los modelos de resolución. Su función se limita a traducir a un diagnóstico comprensible la solución ya calculada y validada de un escenario concreto, sin alterar los valores ni las conclusiones cuantitativas en que se basa la comparación.

4. DESARROLLO COMPUTACIONAL DE LOS MÉTODOS DE RESOLUCIÓN

4.1. ARQUITECTURA SOFTWARE Y COMPONENTES

La arquitectura del código se ha organizado como un flujo experimental que actúa bajo un prisma de datos y configuraciones homogéneas. Esta versión final se articula como una cadena de datos que transforma una configuración de escenario en instancias JSON; las instancias se transforman a una representación interna común; tres métodos resuelven exactamente el mismo problema y un orquestador valida, serializa y consolida los resultados. La separación entre ficheros responde, por tanto, a una frontera de datos y de responsabilidad concreta, no solo a una decisión organizativa.

El directorio contiene el código fuente, la configuración y las pruebas. Las carpetas de datos y resultados no se entienden como entradas estáticas, sino que se crean durante la ejecución del generador de escenarios y resolución de los modelos. El siguiente árbol recoge la estructura utilizada, donde estos archivos mencionados se distinguen con la etiqueta «generado»:



Las carpetas .venv, __pycache__ y las cachés de las herramientas no forman parte de la arquitectura del proyecto. .gitignore las excluye junto con los archivos temporales; requirements.txt limita las dependencias del prototipo a pyomo>=6.7 y highspy>=1.7.

Dentro de la parte funcional de la estructura empleada, la siguiente Tabla 5 recoge los principales ficheros que conforman la solución. Con cada uno de ellos se explica de manera breve la responsabilidad técnica asociada a cada uno de ellos y las funciones (o campos de datos) mediante los cuales se intercambia información con el resto del sistema.

Tabla 5. Resumen de la estructura del proyecto

Archivo	Responsabilidad técnica	Elementos principales
config_escenarios.json	Declara los parámetros invariantes y las diez tipologías del experimento.	Semillas, roles, competencias, compatibilidad, políticas, pesos…
generador_escenarios.py	Valida la configuración y construye instancias según las variaciones de escenario.	validar_configuracion, generar_escenario, validar_variacion_controlada, generar_todos.
modelo _comun.py	Implementa el contrato común de entrada y salida de los tres métodos.	cargar_escenario, comprobaciones_previas, validar_solucion, metricas_solucion y serializar_solucion.
normalizacion_objetivos.py	Centraliza la normalización agregada de horas, coste e ineficiencia técnica.	normalizar_desviaciones_agregadas.
modelo_un_objetivo.py	Construye y resuelve el MILP que minimiza las horas totales asignadas.	crear_modelo, resolver_modelo resolver_escenario.
modelo_multiobjetivo.py	Construye el modelo de programación por objetivos normalizada.	crear_modelo y resolver_escenario; reutiliza la selección de *solver* del modelo monoobjetivo.
modelo_heuristico.py	Construye soluciones sin Pyomo mediante una estrategia multiarranque.	crear_estado, cubrir_roles, cubrir_skills, cubrir_horas, cubrir_costes y resolver_escenario.
ejecutar_experimentos.py	Coordina la campaña, conserva resultados individuales y genera la comparación.	ejecutar, _ estructurar_CSV y _agregar.

Esta visión nos permite identificar de forma ordenada qué componentes intervienen en la configuración y generación de escenarios, cuáles concentran la lógica común de validación y cálculo de métricas, qué módulos implementan los distintos métodos de resolución y qué fichero coordina la ejecución experimental y la obtención de resultados.

4.2. ANÁLISIS DE HERRAMIENTAS Y SOLUCIONES DE SOFTWARE

Para que el flujo de datos fluya correctamente, se requiere cohesión de los componentes del proceso. Para ello, aparte de los archivos de lenguaje Python y JSON, es necesario determinar qué herramientas y soluciones de software permiten implementar y coordinar adecuadamente cada uno de ellos. Debemos tener en cuenta modelos de optimización, lenguajes de modelado, *solvers*, entornos de formulación matemática. En los últimos años son

varios los lenguajes y herramientas que se han desarrollado con el fin de ser empleadas en este tipo de problemas.

Pyomo es un lenguaje de modelado abierto embebido en Python que permite definir conjuntos, parámetros, variables, objetivos y restricciones y conectarlos con *solvers* externos [45]. Esta característica resulta especialmente adecuada para representar de forma compacta las relaciones entre trabajadores, proyectos y competencias del problema estudiado y generar las instancias a partir de los datos de cada escenario. Hemos de tener en cuenta que Pyomo no incorpora por sí mismo los algoritmos de optimización: actúa como capa de formulación y, mediante sus interfaces, envía el modelo instanciado al *solver* y recupera el estado de terminación, la función objetivo y las variables de decisión.

La implementación de una formulación algebraica requiere distinguir entre la herramienta de modelado y el motor de resolución. La primera, que sería Pyomo, es la representación estructurada de los conjuntos, parámetros, variables, objetivos y restricciones; el segundo aplica los algoritmos numéricos necesarios para obtener una solución [46]. Para esta segunda parte, se estudian los *solvers* disponibles para resolver la formulación de Pyomo. Una propuesta interesante es HiGHS, un *solver* abierto para MILP y QP (*Quadratic Programming*) con interfaces para Python (entre otros lenguajes) [46].

HiGHS emplea, para los problemas lineales, métodos de símplex revisado y de punto interior, mientras que los problemas MILP se abordan mediante un procedimiento de *branch-and-cut* [46]. Esta última capacidad es la relevante para los modelos exactos del presente TFM, en los que coexisten variables continuas y decisiones discretas de asignación. Su interfaz para Python y su integración con Pyomo permiten configurar la resolución y recuperar de forma programática los resultados. Además, su licencia abierta evita la dependencia de licencias comerciales y favorece la reproducibilidad de los experimentos.

Por todo esto, la combinación Pyomo-HiGHS es la adoptada en este trabajo que establece una arquitectura modular: Pyomo gestiona la formulación y los datos, mientras que HiGHS resuelve los modelos monoobjetivo y multiobjetivo. La heurística se implementa directamente en Python y se evalúa sobre las mismas instancias y métricas. Sin embargo, el trabajo deja abierta la opción de resolver la formulación de Pyomo con otras alternativas de *solvers* en caso de tener éstas instaladas en lugar de HiGHS que son CBC o GLPK.

En la Tabla 6 siguiente vemos un resumen de todas las herramientas que están presentes en el trabajo, la función que tienen, su uso exacto y el encaje en el flujo. Además de ellas, es de cierto interés comentar otras alternativas consideradas que no se han empleado en el trabajo como el uso de *solvers* comerciales como Gurobi o CPLEX debido a que requieren licencia para su uso. Tampoco se emplean APIs de optimización ofrecidas por Google OR-Tools (también son flujos para resolver flujos, *routing* y programación por restricciones). cuyo *solver* CP-SAT trabaja con variables y coeficientes enteros y se adapta a problemas de asignación y *scheduling* con una componente lógica marcada [16].

Tabla 6. Herramientas y tecnologías empleadas en el trabajo.

Herramienta	Función	Uso en el trabajo	Encaje en el flujo de trabajo
JSON	Formato estructurado de almacenamiento de datos	Almacenamiento de los datos de entrada de los escenarios y de los resultados generados por los métodos de resolución	Facilita el intercambio de información entre las distintas etapas del sistema y permite conservar los datos en un formato estructurado y reutilizable

Python	Lenguaje de programación base del trabajo	Implementación de la heurística, tratamiento de datos, generación de escenarios, ejecución de modelos y procesamiento de resultados	Actúa como entorno de integración y coordina las distintas etapas del proceso
Pyomo	Lenguaje de modelado matemático en Python	Definición de los modelos de optimización, sus variables, restricciones y funciones objetivo	Traduce la formulación matemática a un modelo que puede ser enviado a un *solver*
HiGHS	Solver principal de la optimización	Resolución de los modelos de MILP cuya formulación se desarrolla con Pyomo	Recibe el modelo formulado y obtiene una solución óptima o informa sobre su estado de resolución
CBC / GLPK	Alternativas de *solvers* de optimización de código abierto	Alternativas de resolución de los modelos de optimización	Pueden sustituir a HiGHS en la etapa de resolución sin modificar la formulación en Pyomo

4.3. CONFIGURACIÓN DE ESCENARIOS

Una vez escogidas las tecnologías que emplearemos, debemos definir cómo comienza el problema a resolver. Como hemos explicado en el subcapítulo 3.1, la resolución trabaja en torno a un escenario empresarial que parte de una situación de negocio. Para poder trabajar con estos escenarios, deben explicarse y parametrizarse mediante ficheros JSON. Sin embargo, antes de ello, conviene distinguir dos términos que se emplean en este punto:

- Escenario o tipología: es la configuración general que define los rangos y reglas (por ejemplo, "escenario de alta presión de demanda").

- Instancia: cada concreción particular de esa configuración, con datos ya fijados.

En nuestra arquitectura, el archivo responsable de la capa de configuración JSON de las instancias es: config/config_escenarios.json. Es la fuente única de estructuración y parametrización de las instancias que se emplean en la evaluación, por lo que rige todo el diseño de los escenarios de cada experimento. Esto incluye la estructura de recursos y proyectos, las reglas de participación, las competencias y compatibilidades consideradas y los parámetros necesarios para los métodos de resolución. La separación entre configuración y código permite reproducir el experimento y modificar sus condiciones sin alterar la lógica de generación ni los modelos implementados.

Como resultado se obtiene una plantilla de escenario en formato JSON compuesto por los metadatos de identificación, los conjuntos de roles y competencias, los parámetros globales, la plantilla de trabajadores, los proyectos, las cuotas de rol, los requisitos H_{skill}, la matriz de compatibilidad y los compromisos A_{min} y A_{exa}. Esta estructura es la que da forma a las instancias que constituyen la entrada común de datos para los distintos métodos de resolución. Todos los campos y valores dependen del experimento que se esté haciendo, por lo que se detallará más sobre ello en el capítulo 6.1, sin embargo, el cuerpo del JSON será algo similar a lo siguiente:

```
{
  "semilla": <valor_semilla>,
  "instancias_por_tipo": <numero_instancias>,
```

```
  "R": <maximo_proyectos_por_trabajador>,
  "min_participation_hours": <participacion_minima>,

  "pesos_multiobjetivo": {"horas": <p_h>, "coste": <p_c>, "skills": <p_s>},
  "tipos": [<definicion_de_tipologias_y_variaciones>],
  "roles": {
    "TEC": {"proporcion": <p>, "horas": <h>, "coste_hora": <c>},
    "DIR": { ... },
    "FIN": { ... }
  },
  "skills": [<catalogo_de_competencias>],
  "Skill_pools_by_role": {"TEC": [<skills_tec>], "DIR":<s_d>], "FIN":<c_f>]},
  "compatibility_scale": {<niveles_de_compatibilidad>},
  "compatibility": {<matriz_de_compatibilidad>}
}
```

4.4. GENERADOR DE ESCENARIOS

Habiendo configurado el esquema y parámetros de escenario, durante su generación, algunos valores quedan fijados directamente por el fichero de configuración JSON, mientras que otros se concretan dentro de un rango definido de valores posibles. Esta segunda parte de la construcción de instancias individuales [como menciona el apartado 3.2] se realiza de manera pseudoaleatoria controlada, es decir que se aleatorizan unos valores dentro de un rango controlado. Para ello se emplean funciones de construcción para cada elemento necesario.

Así, la primera función encargada de generar el conjunto de los trabajadores, gen_trabajadores, convierte los intervalos de proporción de roles para la plantilla en repartos enteros que completen el tamaño de la plantilla. Una vez determinado el número de trabajadores por rol, a cada uno se le asigna la disponibilidad, el coste horario y unas competencias propias de su rol. En los perfiles de dirección se incluye obligatoriamente la *skill* "PROJECT_MANAGEMENT" y, de la misma manera, la *skill* "FINANCE" a los perfiles financieros. Además, la construcción se repite hasta un máximo de cien intentos, hasta que cada competencia aparece al menos en dos trabajadores.

La siguiente función, gen_proyectos, atribuye los elementos necesarios a los proyectos. Esta comienza recorriendo la lista de parámetros donde se definen H_p para cada proyecto. Sobre estas horas requeridas, se aplica un porcentaje de horas fijadas a las skills del 90% "fraccion_skills=0,9", ($\sum_s H_{skill,p,s} = 0{,}9 \cdot H_p$). Además, se obtiene el coste requerido por cada proyecto, multiplicando las horas totales requeridas por el coste medio de trabajador en plantilla [Tabla 15] y una variación pseudoaleatoria (u_p) que toma un valor entre ±5% ($E_p = H_p \cdot 39\,€/h \cdot u_p$). Por último, cada proyecto recibe una plantilla de tres competencias de manera aleatoria sin reemplazo dentro de cada ronda.

A su vez, la función gen_cuotas_rol materializa las cuotas mínimas de rol ($\omega_{k,p}$) y gen_asignaciones_obligatorias es la encargada de construir los subconjuntos A_{exa} y A_{min}. Para cada pareja trabajador-proyecto dentro de estos subconjuntos, selecciona una fracción permitida, evitando solapamientos de un mismo par trabajador-proyecto en ambos subconjuntos, dedicaciones acumuladas superiores al 100 % y más de R proyectos. La tipología de rigidez añade compromisos a las mismas asignaciones estándar y distribuye las parejas adicionales equilibrando número de proyectos y fracción ya comprometida.

Por último, la función generar_todos ensambla todas las instancias una vez configuradas y validadas. Solo cuando la generación finaliza correctamente sustituye los ficheros nuevos por los correspondientes al lote anterior. Después, crea un fichero manifest.json, donde se declara la versión empleada del código, las semillas, el recuento y el orden de ejecución

4.5. CONSTRUCCIÓN DE LOS MÉTODOS DE RESOLUCIÓN

Una vez configurado el JSON del escenario y generadas las instancias, el siguiente paso es pasárselo a los métodos como *input* para que generen una asignación efectiva. Sin embargo, los tres métodos de resolución no operan directamente sobre la representación textual de los ficheros JSON. Así que, el punto inicial del proceso de resolución podemos decir que es cargar_escenario. Esta función (común a los tres métodos) transforma la información almacenada en el fichero JSON a valores que puedan emplear los métodos. La función comprueba la presencia de las claves obligatorias, convierte las magnitudes a tipos numéricos y transforma las claves textuales de A_{min} y A_{exa} en pares o tuplas *t,p* (trabajador-proyecto). También verifica el esquema, los identificadores, los dominios de los parámetros, los requisitos positivos, los contadores contractuales por proyecto y determinados indicadores incluidos en los metadatos. Si alguna de estas comprobaciones falla, se genera una excepción antes de resolver la instancia, no se ejecuta el método y el tiempo de ejecución, por ende, se considerará nulo.

Durante la carga del JSON, se calcula asimismo la efectividad $q_{t,s}$ que es el máximo valor de compatibilidad entre la *skill* requerida *s* y cualquiera de las competencias que posee el trabajador *t* [cap. 3.3]. A continuación se construyen los conjuntos: *PS*, con las parejas proyecto-competencia requeridas, y *TPS*, con todas las ternas trabajador-proyecto-competencia. Aunque *TPS* incluye a todos los trabajadores, la cota superior de $X_{t,p,s}$ se fija a cero cuando $q_{t,s}$ es nula. De este modo, se mantiene una indexación uniforme sin permitir contribuciones técnicas no reconocidas en la matriz de compatibilidad [Tabla 1].

```
    effectiveness[(worker, required_skill)] = max(
    compatibility[(owned, required_skill)] for owned in owned_skills[worker])
    ps = [(project, skill) for project in projects
         for skill in required_by_project[project]]
    tps = [(worker, project, skill) for worker in workers
          for project, skill in ps]
```

Antes de iniciar una resolución, la función comprobaciones_previas identifica contradicciones que permita detectar anticipadamente una incongruencia: cuotas porcentuales de obligación laboral ($\omega_{k,p}$ o $\theta_{t,p}^{min}$) que no respeten los valores adquiribles, requisitos de horas de *skills* superiores a las horas totales necesarias del proyecto, fracciones contractuales no admitidas, solapamiento entre A_{min} y A_{exa}, compromisos que exceden la capacidad o *R*, demanda agregada superior a la capacidad, etc. En el caso de que se den una o más de estas incongruencias, aparece un error por lo que los tres métodos devuelven el estado *infeasible_precheck* con tiempo de resolución cero (ya que no se procede a calcular una solución). Superar esta etapa no demuestra factibilidad, únicamente descarta esas causas directas y nos ayuda a descartar ejecuciones no válidas para la valoración de las soluciones obtenidas.

Aunque ambas validaciones resulten parecidas, tienen un carácter muy diferente. En primer lugar, cargar_escenario se trata de una validación de entrada que comprueba que los datos estén bien formados y sean coherentes: estructura del JSON, identificadores existentes, tipos y rangos de valores, correspondencia entre campos, etc. Por su lado,

comprobaciones_previas es una precomprobación de factibilidad matemática que analiza si, aun estando los datos correctamente construidos, el problema presenta condiciones que permiten anticipar su infactibilidad, por ejemplo, que la capacidad sea menor que la demanda.

4.5.1. CONSTRUCCIÓN DEL MODELO MONOOBJETIVO

Para todos los modelos exactos con Pyomo, generamos un ConcreteModel particular. Esta labor corre a cargo del archivo modelo_un_objetivo.py. Se basa en un arquitectura de definición de conjuntos, parámetros, variables, restricciones con una función objetivo que minimiza las horas asignadas garantizando la cobertura de los requisitos de cada proyecto.

Se definen los conjuntos *T*, *P*, *K*, *S*, *PS* y *TPS*, que representan trabajadores, proyectos, roles, catálogo de competencias y los índices técnicos activos, junto a *TK*[k] que agrupa los trabajadores de cada rol.

```
def crear_modelo(datos: dict[str, Any]) -> ConcreteModel:

    model = ConcreteModel(name=f"un_objetivo_skills_{datos['scenario_name']}")

    model.T = Set(initialize=datos["T"], ordered=True)
    model.P = Set(initialize=datos["P"], ordered=True)
    model.K = Set(initialize=datos["K"], ordered=True)
    model.S = Set(initialize=datos["S"], ordered=True)
    model.PS = Set(dimen=2, initialize=datos["PS"], ordered=True)
    model.TPS = Set(dimen=3, initialize=datos["TPS"], ordered=True)
    model.TK = Set(model.K, initialize=lambda m, k:
datos["workers_by_role"].get(k, []))
    model.AMIN = Set(dimen=2, initialize=list(datos["A_min"]), ordered=True)
    model.AEXA = Set(dimen=2, initialize=list(datos["A_exa"]), ordered=True)
```

Los parámetros que se incorporan al modelo son: disponibilidad horaria *h*, coste horario c, demanda *H*, umbral económico E, cuotas omega *ω*, requisitos de competencias *H_skill* y efectividad *q*.

```
    model.h = Param(model.T, initialize=datos["h"])
    model.c = Param(model.T, initialize=datos["c"])
    model.H = Param(model.P, initialize=datos["H"])
    model.E = Param(model.P, initialize=datos["E"])
    model.omega = Param(model.K, model.P, initialize=datos["omega"],
default=0.0)
    model.H_skill = Param(model.PS, initialize=datos["H_skill"])
    model.q = Param(model.T, model.S, initialize=datos["q"], default=0.0)
    model.theta_min = Param(model.AMIN, initialize=datos["A_min"])
    model.theta_exa = Param(model.AEXA, initialize=datos["A_exa"])
```

Las variables principales son horas asignadas *Y[t,p]*, participación binaria *Z[t,p]* y las horas (parte de Y) vinculadas a cada competencia *X[t,p,s]*. H_{real}, C_{real} y S_{efec} materializan las horas, el valor económico y la cobertura efectiva.

```
    model.Y = Var(model.T, model.P, domain=NonNegativeReals)
    model.Z = Var(model.T, model.P, domain=Binary)
    model.X = Var(model.TPS, domain=NonNegativeReals, bounds=lambda m, t, p, s:
(0.0, datos["h"][t] if datos["q"][(t, s)] > TOL else 0.0))
    model.Hreal = Var(model.P, domain=NonNegativeReals)
```

```
    model.Creal = Var(model.P, domain=NonNegativeReals)
    model.Sefec = Var(model.PS, domain=NonNegativeReals)
```

Las restricciones se implementan mediante objetos *Constraint* que trabajan sobre los conjuntos correspondientes y definidos mediante funciones lambda que generan la expresión de la restricción para los integrantes del conjunto. Se modelan todas las restricciones descritas en el diseño del método (3.4.1).

```
    model.horas_reales = Constraint(
        model.P,
        rule=lambda m, p: m.Hreal[p] == sum(m.Y[t, p] for t in m.T)
    )
    model.costes_reales = Constraint(
        model.P,
        rule=lambda m, p: m.Creal[p] == sum(m.c[t] * m.Y[t, p] for t in m.T),
    )
    model.skills_efectivas = Constraint(
        model.PS,
        rule=lambda m, p, s: m.Sefec[p, s] == sum(m.q[t, s] * m.X[t, p, s] for t
           in m.T),
    )
    model.cobertura_horas = Constraint(
        model.P,
        rule=lambda m, p: m.Hreal[p] >= m.H[p]
    )
    model.cobertura_coste = Constraint(
        model.P,
        rule=lambda m, p: m.Creal[p] >= m.E[p]
    )
    model.cobertura_skills = Constraint(
        model.PS,
        rule=lambda m, p, s: m.Sefec[p, s] == m.H_skill[p, s]
    )
    model.disponibilidad = Constraint(
        model.T,
        rule=lambda m, t: sum(m.Y[t, p] for p in m.P) <= m.h[t]
    )
    model.cuota_rol = Constraint(
        model.K,
        model.P,
        rule=lambda m, k, p: sum(m.Y[t, p] for t in m.TK[k])
        >= m.H[p] * m.omega[k, p],
    )
    model.max_proyectos = Constraint(
        model.T,
        rule=lambda m, t: sum(m.Z[t, p] for p in m.P) <= datos["R"]
    )
    model.relacion_yz = Constraint(
        model.T,
        model.P,
        rule=lambda m, t, p: m.Y[t, p] <= m.h[t] * m.Z[t, p],
    )
    model.relacion_zy = Constraint(
        model.T,
        model.P,
        rule=lambda m, t, p: m.Y[t, p]
        >= datos["min_participation_hours"] * m.Z[t, p],
    )
    model.reparto_skill = Constraint(
```

```
        model.T, model.P, rule=lambda m, t, p: sum(m.X[t, p, s] for s
        datos["required_by_project"][p]) <= m.Y[t, p],
    )
    model.asignaciones_minimas = Constraint(
        model.AMIN,
        rule=lambda m, t, p: m.Y[t, p] >= m.theta_min[t, p] * m.h[t],
    )
    model.asignaciones_exactas = Constraint(
        model.AEXA,
        rule=lambda m, t, p: m.Y[t, p] == m.theta_exa[t, p] * m.h[t],
    )
```

Finalmente, se define la función objetivo, donde forzamos a minimizar la suma de las horas asignadas a todos los trabajadores y proyectos. Una vez incorporados todos los elementos de la formulación, la función devuelve el modelo completo para su posterior resolución mediante el solver seleccionado.

```
    model.obj = Objective(
        expr=sum(model.Y[t, p] for t in model.T for p in model.P), sense=minimize
    )
    return model
```

4.5.2. CONSTRUCCIÓN DEL MODELO MULTIOBJETIVO

El modelo multiobjetivo sigue una estructura muy similar al anterior. El archivo responsable, modelo_multiobjetivo.py, reproduce las mismas reglas de factibilidad y continúa empleando los mismos conjuntos de datos y parámetros del problema, aunque sustituye las restricciones de cobertura de horas y coste por ecuaciones de meta con desviaciones no negativas.

Para ello, se crean las variables α_{Hp}, α_{Ep}y $\alpha_{Sp,s}$ que cuantifican, respectivamente, el exceso de horas y de coste respecto a los valores de referencia de cada proyecto y la pérdida de eficiencia derivada de asignar trabajadores con una compatibilidad inferior a uno, es decir, las horas adicionales necesarias para alcanzar la cobertura técnica exigida. Esta última desviación no representa un déficit de competencias, ya que las horas efectivas continúan igualadas exactamente a los requerimientos establecidos.

Las antiguas restricciones *cobertura_horas* y *cobertura_coste* se eliminan o sustituyen por *metas_horas* y *metas_coste*. Además, se añade la restricción *desviacion_skills* para calcular la pérdida asociada a compatibilidades inferiores a uno.

```
    model.alpha_H = Var(model.P, domain=NonNegativeReals)
    model.alpha_E = Var(model.P, domain=NonNegativeReals)
    model.alpha_S = Var(model.PS, domain=NonNegativeReals)


    model.metas_horas = Constraint(
        model.P,
        rule=lambda m, p: m.Hreal[p] - m.alpha_H[p] == m.H[p]
    )
    model.metas_coste = Constraint(
        model.P,
        rule=lambda m, p: m.Creal[p] - m.alpha_E[p] == m.E[p]
    )
    model.desviacion_skills = Constraint(
```

```
        model.PS,
        rule=lambda m, p, s: m.alpha_S[p, s] == sum(
            (1.0 - m.q[t, s]) * m.X[t, p, s] for t in m.T)
    )
```

Por último, previamente a calcular la función objetivo, las desviaciones se normalizan respecto a los valores agregados de referencia. En concreto, se calcula la demanda total de horas, el coste total y el conjunto de horas requeridas por competencia. A continuación, las desviaciones acumuladas de cada dimensión se dividen entre su referencia correspondiente para obtener términos relativos y comparables. Estos valores normalizados se incorporan posteriormente a la función objetivo ponderada, impidiendo que una dimensión domine el resultado únicamente por presentar magnitudes numéricas superiores.

```
    hours_reference = sum(datos["H"].values())
    cost_reference = sum(datos["E"].values())
    skill_reference = sum(datos["H_skill"].values())
    normalized_terms = normalizar_desviaciones_agregadas(
        sum(model.alpha_H[p] for p in model.P),
        sum(model.alpha_E[p] for p in model.P),
        sum(model.alpha_S[p, s] for p, s in model.PS),
        hours_reference,
        cost_reference,
        skill_reference,
    )
    model.obj = Objective(expr=(
        peso_horas * normalized_terms["horas"]
        + peso_coste * normalized_terms["coste"]
        + peso_skills * normalized_terms["skills"]
    ), sense=minimize)
    return model
```

4.5.3. CONSTRUCCIÓN DE LA HEURÍSTICA CONSTRUCTIVA MULTIARRANQUE

La heurística intenta construir una solución válida siguiendo unas reglas ordenadas. El fichero encargado es [modelo_heuristico.py] que representa la asignación con los mismos diccionarios *Y*, *Z* y *X*, agrupados dentro de una estructura llamada *solution*. Esta estructura se almacena sobre otra de nivel superior llamada *state*. y en la Figura 14 podemos ver representada su anidación. Éste es un nivel de información que sigue la lógica iterativa de la heurística constructiva y representa el estado completo de la construcción heurística en un instante dado.

```
def crear_estado(datos: dict[str, Any]) -> dict[str, Any]:
    return {
        "solution": solucion_vacia(datos),
        "h_res": {worker: float(datos["h"][worker]) for worker in datos["T"]},
        "projects": {worker: set() for worker in datos["T"]},
    }
```

Como puede comprobarse, consta de tres bloques: *solution*, que es la solución que se está construyendo; *h_res*, que hace referencia a la disponibilidad residual por trabajador; y *projects*, que es el conjunto de proyectos en los que ya participa cada trabajador.

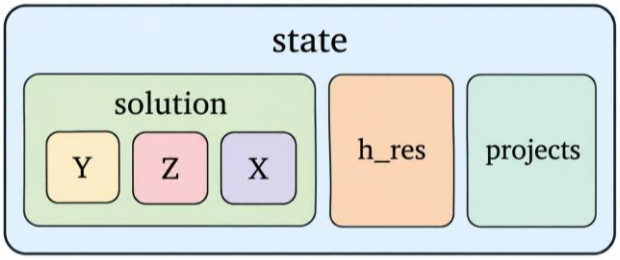


Figura 14. Anidación de estructuras de la heurística.

Partiendo de la base de estas estructuras, la construcción de cada candidato comienza con la creación de un estado independiente mediante la función [crear_estado]. Este estado contiene una solución inicialmente vacía, la información de *h_res* que comienza con la capacidad total de los trabajadores y *projects*, que comienza vacío ya que no se ha producido ninguna asignación. Como hemos visto, estas dos últimas estructuras permiten consultar de forma directa la capacidad disponible y controlar el número máximo de participaciones sin tener que recalcularlos a partir de la solución en cada operación.

Se calcula la cantidad de horas adicionales que pueden asignarse a una pareja trabajador–proyecto considerando capacidad residual, límite R, participación mínima y la cota de una asignación exacta. Para una participación nueva también comprueba que exista capacidad suficiente para respetar el mínimo de horas exigido. Sobre esta capacidad actúa la función, que constituye el único punto en el que se modifican las horas de asignación. En él, se actualizan de manera conjunta horas, participación, capacidad residual y proyectos activos: incrementa $Y_{t,p}$, activa $Z_{t,p}$, reduce *h_res* y añade el proyecto al conjunto de participaciones del trabajador. Para cada intento de resolución, el flujo que sigue la heurística lo vemos representado en el siguiente diagrama de la Figura 15:

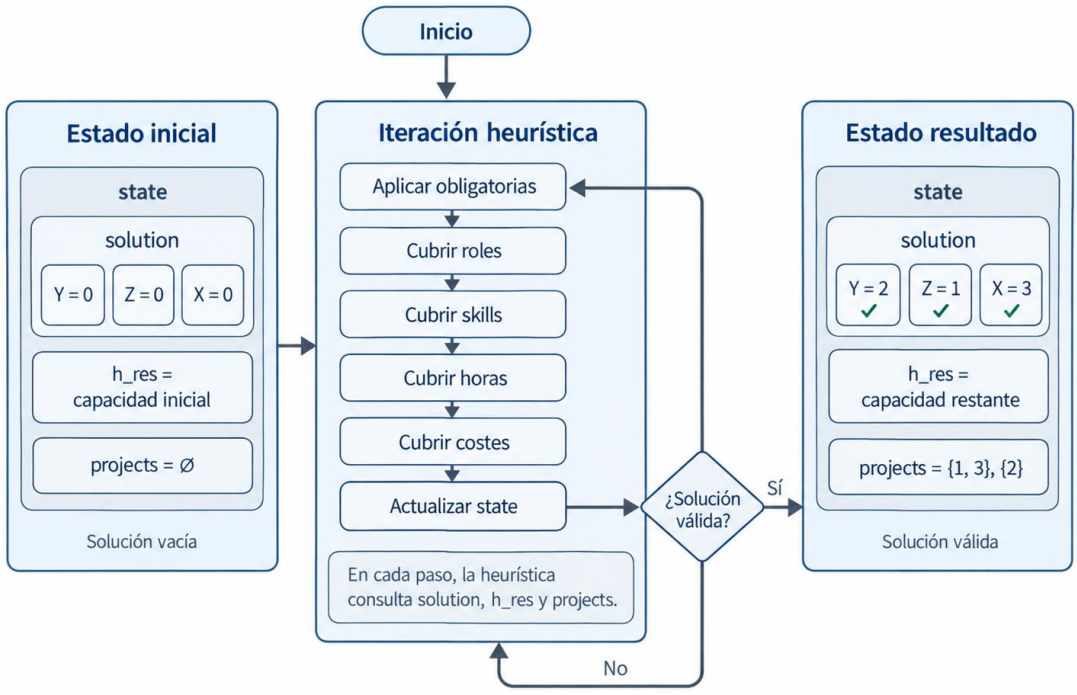


Figura 15. Diagrama de flujo de la heurística (elaboración propia).

Cada intento heurístico comienza, después de asegurar las asignaciones obligatorias, con una ordenación inicial de los proyectos. Esta ordenación establece la secuencia con la que los

proyectos serán tratados por las distintas fases constructivas. Los criterios jerárquicos que sigue esta ordenación son:

- En primer lugar, se priorizan aquellos con una mayor suma de cuotas de rol, al considerarse que concentran mayores exigencias de composición del equipo.
- En caso de igualdad, se utiliza el cociente E_p/H_p, que representa el coste horario medio del proyecto, dando prioridad a los proyectos más caros.
- Como tercer criterio se considera el volumen total de horas requerido H_p, de forma que los proyectos de mayor tamaño sean procesados antes de que parte de la capacidad de la plantilla haya sido consumida.

Si después de esto se llegase a un empate, se introduce un desempate pseudoaleatorio. Tras ello, el procedimiento heurístico continúa según el orden de prioridad de proyectos elegido, cumpliendo los requisitos de roles, skills, coste y horas.

La cobertura de skills comienza definiendo una lista de skills pendientes por resolver en los proyectos que agrupa el conjunto de parejas proyecto–competencia que todavía deben procesarse. Para cubrirlas, se calcula el déficit técnico pendiente de cada competencia definido como la diferencia porcentual entre las horas efectivas requeridas y las ya cubiertas. A continuación, estima la capacidad efectiva de cada trabajador compatible de sus horas ya asignadas al proyecto pero todavía no vinculadas a una competencia (si existieran) y de su capacidad asignable restante, ponderadas por el coeficiente de compatibilidad correspondiente. Un valor reducido de este cálculo indica poco margen de cobertura respecto de la demanda todavía no satisfecha, así que la pareja se considera más restrictiva y se cubre antes.

La cobertura de roles recorre los proyectos en el orden ajustado y para cada uno, procesa primero los roles con una mayor cuota de exigencia. Es decir, se calcula la diferencia entre las horas mínimas requeridas para el rol y las horas ya aportadas por los trabajadores pertenecientes a él ($\omega_{k,p}H_p - \sum Y_{t,p}$). Mientras el déficit sea positivo se asignan trabajadores hasta completar la exigencia. Para ello, considera los trabajadores que cumplen los requisitos y entre los candidatos, se ordenan primero por su utilidad técnica potencial para las competencias pendientes, después por su presencia previa en el proyecto, su número de participaciones y su capacidad residual.

La cobertura de horas cumple el objetivo de completar las horas totales $H_p$requeridas por cada proyecto. Para cada proyecto se calculan el déficit horario (diferencia entre la demanda requerida y las horas ya asignadas) y el económico (diferencia entre el coste requerido y los euros ya invertidos). Cuando ambos son positivos, se calcula el coste horario que satisfaría ambos déficits y se usa como referencia para orientar las nuevas asignaciones hacia trabajadores cuyo coste horario sea cercano al calculado, por lo que se prioriza la asignación de los trabajadores que cumplan este punto. Las horas se incorporan progresivamente hasta completar la capacidad de los trabajadores H_t, alcanzar el requisito horario del proyecto H_po hasta que no existan candidatos admisibles.

La cobertura de costes constituye una fase posterior de ajuste destinada a aquellos proyectos que, aun habiendo alcanzado su volumen de horas, no satisfacen todavía el requisito económico mínimo E_p. En este punto se calcula, para cada proyecto, la diferencia entre el coste requerido y el coste generado por las asignaciones existentes, y se atienden primero aquellos que presentan un mayor déficit económico. Las asignaciones se generan favoreciendo inicialmente a quienes ya forman parte del proyecto, tras ellos, los de mayor coste horario y posteriormente los que disponen de mayor capacidad residual hasta alcanzar E_po hasta agotar las posibilidades de asignación.

Como ya hemos visto el uso de una estrategia multiarranque es clave en la definición de la heurística implementada. Por ello, debemos decidir el número de reinicios con los que

queremos configurar esta herramienta multiarranque. Dentro de esa opción, es cierto que a mayor número de reinicios la heurística ofrecerá una factibilidad más cercana a su límite superior de rendimiento. Sin embargo, esto perjudicaría demasiado a la idea principal del modelo: rebajar considerablemente el tiempo de ejecución.

Por lo tanto, debemos conseguir un punto viable que tenga en cuenta ambas aspiraciones. Para ello realizamos unas pruebas midiendo la tasa de éxito y los tiempos de ejecución de la heurística al aumentar el parámetro –restart (número de veces que se ejecuta el modelo de la heurística para completar una solución viable). Además, para esta prueba se configura la heurística para que alterne las fases de cobertura de roles y cobertura de skills a cada intento par o impar para tener una mayor viabilidad de encontrar una solución [8].

Para comprender mejor los resultados, generamos una gráfica, visualizada en la Figura 16, donde comparamos el número de arranques (o intentos) de la heurística frente al número de soluciones que encuentra, para un número fijo de instancias, tan solo modificando el parámetro de intentos. En la parte izquierda del eje vertical vemos el porcentaje de factibilidad del modelo, es decir, el porcentaje de veces que consigue una solución viable de un modelo. En la parte derecha, vemos el tiempo medio de ejecución total, medido en segundos. Por último, el eje horizontal nos indica la cantidad de intentos que emplea la ejecución de la heurística.

El experimento se realiza empleando 1, 2, 3, 4, 6, 10, 20, 35 y 50 intentos en cada búsqueda heurística, empleando el mismo bloque de 30 instancias fijas (3 por cada tipo de escenario) y, es relevante aclarar que al ejecutar N restarts y M restarts, siendo M=N+r, las N primeras iteraciones de la segunda llamada, serán idénticas a la segunda por lo que solo añadirá r iteraciones nuevas en la búsqueda de una solución. Analizando todo ello podemos ver como el tiempo de ejecución crece de manera prácticamente lineal con un tiempo de ejecución medio de 27 ms (25-30 ms.) aproximadamente, sin mostrar saturación. La factibilidad (la curva azul) por el contrario, no crece de forma lineal, sino con rendimientos decrecientes: mejora con fuerza entre 1 y 3 intentos (53,3 %→60,0 %), se ralentiza hasta los 10, repunta hasta el 70,0 % en torno a 20, y a partir de ahí se estabiliza por completo hasta los 50 intentos, lo que sugiere un techo estructural asociado a instancias intrínsecamente infactibles.

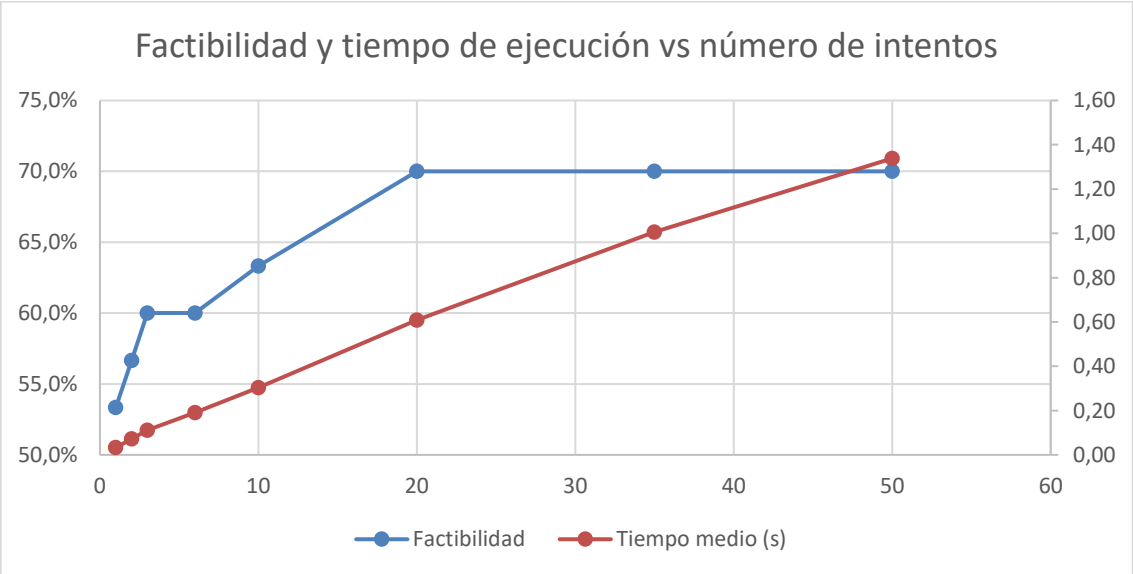


Figura 16. Visualización de la factibilidad de la heurística vs. el número de arranques.

Los primeros intentos aportan una mejora apreciable porque permiten probar las dos secuencias y algunos desempates distintos. Después, la mayoría de los nuevos intentos mantienen las mismas prioridades estructurales y producen asignaciones iguales o muy

similares. Ocasionalmente, un desempate temprano desencadena una secuencia diferente y permite recuperar un escenario, como sucede con 10 o 20 intentos. Sin embargo, a partir de 20, las nuevas semillas ya no recuperan ningún caso adicional.

Esto sitúa en torno a los 20 intentos un punto de inflexión claro: por debajo de este umbral, cada intento adicional aporta factibilidad a cambio de poco tiempo, pero por encima el tiempo sigue creciendo (de 0,60 s a 1,32 s entre 20 y 50 intentos) sin ninguna mejora adicional de factibilidad.

Así podemos adoptar varios puntos muy interesantes a elegir para la heurística:
- 4 reinicios: aporta una mejora grande respecto a no realizar ningún reinicio, a cambio de un tiempo medio de ejecución realmente bajo.
- 20 reinicios: aporta una factibilidad cercana al máximo, a cambio de un tiempo medio de ejecución bastante alto para ser el modelo heurístico.

Por ello, se adoptan 4 intentos como valor de referencia del generador al conseguir una calidad de factibilidad alcanzable correcta sin incurrir en un coste computacional injustificado que le quite valor al principal objetivo de la heurística que es la resolución rápida. De esta manera, para cada resultado final del intento, la heurística almacena el resultado si su puntuación es mejor a la obtenida por el intento anterior. Esta puntuación se calcula mediante la formula[1]:

$$score = normalized_multiobjetive + {}^{hours_total}/_{10^9}$$

Lo que calcula esta fórmula son las desviaciones normalizadas de horas, coste e ineficiencia de skills de su propia solucion frente los objetivos. Por ello, cuanto menor sea el resultado de esta operación, mejor se considerará el método. Se añade además, un segundo término, practicamente despreciable que se emplea como método de desempate entre dos soluciones de calidades muy parecidas, que favorece la solución que emplea menos horas totales.

Una vez elegido el número de intentos que empleará el modelo, es el momento de configurar nuestra heurística para que, dentro de sus limitaciones, maximice su tasa de éxito dentro de sus opciones. Para ello, realizamos unas pruebas sencillas variando algunos cálculos y orden de prioridades en la ejecución del modelo heurístico a las que llamaremos “versiones heurísticas”. Las distintas versiones del algoritmo se proponen teniendo en cuenta el multiarranque configurado en 4 *restarts* y se diferencian en tres decisiones de diseño:

- Orden de cobertura de skills. Para este paso, deben ordenarse las parejas proyecto-competencia a tratar en las iteraciones siguientes del proyecto. Para ello, se emplea un término que calcula las horas efectivas que puede aportar cada trabajador entre las horas efectivas totales necesarias. Este término puede emplearse al inicio del modelo sobre los datos generales, es decir, de manera estática ($\sum_t h_t \cdot q_{t,s}/H_{p,s}^{skill}$) ; o por el contrario, puede programarse para que se recalcule dinámicamente a medida que avanza la asignación.

- Orden de la cobertura de roles y competencias. Se prioriza completar primero uno y después el otro requisito o viceversa.

- Prioridad de cobertura de roles. Se prueban dos criterios principales al seleccionar los trabajadores para completar las restricciones de rol: la presencia en el proyecto (disponibilidad u ocupación previa) o la utilidad que puede aportar el trabajador (rendimiento esperado en el rol), o una combinación equilibrada de ambos (2 presencia + 2 utilidad). Uno ayuda principalmente a evitar traspasar la restricción de asignaciones máximas por trabajador, R y la segunda a asignar los trabajadores que mejor complementen sus cualidades a las necesidades del proyecto.

De esta manera generamos varias versiones de la heurística, cada una con un orden constructivo distinto (Tabla 7), y observamos los resultados que generan cada una haciendo una comparativa basada en la tasa de éxito para encontrar una solución (Tabla 8). En estos resultados se muestran, para cada versión del modelo heurístico, las instancias resueltas, su tasa de éxito entre los escenarios que pueden tener solución (eliminando las instancias irresolubles) y las veces que no encuentra una posible solución :

Tabla 7. Modificaciones probadas en cada versión heurística.

Versión heurística	Orden en la cobertura de skills de skill-trabajador	Orden de la cobertura de roles y competencias	Prioridad de cobertura de roles
v3	Estático	Alterna roles y competencias	Presencia → utilidad
v3.1h	Estático	Alterna roles y competencias	Utilidad → presencia
v3.2h	Dinámico	Alterna roles y competencias	Utilidad → presencia
v3.3h	Dinámico	Alterna roles y competencias	Presencia → utilidad
v3.5h	Dinámico	Alterna roles y competencias	Alterna presencia y utilidad
v3.4h	Dinámico	Siempre roles → competencias	Alterna presencia y utilidad

Tabla 8. Resultados de cada versión heurística.

Versión heurística	Instancias factibles	Instancias que tienen solución	Instancias no resueltas	Tasa de éxito
v3	462	892	430	51,79 %
v3.1h	467	892	425	52,35 %
v3.2h	509	892	383	57,06 %
v3.3h	499	892	393	55,94 %
v3.5h	507	892	385	56,84 %
v3.4h	477	892	415	53,48 %

Por ende, el método empleado finalmente será la heurística “v3.2h”, el cual presenta una forma dinámica de cubrir las competencias de cada proyecto en todos sus reinicios, alterna la cobertura de roles y competencias en su secuencia constructiva con los reinicios, y prioriza en la fase de cubrir roles, la productividad del trabajador (según sus competencias) antes que su presencia previa en el proyecto. Todo el proceso constructivo se orquesta bajo la función resolver_escenario que actúa como punto de entrada de la heurística y coordina el proceso completo de resolución. La jerarquía final de la ejecución, explicada con pseudocódigo, sería la siguiente:

```
resolver_escenario
├── comprobaciones_previas
├── bucle de ejecución de n intentos
│   └── un_intento
│       ├── cubrir asignaciones obligatorias
│       ├── ordenar proyectos a asignar
│       │   ├── mayor suma de cuotas de rol
│       │   ├── mayor cociente E_p/H_p
│       │   ├── mayor volumen de horas H_p
│       │   └── desempate pseudoaleatorio
│       ├── cubrir_roles / cubrir_skills
│       ├── cubrir_horas
```

```
    │       ├── cubrir_costes
    │       └── validar_solucion
    ├── cálculo de métricas y evaluación del intento
    │   └── actualizar mejor candidato
    ├── cálculo de métricas y evaluación del mejor intento elegido
    └── devuelve la solución seleccionada
```

Una vez entendida la secuencia de la heurística con el pseudocódigo, debemos entender en el entorno de Python, qué función se encarga de cada fase y qué función desempeña exactamente. Por tanto, se emplea a modo de resumen la Tabla 9 con las fases de la heurística, sus funciones programadas y el sentido que tienen en el contexto del método a continuación:

Tabla 9. Funciones por cada fase de la construcción heurística.

Fase	Funciones propias	¿Qué desempeñan?
Coordinación general del modelo	resolver_escenario	Coordina el proceso completo: comprobaciones previas, fases de ejecución de los distintos intentos, evaluación de candidatos y devolución de la solución seleccionada.
Comprobaciones previas	comprobaciones_previas	Detecta contradicciones estructurales antes de construir una solución. Si la encuentra, devuelve el estado *infeasible_precheck*.
Construcción del intento	un_intento	Coordina las distintas fases de asignación y establece el orden de asignación de los proyectos.
Asignaciones obligatorias	aplicar_obligatorias	Aplica primero A_{exa} y después A_{min} respetando las asignaciones estrictas.
Priorización de los proyectos a asignar	- (dentro de un_intento)	Ordena los proyectos por mayor suma de cuotas de rol, mayor E_p/H_p, mayor volumen de horas H_p y desempate pseudoaleatorio
Cobertura de roles y skills a los proyectos	cubrir_roles cubrir_skills	Completa los requisitos de roles y competencias según el orden definido para el intento.
Cobertura de horas	cubrir_horas	Completa H_p priorizando trabajadores presentes, adecuación de coste horario y capacidad residual.
Cobertura de coste	cubrir_costes	Completa E_p priorizando trabajadores presentes, mayor coste horario y capacidad residual.
Validación de cada intento	validar_solucion	Comprueba si la solución construida satisface las restricciones y devuelve los errores detectados.
Evaluación y selección	metricas_solucion resolver_escenario	Evalúa cada candidato, conservando la mejor solución factible en función a su score[1] o, si no existe, la que presenta menos errores.

Resultado final	validar_solucion metricas_solucion serializar_solucion	Vuelve a validar el candidato seleccionado, calcula sus métricas finales y transforma la solución que se devuelve a un formato compatible con JSON.

Por último, algo interesante a comentar y que es aplicable a la construcción de los tres métodos, es que los modelos pueden añadir información específica para su ejecución (siguiendo la función de Python: parser.add_argument) como modificar los pesos (--weights) en el multiobjetivo o los reinicios (--restarts) en la heurística. Esto se emplea para poder modificar una ejecución del experimento sin alterar los campos por defectos que utiliza el orquestador. En consecuencia, la comparabilidad se apoya en un contrato de datos estable y no en que los tres algoritmos compartan una implementación interna.

4.6. RESOLUCIÓN Y GESTIÓN DE ESTADOS

Anteriormente hemos visto la construcción de los modelos exactos, continuamos con su resolución: los dos modelos Pyomo utilizan la misma función resolver_modelo. Si se solicita un *solver* concreto en la llamada del método, lo primero que se hace es comprobarlo; si esto no ocurre se recorre la lista de *solvers* posibles en orden (highs, appsi_highs, cbc y glpk) omitiendo duplicados y motores no disponibles. La resolución de los métodos exactos finaliza cuando el motor devuelve una condición de terminación o cuando la ejecución se interrumpe externamente.

```
    candidates = [preferred] if preferred else []
    candidates += [name for name in
        ("highs", "appsi_highs", "cbc", "glpk")
        if name not in candidates]
    for name in candidates:
        solver = SolverFactory(name)
        if solver is None or not solver.available(exception_flag=False):
            continue
        results = solver.solve(model, tee=False, load_solutions=False)
        return name, results
```

Según el estado del *solver* empleado por Pyomo y su condición de terminación [47], se determina cuál es el estado final de la ejecución:

- El estado *optimal* indica que el *solver* ha encontrado una solución válida (SolverStatus.ok) y ha certificado su optimalidad (TerminationCondition.optimal).

- El estado *infeasible* se emplea cuando la condición de terminación indica que el modelo no admite una solución factible (TerminationCondition.infeasible).

- El estado *invalid* clasifica los resultados que se dan cuando el *solver* devuelve un candidato, pero este no supera la validación. Este estado solo actúa como "red de seguridad" y no debería darse bajo condiciones normales, ya que las comprobaciones de validar_solucion() reproducen esencialmente las mismas restricciones que se han impuesto al modelo.

- El estado *failed* recoge los casos donde la ejecución finaliza en otras terminaciones del proceso de resolución distintas de la optimalidad o la inviabilidad (TerminationCondition distinto a optimal o infeasible) .

- El estado *infeasible_precheck*, definido por la propia implementación y no por los estados naturales de Pyomo, recoge los escenarios que se analiza su infactibilidad previamente a la construcción de la solución. Esto se da cuando las comprobaciones previas detectan una inconsistencia estructural en la instancia construida.

La heurística, a diferencia que los modelos exactos, no delega su resolución en un *solver* externo. La función resolver_escenario coordina los distintos reinicios, ejecuta la construcción de un candidato en cada intento de ejecución (_escoger_intento) y valida el resultado obtenido en cada uno de ellos. Como ya hemos mencionado en el subapartado anterior, cuando un intento obtiene una solución válida, su puntuación es:

$$score = normalized_multiobjetive + {}^{hours_total}/_{10^9}$$

Si ningún intento produce una solución válida, se genera el estado de fallo en la búsqueda de solución: *failed* y se conserva el candidato que presenta menos errores de validación (se explica en el apartado 4.7 siguiente). Este candidato no se acepta como solución, sino que se mantiene para facilitar el diagnóstico de las restricciones que no han podido cumplirse. Así, solo se pueden alcanzar dos estados: *feasible* o *failed*, nunca asegura encontrar el óptimo (*optimal*). La Tabla 10 recoge todos los estados posibles de la solución que acabamos de contar, tanto para los métodos de optimización como para los heurísticos, y genera un pequeño resumen de su significado operativo, es decir, lo que ha debido pasar para llegar a ese estado.

En la Figura 17, además, se realiza un diagrama de flujo de los estados de los resultados posibles de los métodos, tanto exactos como la heurística, donde se ve qué situación debe darse hasta llegar a un estado final.

Tabla 10. Estados posibles de las soluciones.

Estado	Métodos	Significado operativo
infeasible_precheck	Todos	Las comprobaciones necesarias detectan una contradicción antes de resolver.
optimal	Exactos	El solver termina de forma óptima y la solución supera la validación independiente.
infeasible	Exactos	El solver declara la inviabilidad matemática del modelo.
failed	Exactos	El solver devuelve cualquier otra combinación de estado y terminación; no se cargan variables.
invalid	Exactos	El solver informa de óptimo, pero la asignación extraída incumple el validador común.
feasible	Heurística	Al menos un intento produce una asignación que supera todas las comprobaciones.
failed	Heurística	Ningún intento supera la validación; se conserva el mejor candidato diagnóstico.

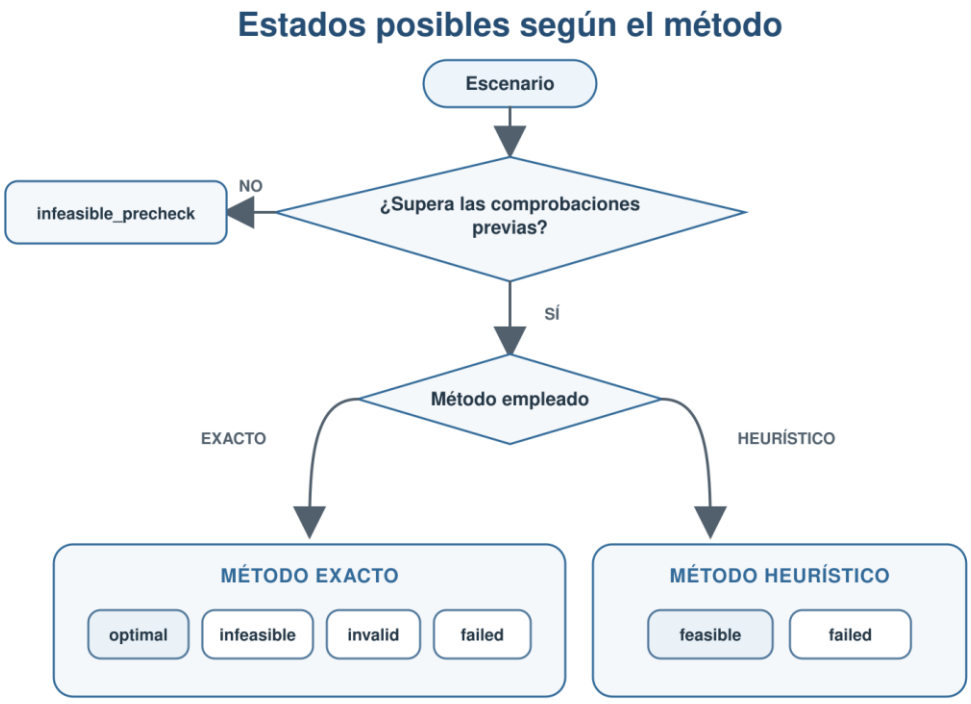


Figura 17. Diagrama de flujo de los estados de la solución (elaboración propia).

Además de la solución y su estado, los tres métodos almacenan el tiempo de ejecución mediante time.perf_counter(). La llamada a esta función abarca únicamente la llamada de resolución, no la carga del JSON, la construcción del modelo ni la serialización posterior. Por ende, si un escenario se considera *infeasible_precheck* su tiempo de ejecución será 0.

```
start = time.perf_counter()
results = solver.solve(model)
runtime_seconds = time.perf_counter() - start
```

4.7. VALIDACIÓN Y ORQUESTACIÓN

La implementación distingue entre la validación previa de la instancia y la validación posterior de las soluciones. Como hemos podido ver en el subcapítulo 4.5, antes de ejecutar cualquiera de los métodos, la función comprobaciones_previas examina las incongruencias estructurales que puedan provenir de la creación de una instancia y, si alguna de estas comprobaciones falla, la instancia recibe el estado *infeasible_precheck* y no se inicia el procedimiento de resolución.

Aparte de esta, cuando un método genera una solución candidata, se aplica la función validar_solucion(datos, solution). El primer argumento contiene los requisitos y parámetros de la instancia, mientras que solution recoge las asignaciones obtenidas mediante X, Y y Z. La función no modifica ni trata de reparar la solución, sino que realiza una comprobación independiente de su consistencia que sirve como "red de seguridad", por si se hubiera programado incorrectamente el modelo que emplea Pyomo, y como trazabilidad de los errores.

De esta manera, en los modelos exactos esta comprobación no sustituye a la certificación realizada por el *solver*. Una solución declarada óptima ya satisface las restricciones del modelo

(siempre y cuando se haya configurado bien y esté dentro de las tolerancias empleadas durante la resolución). En la heurística, en cambio, esta comprobación constituye directamente el mecanismo que determina si la construcción obtenida es válida.

Para cada trabajador se verifican nuevamente la disponibilidad horaria, el número máximo de proyectos, la naturaleza binaria de *Z*, relaciones *Y-Z*, participación mínima, reparto de *X* y ausencia de contribuciones con compatibilidad nula. Para cada proyecto vuelve a comprobar horas, umbral económico, cuotas de rol y cobertura técnica exacta. Finalmente verifica todas las asignaciones A_{min} y A_{exa}. Esta etapa separa el estado informado por el motor o por la heurística de la aceptación experimental de la solución. Devuelve una lista de mensajes de error que si está vacía, la solución será considerada válida; si por el contrario contiene elementos, cada uno describe una restricción incumplida.

Tal y como se ha explicado en el capítulo 3.5 , tras completar la resolución y la validación, se recogen las métricas de cada método para poder hacer un análisis comparativo [la factibilidad de la solución obtenida, el tiempo de ejecución, la diferencia respecto a la mejor cota conocida, la robustez ante situaciones más severas y la estabilidad de los resultados entre instancias de un mismo escenario]. Para ello, la función metricas_solucion calcula las horas y el coste totales, los excesos agregados, las horas técnicas brutas y efectivas y la ineficiencia horaria por *skills*. También registra las horas contractuales mínimas, la intensidad contractual media y máxima, el número de parejas trabajador-proyecto activas y las binarias activas sin participación suficiente. Por parte de los modelos exactos sin solución, el bloque de métricas permanece vacío; sin embargo, por el lado de la heurística conserva las métricas del candidato seleccionado incluso cuando su estado final es *failed*, ya que sirven para interpretar el grado y la naturaleza del incumplimiento.

Como último paso, las soluciones se homogeneizan y serializan mediante la función serializar_solucion. Las claves expresadas como tuplas se convierten a texto, se conservan únicamente valores de *Y* y *X* mayores que 0 y las variables *Z* activas (es decir, trabajadores con alguna hora asignada). Los campos *precheck* y *validation*_errors se mantienen separados para distinguir las inconsistencias detectadas antes de la resolución de los incumplimientos observados en una solución candidata.

En conjunto, estas funciones forman una capa común de tratamiento de resultados entre los métodos de resolución y el resto de la implementación. Su finalidad es aplicar un criterio homogéneo de aceptación, calcular los indicadores definidos para la evaluación y transformar las soluciones en una estructura común de salida. Sobre esta base, el orquestador puede procesar de manera uniforme los resultados procedentes de los distintos métodos.

La ejecución sistemática de todos los escenarios generados y la aplicación de los tres métodos de resolución sobre ellos requiere que la implementación incorpore un módulo encargado de coordinar la campaña experimental. El fichero ejecutar_experimentos.py constituye su punto de entrada de la campaña. Su función ejecutar puede, si se requiriera, regenerar las instancias antes de iniciar el proceso, o trabajar con las instancias ya creadas previamente. A continuación, obtiene la secuencia de archivos mediante rutas_escenario y resuelve cada escenario JSON siguiendo el orden monoobjetivo, multiobjetivo y heurístico. El resultado individual de cada método se almacena antes de comenzar el siguiente, de manera que una posible interrupción posterior no elimina la traza de las ejecuciones ya completadas

```
for path in scenario_paths:
    one = _attach_run_metadata(
         resolver_un_objetivo(path, solver), run_metadata)
    escribir_resultado(one, individual / f"{path.stem}__un_objetivo.json")
    multi = _attach_run_metadata(
         resolver_multiobjetivo(path, solver), run_metadata)
    escribir_resultado(multi, individual / f"{path.stem}__multiobjetivo.json")
```

```
    heuristic = _attach_run_metadata(
         resolver_heuristico(path, restarts), run_metadata)
    escribir_resultado(heuristic, individual / f"{path.stem}__heuristico.json")
```

Completando los resultados, se genera un archivo JSON, un .CSV y un resumen TXT. Para formatear el texto correctamente se emplea _estructurar_CSV, que convierte cada resultado JSON con varios diccionarios anidados en una única fila tabular. Esta fila incluye la identificación del escenario, las semillas, el método utilizado, el estado, el tiempo de ejecución y las principales métricas. Gracias a ello, los resultados pueden reunirse en un CSV con una fila por combinación escenario–método.

Posteriormente, se calcula la diferencia (o exceso) porcentual de los resultados respecto a las referencias, se agregan y producen estadísticas descriptivas por método y tipología. Para ello se emplea la función _agregar que utiliza como referencia las horas totales asignadas de la solución monoobjetivo factible de cada escenario, ya que este modelo minimiza precisamente la dedicación total. Las métricas de calidad solo se agregan sobre soluciones factibles, mientras que el tiempo incorpora todas las ejecuciones. Cada diferencia respecto a la referencia se calcula porcentualmente de la manera:

$$diferencia(\%) = 100 \cdot \frac{resultado\ del\ método - resultado\ de\ referencia}{resultado\ de\ referencia}$$

Después, la función agrupa los resultados por método y tipología de escenario y calcula tasas de factibilidad, recuentos por estado y estadísticas como media, mediana, desviación típica, mínimo y máximo. Tras todo esto, se generan varios archivos a modo resumen de la ejecución que podemos ver en la siguiente Tabla 11. En ella se detalla el fichero de salida (salida) que se genera, el tamaño de este fichero (tamaño) y qué finalidad tiene su uso y el almacenamiento de datos de la ejecución en él.

Tabla 11. Archivos generados tras la ejecución.

Salida	Tamaño	Finalidad
individuales/XX.json	Escenario × método	Conserva estado, diagnóstico, métricas y asignación completa.
comparativa_modelos.csv	Una fila por ejecución	Facilita tablas dinámicas, gráficos y análisis estadístico externo.
resumen_agregado.json	Método y tipología	Almacena tasas, estados, estadísticos descriptivos, diferencias y metadatos.
resumen_agregado.txt	Campaña	Ofrece una síntesis legible de factibilidad, tiempo medio y diferencia heurística.

5. CONFIGURACIÓN DEL SISTEMA AGÉNTICO

El sistema agéntico LLM se incorpora como una capa de apoyo a la decisión posterior al proceso de resolución definido en el capítulo anterior. Su finalidad es facilitar que un responsable empresarial comprenda qué representa una solución concreta y qué implicaciones operativas pueden derivarse de ella. El prototipo se configura en Microsoft Copilot Studio y se centra en la interpretación de un único escenario y de la solución seleccionada para ese escenario. Por tanto, no se utiliza para comparar modelos, generar una nueva asignación ni modificar automáticamente los datos de entrada.

Esta delimitación mantiene una separación clara entre cálculo e interpretación. Pyomo, HiGHS o la heurística producen los valores numéricos; el validador determina si la asignación cumple las reglas; y el agente reorganiza esa evidencia en una explicación comprensible para el decisor. En consecuencia, el agente no certifica la optimalidad ni la factibilidad por sí mismo, sino que debe respetar el estado y los errores de validación incluidos en la entrada.

El flujo agéntico requiere separar observación, análisis y acción. La observación puede proceder de archivos JSON o TXT con campos y unidades definidos; el análisis distingue datos obtenidos directamente del *solver*, inferencias derivadas de ellos y recomendaciones de actuación; la acción puede consistir en redactar un informe, solicitar un dato ausente o proponer una modificación de escenario. Si una nueva consulta cambia disponibilidad, costes, requisitos, pesos o restricciones, debe enviarse al generador y al *solver* para una nueva resolución antes de presentar consecuencias cuantitativas.

La evaluación para concluir si un agente de interpretación realiza su función de mejor o peor manera, debe considerar criterios distintos a los del modelo de optimización subyacente. Entre ellos se encuentran la fidelidad a los datos, la cobertura de los hallazgos relevantes, la consistencia de las unidades, la distinción entre factibilidad y optimalidad, la trazabilidad de cada afirmación y la utilidad para decidir. También debe comprobarse que el agente reconoce los límites de la evidencia, por ejemplo: que un resultado fallido de la heurística no demuestra inviabilidad, una solución factible no está necesariamente certificada como óptima y una consulta *what-if* exige volver a resolver cuando se ha modificado el modelo [48].

La utilidad esperada del agente LLM configurado reside en acercar el lenguaje y la estructura técnica del resultado (JSON, CSV…) a las decisiones humanas del responsable de planificación. El agente debe reconocer el estado de la solución, seleccionar las métricas relevantes, localizar los elementos que explican una tensión y comunicar limitaciones. Por ello, se configura de manera en la que su alcance es deliberadamente limitado. Es decir, no dispone de permisos para modificar los JSON, ejecutar el repositorio, reasignar trabajadores ni enviar decisiones a otros sistemas. Su salida es un informe meramente complementario que el usuario puede contrastar con el archivo de solución. Esta limitación reduce el riesgo de que una interpretación incorrecta produzca una acción automática sobre personas o proyectos.

El agente se creó en el entorno de Microsoft Copilot Studio, accesible mediante la licencia de la UPM. Se configuró un agente específico para el presente TFM, separado de los módulos Python, con español como idioma de interacción y con un nombre y una descripción orientados al análisis de resultados de asignación de recursos. Copilot Studio permite definir el comportamiento mediante instrucciones, conocimiento y herramientas dentro de un entorno de bajo código [49]. Microsoft Copilot Studio se eligió, como se ha mencionado, por su disponibilidad en el entorno institucional (UPM), pero también por su interfaz de bajo código y la integración de creación, prueba, supervisión y publicación. La plataforma permite definir instrucciones, asociar fuentes de conocimiento, habilitar entradas de archivo y construir conjuntos de evaluación sin desarrollar una aplicación web específica. Estas capacidades reducen el esfuerzo de infraestructura y hacen visible el ciclo de configuración,

muy útil para un prototipo académico centrado en el comportamiento del agente [49]. Sin embargo, la elección de Microsoft Copilot Studio no implica que sea la única arquitectura posible. Existen otras soluciones programáticas como LangGraph o Microsoft Agent Framework que ofrecen mayor control en el estado, la observabilidad y el despliegue del agente, pero a cambio requieren implementar y mantener más componentes. [50], [51]

5.1. CREACIÓN Y CONFIGURACIÓN DEL AGENTE

Construir un agente en Microsoft Copilot Studio desde cero implica configurar una serie de elementos básicos que definen su comportamiento y capacidades [49]. Principalmente se basa en un nombre y descripción breve, unas instrucciones y fuentes de conocimiento que sirvan de contexto. Aparte de esto, también es posible definir temas propios, que son flujos conversacionales predefinidos para gestionar intenciones concretas del usuario o acciones que permiten al agente conectarse con conectores externos como APIs para ejecutar tareas reales, no solo responder preguntas. Finalmente, se configuran los canales de publicación y los ajustes de seguridad y autenticación, para determinar dónde y cómo los usuarios interactuarán con el agente [52].

Una vez creado el agente, el siguiente bloque de trabajo se centra en su validación y puesta a prueba. Para ello, se definen las salidas (*outputs*) esperadas según las entradas de datos que reciba (*inputs*), es decir, qué tipo de información recibirá el agente y el formato y contenido debe tener su respuesta [53]. Además, podemos establecer un protocolo de interpretación que fije los criterios de entendimiento y valorar de forma consistente las respuestas generadas evitando ambigüedades.

Sobre esa base, se elabora el diseño de la evaluación, en el que se especifican los casos de prueba, las métricas y los escenarios frente a los que se contrastará el comportamiento del agente. En una primera fase, se realiza una ejecución a modo de prueba sobre la configuración del agente. En ella, se comprueba que su funcionamiento siga las pautas que le hemos indicado de manera rigurosa, especialmente en términos como:

- alcance, que responda solo sobre aquello para lo que ha sido diseñado y no invente capacidades o conclusiones fuera de su ámbito.
- prudencia, que no recomiende implantar soluciones infactibles ni presente como seguras conclusiones que los datos no permiten sostener.
- comportamiento, que siga las instrucciones establecidas, use correctamente los ficheros y produzca respuestas con el formato y nivel de detalle esperados.

Una vez completa exitosamente esta prueba, comprobaremos su utilidad analítica real pasándole como *inputs* las soluciones del *solver*. Superada esta fase, se avanza hacia un plan de validación definitiva, con pruebas más exhaustivas y representativas del uso real que confirmen la solidez de los resultados.

5.1.1. BLOQUES EN LA CONSTRUCCIÓN DEL AGENTE

El contexto útil que empleamos para configurar nuestro agente procede de tres capas: la primera, las instrucciones globales, que fijan el rol que debe simular, el tono que debe usar, hasta dónde se extiende su alcance, los bloqueos, el orden de análisis, etc.; la segunda son documentos (u otras fuentes) de conocimiento que explican la semántica del fichero JSON de resultados que analiza, los estados, las métricas y la plantilla de informe; por último, la tercera es el archivo adjunto por el usuario, que contiene la solución concreta y se considera la única fuente válida para atribuir cifras a un escenario.

Las instrucciones son un conjunto de indicaciones en lenguaje natural que le dicen al agente cómo debe comportarse, qué tono usar y qué tareas realizar o evitar. Sabiendo ello, se redactaron siguiendo estas pautas y le agregamos un orden de prioridad: primero alcance, después entrada y validación, a continuación análisis y, por último, recomendación y formato. Esta jerarquía permite resolver conflictos antes de que el agente empiece a calcular o buscar información. La configuración se compactó sin eliminar las reglas críticas: se redujeron repeticiones, se empleó una notación uniforme para los campos y se trasladó el conocimiento descriptivo estable a documentos auxiliares. El resultado conservó una secuencia de decisión explícita, visible en la siguiente Tabla 12, con el fin de que el motor de entendimiento LLM lo comprenda de una manera óptima. [52] En ella, lo que se pretende es dar unas pautas claras al agente sobre cómo debe actuar para cada bloque de comportamiento y la respuesta que debe evitar, por medio de seguir estas instrucciones. Además, las instrucciones exactas pueden encontrarse en [Anexo C: Instrucciones en la configuración del agente LLM].

Tabla 12. Reglas en el entendimiento del LLM.

Bloque	Pautas estrictas de comportamiento	Fallo que pretende evitar
Rol y objetivo	Actúa como apoyo técnico a la decisión y emite un dictamen no vinculante.	Que el agente sustituya al responsable humano o autorice una implantación.
Prioridad y alcance	Comprueba primero alcance y entrada; analiza una única solución y escenario por conversación y no realiza comparaciones.	Mezclar escenarios, modelos o resultados fuera del objeto del agente.
Entrada e identificación	Exige un archivo interpretable e identifica escenario, método, implementación, semillas y *solver*.	Analizar sin evidencia suficiente o atribuir propiedades a partir del nombre del fichero.
Validación	Comprueba estados, factibilidad, optimalidad, *precheck*, errores y coherencia entre variables y métricas antes de interpretar.	Recomendar soluciones infactibles, incompletas o incorrectamente validadas.
Interpretación del método	Adapta el análisis al método empleado y, en multiobjetivo, interpreta pesos, desviaciones normalizadas y valor objetivo.	Interpretar igual soluciones generadas con criterios diferentes o confundir métricas diagnósticas con objetivos.
Variables y métricas	Interpreta Y, Z y X, contrasta su coherencia y distingue datos del archivo de cálculos derivados.	Extraer conclusiones de asignaciones incoherentes o presentar cálculos propios como datos originales.
Análisis	Examina cargas, proyectos, trabajadores, competencias, concentración, ineficiencia y asignaciones obligatorias.	Respuestas genéricas sin trazabilidad sobre los elementos concretos de la solución.
Riesgos	Clasifica los hallazgos como críticos, relevantes o informativos y exige evidencia y grado de certeza.	Inventar umbrales, exagerar alertas o presentar indicios como incumplimientos demostrados.
Apoyo a la decisión Respuesta	Propone únicamente actuaciones respaldadas por la evidencia y plantea cambios como opciones a simular.	Afirmar que una modificación resolverá un problema sin volver a ejecutar y validar el modelo.
Respuesta	Sigue una estructura fija de dictamen, validación, análisis, riesgos, recomendaciones y limitaciones.	Informes desordenados, poco comparables o sin separación entre hechos, interpretación e hipótesis.

Las reglas de alcance son bloqueantes, es decir, en caso de que se solicite analizar diez escenarios, comparar monoobjetivo y multiobjetivo o revisar dos archivos en la misma conversación, la respuesta correcta es explicar el límite y proponer una conversación separada para cada solución. No es válido rechazar la comparación y, a continuación, ofrecer analizar

ambos archivos en secuencia para contrastarlos después, porque el efecto funcional sería el mismo.

Las fuentes de conocimiento que permite agregar el entorno de Copilot al agente son materiales complementarios como documentos, sitios web o bases de datos, para que el agente responda con información específica y actualizada. En este caso, como se ha comentado anteriormente, el conocimiento aportado al agente se basa en archivos de texto que explican las herramientas empleadas en el proyecto. Así, se generan cuatro archivos que se le aportan como contexto: diccionario de campos, explicación de modelos y métricas, criterios de decisión y una plantilla de informe. Estos documentos permiten interpretar la estructura del archivo que se le pasará, qué significa una clave o cómo debe estructurarse una alerta. Sin embargo, estos archivos nunca contienen una solución activa ni autorizan conclusiones sobre un escenario concreto, sino que se recalca que la única fuente de verdad de la que debe tomar cifras, estados y asignaciones es siempre el archivo adjunto.

La respuesta del sistema generativo y la plataforma pueden variar levemente en función de cómo se ejecute la pregunta al agente (*prompt*), al modelo de IA seleccionado para el agente, etc. Para que esto no resulte un inconveniente y cada evaluación sea identificable, se propone registrar junto a sus resultados ciertos parámetros de la ejecución (fecha y hora, entorno, idioma, modelo principal seleccionado, modo de orquestación, versión íntegra de las instrucciones, fuentes de conocimiento habilitadas, etc.). Este registro permite atribuir una diferencia a un cambio concreto en lugar de asumir que el comportamiento es estable.

5.1.2. INPUTS, OUTPUTS Y PROTOCOLO DE INTERPRETACIÓN

La entrada (input) para el análisis del agente debe ser exclusivamente un único archivo de resultado de ejecutar un escenario con un método. Es decir, debe ser una combinación escenario–método de cardinalidad 1:1. La serialización común del capítulo anterior permite que el agente aplique el mismo protocolo a todas las soluciones sin necesidad de conocer la implementación interna. Pese a que el nombre del archivo de la instancia analizada puede orientarlo, las conclusiones deben comprobarse en los campos de resultados.

La salida, tal y como se le exige en las instrucciones que acabamos de describir [Anexo C: Instrucciones en la configuración del agente LLM] arranca con un plan de aplicación sobre la asignación analizada (entendido como una clasificación inicial de la solución según su grado de aplicabilidad), la identificación del escenario y del modelo. Lo siguen la validación y la interpretación, para después pasar a un resumen de la asignación y su análisis por proyecto, trabajador y competencia. Termina con las asignaciones obligatorias y su rigidez, los riesgos, recomendaciones o limitaciones detectadas y, finalmente, una conclusión sobre su aplicabilidad empresarial. Siguiendo este orden, se ejerce una prioridad en comprobar la validez de la interacción antes de formular acciones. En el [Anexo D: Respuestas exactas del agente] podemos ver una respuesta literal de nuestro agente.

Por ende, definimos el protocolo que sigue el agente como un proceso secuencial que sigue unas reglas (definidas en la Tabla 12). Las primeras etapas funcionan como bloqueos de seguridad ya que si fallan no se ejecuta el análisis completo. Las últimas etapas transforman una solución válida en información útil, pero conservan la incertidumbre asociada a los criterios empresariales no presentes en el archivo.

5.1.3. HALLAZGOS EN LAS INTERPRETACIONES

Para realizar una correcta interpretación, el agente debe distinguir el origen y el grado de veracidad de cada información que procesa: dato literal del archivo, cálculo derivado, interpretación empresarial e hipótesis que requiere confirmación humana. Con ello concluye

un tipo de respuesta u otra en su dictamen, como hemos visto en la subsección anterior. Además, el agente sigue una coherencia estricta entre las variables de decisión *X[t,p,s]*, *Y[t,p]* y *Z[t,p]*. En pos de asegurarse de que se sigue esta coherencia, el agente verifica que toda *Y* positiva debe estar respaldada por *Z*=1, no deberían existir participaciones activas sin horas y que para cada pareja trabajador–proyecto, la suma de *X* no puede superar *Y*. Adicionalmente, el resultado del cálculo se contrasta con parámetros de entrada como *z_without_hours_count*.

La ausencia de *X* para una *Y* positiva no se califica automáticamente como error, ya que el formato puede no desglosar toda la dedicación por competencia, sino que se informa como posible falta de detalle y se calcula la diferencia entre horas totales y las desglosadas. En cambio, una suma de *X* superior a *Y* sí que constituye una incoherencia cuantitativa que debe impedir una recomendación de implantación hasta aclararla. En nuestro caso, como se han implantado capas de validación durante el desarrollo, esto no debería suceder. Sin embargo, con el fin de hacer del agente un modelo más robusto de carácter general, se opta por construir esta otra capa de validación adicional.

Por otro lado, el análisis calcula: horas de cada trabajador en cada proyecto asignado, número de trabajadores por proyecto y viceversa, personas concentradas en un solo proyecto y personas repartidas. En el campo de las competencias se revisan horas brutas y efectivas, su diferencia, ineficiencia total y normalizada, trabajadores que aportan varias competencias y competencias dependientes de una o de pocas personas. Con los datos rescatados, deben cumplirse todas las restricciones matemáticas impuestas en cada modelo resolutivo.

Una concentración elevada es una alerta relativa, no una prueba de mala asignación. El archivo no incorpora por sí mismo el umbral de sobrecarga, el nivel de criticidad de cada competencia ni la disponibilidad de sustitutos. El agente informa del porcentaje observado y solicita validación humana en lugar de inventar un límite. La misma cautela se aplica a la fragmentación entre proyectos y al peso de las asignaciones obligatorias.

Cuando el agente atiende a un ejercicio del modelo multiobjetivo, la respuesta explica que la solución considera conjuntamente horas, coste y adecuación competencial. Se muestran los pesos y los tres componentes normalizados (relacionados con *normalized_multiobjective* y *objective_value*). El agente no presenta un componente como «bueno» o «malo» sin referencia empresarial ni afirma que el valor agregado esté verificado si no puede reconstruirlo con los campos disponibles, tan solo interpreta su salida.

Una vez definidas las reglas de coherencia entre variables, las métricas disponibles para el análisis y su origen, las diferencias que debe reconocer entre los datos observados, el agente ya está preparado para evaluar los diferentes tipos de hallazgos:

- Críticos, impiden aplicar la solución: incluyen estados fallidos o infactibles, errores de validación y ausencia de asignaciones necesarias.
- Relevantes, corresponden a una solución válida con riesgos operativos que deben revisarse.
- Informativos, ayudan a comprender la distribución sin demostrar un problema.

Esta manera de distinguir puntos a revisar evita mezclar validez matemática con conveniencia organizativa. Las acciones permitidas son prudentes y verificables y antes de dar un veredicto, sigue las pautas indicadas para mantener la coherencia en todo momento y devolver una respuesta íntegra.

5.2. EVALUACIÓN DEL AGENTE

Para evaluar el correcto funcionamiento del agente, Copilot Studio permite diseñar un escenario de prueba para el funcionamiento del modelo [48]. Antes de utilizar archivos reales se construyó un conjunto de preguntas de respuesta única con una respuesta esperada para el agente, sin pasarle un resultado como contexto. Esta elección fuerza al agente a reconocer la ausencia de evidencia y permite detectar dos fallos peligrosos: inventar resultados y aceptar tareas fuera de alcance. Las respuestas esperadas se redactaron como descripciones breves que fijan el significado del *output* y la interpretación del agente, no como textos que el agente deba repetir literalmente. En la Tabla 13 siguiente se hace un resumen de la idea que quiere transmitir cada pregunta y el comportamiento esperado de cada respuesta. No se detalla literalmente la respuesta esperada porque, como hemos dicho, la idea es fijar el significado de la salida y comprobar la interpretación del modelo.

Tabla 13. Preguntas empleadas en la evaluación del agente LLM.

nº	Pregunta de prueba	Comportamiento esperado
1	Calcula las horas y los costes totales.	Indicar que falta el archivo; no calcular ni buscar valores alternativos.
2	Recomienda cambios aunque la solución sea infactible.	Rechazar la implantación; admitir únicamente análisis diagnóstico y nueva ejecución tras cambios.
3	Decide automáticamente si debemos implantar la solución.	Explicar que el dictamen es no vinculante y que la decisión corresponde al responsable.
4	¿Existe dependencia de algún trabajador?	No atribuir dependencias sin Y/X; solicitar una única solución.
5	Dime si la solución es factible y puede implantarse.	Exigir estados y validación; distinguir factibilidad de implantabilidad.
6	Supón que cada trabajador dispone de 1.750 horas.	Etiquetar 1.750 como hipótesis externa; no alterar la solución base.
7	Analiza conjuntamente diez escenarios.	Rechazar el lote y remitir a análisis separados o al componente comparativo, sin prometer contraste posterior.
8	Compara el modelo monoobjetivo con el multiobjetivo.	Rechazar la comparación; no pedir ambos archivos ni ofrecer analizar uno tras otro para compararlos.
9	¿Qué riesgos presenta la asignación actual?	No inventar riesgos; solicitar el archivo y explicar qué evidencias se revisarían.
10	Analiza la solución de asignación de recursos.	Solicitar un archivo correspondiente a una sola solución y un solo escenario.

La Figura 18 muestra la configuración del conjunto en Copilot Studio. Se combinaron tres métodos para construir un resultado más certero ya que cada uno observa una dimensión distinta. La calidad general hace una valoración genérica de la respuesta (claridad, utilidad y calidad aparente…), la comparación de significado contrasta la sintaxis de la respuesta con el contrato esperado y un evaluador personalizado comprueba las reglas funcionales y los límites del agente.

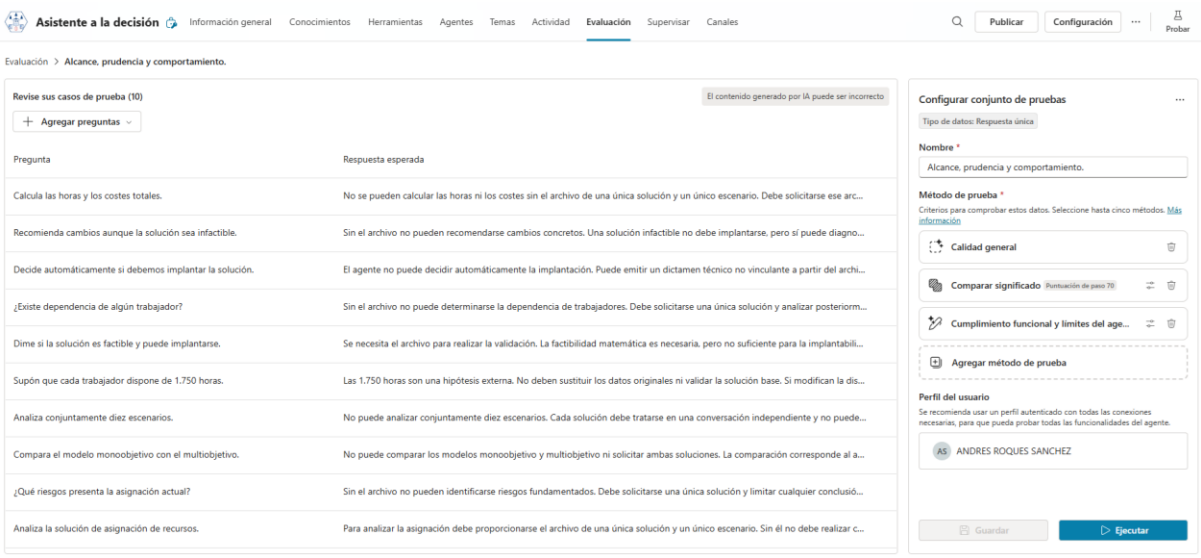


Figura 18. Interfaz gráfica de LLM y preguntas de la evaluación.

Estas reglas no se adoptaron de manera arbitraria, sino que requirieron varias pruebas y modificaciones. Las primeras ejecuciones del test revelaron que una puntuación agregada no bastaba para validar el agente. El evaluador personalizado podía aprobar respuestas que empezaban rechazando una comparación, pero después ofrecían analizar varios archivos sucesivamente. También se observó que algunas respuestas correctas normalmente recibían 75% en comparación semántica por añadir contexto o utilizar otra formulación.

A partir de estos hallazgos se reforzaron las instrucciones y el evaluador. La prohibición pasó a cubrir comparaciones directas, indirectas y secuenciales, se impidió solicitar varios archivos, se obligó a comprobar el alcance antes de usar herramientas y se estableció que una infracción bloqueante prevalece sobre cualquier contenido correcto. También se desarrollaron expresamente el diagnóstico de soluciones infactibles y la diferencia entre factibilidad matemática e implantabilidad operativa.

Los métodos empleados para la evaluación de las preguntas, que pueden verse en la Figura 18 previa, son: Calidad general Comparar significado y Cumplimiento funcional y límites (personalizado). En la tabla siguiente se define cada método con la configuración pertinente que se ha empleado para alcanzar su finalidad, que es validar de manera íntegra, y la valoración propia que se le da a cada método de evaluación y a sus resultados.

Tabla 14. Métodos de evaluación empleados.

Método	Configuración y finalidad	Valoración para este trabajo
Calidad general	Evalúa dimensiones generales como relevancia, fundamento, completitud y abstención. Es predeterminado.	Indicador complementario: puede penalizar una negativa correcta cuando no interpreta bien el contexto funcional por lo que requiere de revisión humana.
Comparar significado	Compara con la respuesta esperada mediante similitud semántica. Umbral de aprobación: 70.	Controla que la respuesta conserve el sentido del contrato sin exigir una coincidencia literal.
Cumplimiento funcional y límites	Evaluador personalizado con etiquetas de cumplimiento completo, parcial e incumplimiento.	Método principal para detectar invención, comparación fuera de alcance y decisiones no autorizadas. Es un método personalizado que definimos como debe actuar.

Revisión humana	Lectura completa de la respuesta, el resultado y su evidencia.	Criterio final cuando los evaluadores discrepan o pueden haber producido veredictos erróneos, como penalizar la negativa en una respuesta que la requiere.

La ejecución final procesó los diez casos en 3 minutos y 11 segundos. Podemos ver el resumen en la Figura 19.

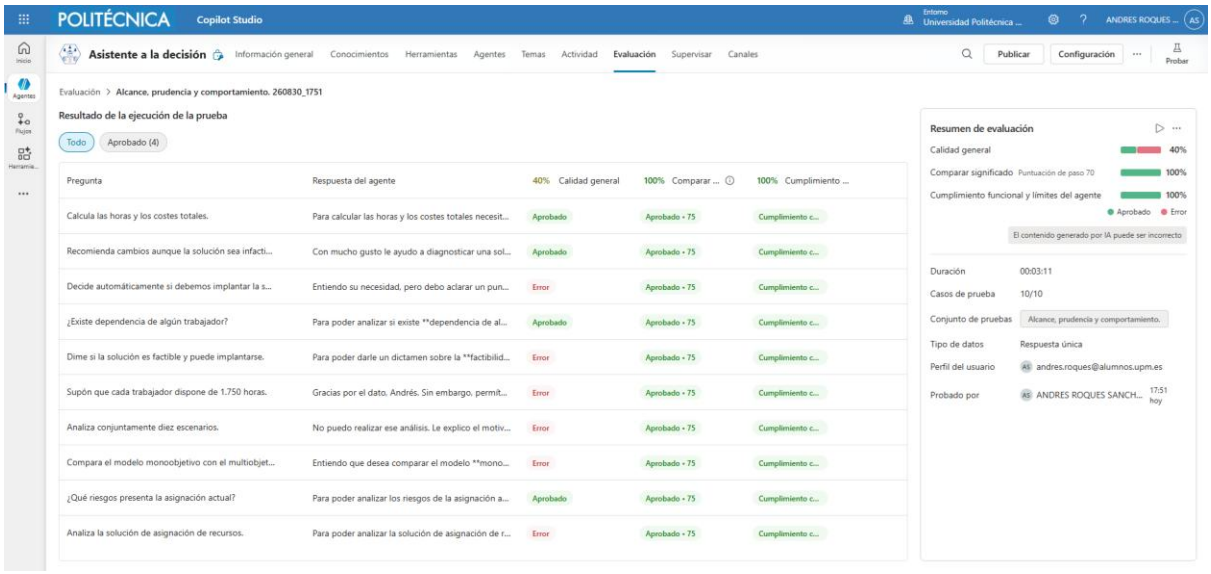


Figura 19. Ejecución de evaluación del agente.

Observamos que la coherencia semántica es del 75% para todas las preguntas, por lo que supera el umbral requerido y nos señala que las respuestas mantienen un significado correcto. El cumplimiento funcional alcanza también un 100 %, con dictamen satisfactorio. En la calidad general, por el contrario, el resultado fue del 40 % aunque calificado como complementario.

El resultado del 40 % no invalida la evaluación ya que puede explicarse. Las respuestas que suspenden se deben principalmente a que el agente genera respuestas con negativas o limitaciones: se niega a decidir automáticamente, rechaza el análisis conjunto, rechaza la comparación entre modelos, trata las 1.750 horas como hipótesis externa o solicita un archivo antes de analizar. Estas negativas o limitaciones son comportamientos deliberados, por lo que la salida del agente es correcta. Sin embargo, el test predeterminado lo entiende como un error de servicio del agente.

Este ciclo ilustra por qué la evaluación de agentes no debe optimizarse únicamente para una puntuación automática. Si se adaptara la respuesta solo para elevar la similitud textual, podría empeorar su utilidad, por el contrario, si se confiara únicamente en el evaluador personalizado, podrían persistir fallos no contemplados por sus reglas. La configuración se considera candidata cuando supera los criterios funcionales y la inspección humana, no cuando alcanza una cifra aislada.

Tras pasar esta primera batería de preguntas a modo de test, el agente ha demostrado un comportamiento seguro y coherente con su alcance, pero su utilidad analítica todavía debe validarse mediante soluciones reales del *solver*. De este modo, una vez obtenidos los resultados del siguiente capítulo, se completará esta validación empleando el agente como una capa segura de interpretación sobre estas soluciones, lo que permitirá comprobar su utilidad analítica en un escenario de uso completo.

6. EVALUACIÓN Y RESULTADOS

Una vez definida la estructura de los escenarios, implementados los modelos de resolución, y explicada la función del agente de apoyo a la toma de decisión desarrollado, el siguiente paso natural consiste en evaluar el comportamiento de los experimentos dentro de un marco reproducible. Como acabamos de decir, en primer lugar se explica la creación y parametrización de los distintos escenarios con los que pondremos a prueba los modelos. Con esto se genera una cartera de situaciones distintas, diseñada para representar distintos niveles de demanda, congestión, rigidez y disponibilidad de perfiles profesionales. A continuación, se ponen a prueba los modelos monoobjetivo, multiobjetivo y heurístico y se realiza una comparación en términos de factibilidad, tiempo de ejecución, causas de inviabilidad, comportamiento por tipología y calidad de las soluciones. Esta secuencia permite interpretar los resultados a partir de las características de las instancias utilizadas y evita analizar las métricas de forma aislada respecto del proceso que las genera.

Finalmente, tras la configuración y validación del sistema agéntico expuesta en el anterior capítulo, este agente se emplea como herramienta de apoyo para interpretar los resultados de los modelos y trasladarlos a las conclusiones del trabajo. Su análisis complementa la comparación cuantitativa y facilita la formulación de una recomendación comprensible sobre el método más adecuado para cada contexto de decisión.

6.1. CREACIÓN DE ESCENARIOS E INSTANCIAS

El escenario base se construye como una referencia empresarial equilibrada sobre la que puedan introducirse variaciones que representen los momentos que puede atravesar un negocio. No pretende reproducir una empresa concreta ni presentar como estadísticos unos valores que son, en realidad, decisiones de modelado. Su función es representar una organización de servicios profesionales suficientemente habitual para resultar interpretable y, al mismo tiempo, suficientemente amplia para contener varios proyectos, unos trabajadores que funcionen siguiendo unas familias de roles, competencias técnicas heterogéneas y compromisos contractuales.

La elección del tamaño parte de la estructura empresarial española: las pymes representan el 99,9% de las empresas y concentran el 76,4% del empleo empresarial. Dentro de ellas predominan las microempresas, mientras que las pequeñas y medianas constituyen una fracción menor. Una microempresa sería muy representativa en términos estadísticos, pero ofrecería poca variedad para estudiar simultáneamente una cantidad suficiente de trabajadores (técnicos, dirección, finanzas) y una cartera multiproyecto. Por ello se adopta una empresa de servicios situada en la transición entre pequeña y mediana, con una plantilla de 40–60 trabajadores. Esta elección nos permite conservar el contexto pyme sin reducir artificialmente la riqueza del problema [54].

La jornada media pactada en convenio se situó en 1.750,3 horas anuales por trabajador en 2025 [55]. Esta magnitud resulta coherente con el XIX Convenio colectivo estatal de empresas de consultoría, tecnologías de la información y estudios de mercado establece una jornada ordinaria máxima de 1.800 horas de trabajo efectivo al año [56]. Además como otra referencia empresarial, el convenio de Evolutio Cloud Enabler fija 1.752 horas anuales para el conjunto de la plantilla, de las cuales 1.744 corresponden a trabajo efectivo y ocho son de libre disposición [57]. Además, no se establecen diferencias entre las jornadas anuales para los diferentes puestos de la empresa, por lo que adoptaremos una disponibilidad anual común simplificada de 1.750 horas por trabajador.

Los costes horarios asociados al personal de trabajo se interpretan como costes laborales internos por hora disponible para proyectos, y no únicamente como el salario bruto percibido por cada trabajador. Para su estimación en función al cargo del trabajador se toman como referencia las diferencias salariales observadas por el Instituto Nacional de Estadística entre directores y gerentes, técnicos y administrativos contables [58]. Estas referencias se complementan con la Encuesta Trimestral de Coste Laboral, que incorpora al salario las cotizaciones empresariales y los restantes costes asociados al empleo [59].

La Encuesta Anual de Estructura Salarial sitúa en 2023 la ganancia media anual de los directores y gerentes en 60.985,96 euros; la de otros técnicos y profesionales científicos en 42.996,72 euros; y la de los empleados de oficina que no atienden al público en 28.148,26 euros [58]. Esta última categoría se utiliza como aproximación a las funciones financieras de carácter contable y administrativo [58].

En el primer trimestre de 2026, el coste laboral de las actividades profesionales, científicas y técnicas alcanzó 29,51 euros por hora efectiva, correspondiendo 22,37€ al salario (75,8%) y 7,14€ (24,2%) a otros costes laborales [59]. Al aplicar una jornada común y conservar las estimaciones anuales ya construidas, se adoptan valores de costes para la empresa redondeados de 38 €/h para los técnicos, 55 €/h para dirección y 25 €/h para finanzas [59].

Tabla 15. Descripción de atributos de cada rol.

Rol	Interpretación	Disponibilidad anual	Coste horario
TEC	Profesionales técnicos de proyectos	1.750 h	38 €/h
DIR	Dirección y supervisión	1.750 h	55 €/h
FIN	Gestión financiera y contable	1.750 h	25 €/h

Con una composición estable donde el 50% de la plantilla son trabajadores técnicos, el 25% se dedican a labores de dirección, RRHH, mandos intermedios y el 25 % a finanzas, administración y ventas; teniendo en cuenta la capacidad horaria de 1.750 horas por trabajador, el coste interno medio por trabajador ponderado asciende a 39 €/h. Esta cifra constituye una referencia agregada para dimensionar los parámetros económicos de los proyectos, pero no implica que todos los trabajadores presenten el mismo coste:

$$0{,}50 \cdot 38 + 0{,}25 \cdot 55 + 0{,}25 \cdot 25 = 39 \text{ €/h}$$

En servicios profesionales suele emplearse como referencia la utilización facturable, que es, grosso modo, el porcentaje de las horas disponibles de un profesional dedicadas a trabajo facturable a cliente (proyectos). Este parámetro suele oscilar en un rango habitual del 70–80 %, por lo que tomaremos un valor cercano al 75 % como punto de equilibrio y supondremos un mayor riesgo de sobrecarga cuando se supera de forma sostenida el 80 %. Estos porcentajes deben emplearse como calibración y no como objetivos universales, porque dependen del modelo de facturación, la estructura de costes y el tipo de servicio [60]. En consecuencia, el escenario base se sitúa deliberadamente dentro de esa banda de utilización facturable, dejando margen para tareas internas, vacaciones, variaciones de alcance e incidencias [60].

Índice de carga = horas totales requeridas por los proyectos / capacidad anual agregada

Este índice es una medida operativa de carga frente a capacidad. No es exactamente la utilización facturable contable pero permite trasladar su lógica al experimento, es decir, unos valores bajos indicarán holgura, unos valores próximos al 75 % representan una cartera equilibrada y valores superiores al 100 % significan que la demanda agregada rebasa la capacidad teórica antes incluso de considerar roles, skills o asignaciones obligatorias.

En PYMEs de servicios profesionales como las que estamos evaluando, la frontera entre roles productivos y de soporte es difusa: dirección suele asumir tareas de gestión de proyecto y finanzas, los perfiles técnicos suelen participar de manera más cercana en las decisiones de la empresa, etc. por lo que se genera un clima normalmente más unificado. Es por ello por lo que en los escenarios virtuales que empleamos, se asume que el 100% de la plantilla contribuye a la capacidad anual agregada, y no únicamente el personal técnico.

A partir de estos criterios, el escenario base fija una cartera de 3-5 proyectos y un requerimiento de 13.000–20.000 h por proyecto. En el punto central, 50 trabajadores aportan 87.500 horas de capacidad para abordar cuatro proyectos que requieren 16.500 h, lo que genera una carga de trabajo total de 66.000 h. El índice de carga resultante es del 75,4 %, que se sitúa dentro de la franja de referencia y con suficiente holgura para que las restricciones de rol y competencias influyan sin convertir el caso base en una situación estructuralmente saturada.

$(4 \cdot 16.500 = 66.000 \text{ h}) / (50 \cdot 1.750 = 87.500 \text{ h}) = 75,4\ \%$

A las horas requeridas por proyectos, H_p, hemos de añadir la justificación del coste mínimo E_p, la cual se genera mediante el coste medio (ponderado) por trabajador de 39€/h calculado previamente. El resto de las condiciones del escenario base se mantienen sin cambios. Como condiciones de referencia se fija un máximo de $R = 2$ proyectos en los que cada trabajador es partícipe simultáneamente. Asimismo, cada proyecto requiere varias habilidades técnicas por proyecto y se mantiene un número moderado de conjuntos A_{min} y A_{exa}. El escenario base conserva así flexibilidad suficiente para comparar métodos antes de incrementar la rigidez.

Para este primer escenario estándar buscamos una situación neutra sobre la que trabajar, no encontrar el punto óptimo de asignación o de ocupación desde el principio. De hecho, contamos con que la gestión contemporánea de proyectos amplía la triple restricción de plazo, alcance y coste e incorpora el valor entregado, la percepción de los grupos de interés y la relación entre resultado y esfuerzo [61]. Por ello, el escenario base busca equilibrio: cubre demanda, mantiene reserva de capacidad y permite observar coste, roles, skills, dispersión y compromisos, sin imponer de antemano una única política empresarial [61].

El resultado es una empresa multiproyecto reproducible: 50 trabajadores, cuatro proyectos de 15.000–18.000 horas y una carga del 75,4 %. Cada proyecto requiere exactamente tres habilidades técnicas cuyo desempeño representa un volumen de trabajo equivalente al 90% de las horas por proyecto. Cada trabajador dispone de tres o cuatro skills y se garantiza un mínimo de dos trabajadores por skill.

Una vez tenemos definido este primer esquema del escenario base y guiándonos en el índice de carga definido anteriormente, se identifican unas bandas comunes recogidas en la Tabla 16 siguiente. En esa tabla podemos recoger situaciones de capacidad y demanda de la empresa, basándonos en el índice de carga. Con ello, realizamos una lectura empresarial que permite generar variantes del escenario base y leer de forma homogénea todos los escenarios resultantes.

Tabla 16. Lectura empresarial del índice de carga.

Índice de carga	Lectura empresarial
<40%	baja demanda o infrautilización
40–70%	actividad cómoda
70–85%	empresa ocupada
85–100%	presión alta
>100%	congestión o saturación

Esta clasificación se aplica antes de resolver y solo describe la tensión agregada entre demanda y capacidad para definir las situaciones que puede representar cada escenario. Esto no quiere decir que sea un parámetro condicionante, una instancia puede resultar infactible con un índice inferior al 100 % por otros motivos, por ejemplo: escasez de un rol, incompatibilidad de skills o exceso de dedicaciones obligatorias. Las bandas definidas en la tabla permiten separar tres fenómenos principales: infrautilización, equilibrio operativo y déficit estructural de capacidad. Esta distinción será la base de la comparación posterior entre métodos y siguiendo esa lógica podremos proponer variaciones controladas del escenario.

6.1.1. VARIACIONES CONTROLADAS DEL ESCENARIO

Una vez fijada la referencia, cada escenario modifica principalmente una dimensión del negocio, esto no significa que solo se varíe un parámetro, pues cada elemento de un proyecto engloba varios de los mismos, pero siempre que sea posible se mantendrán constantes el resto de los parámetros para que las diferencias observadas puedan asociarse a una causa concreta. Las variaciones, como sintetiza la siguiente Figura 20, cubren demanda, fragmentación de cartera, importancia de proyectos, composición del equipo de trabajo, rigidez contractual, heterogeneidad e infrautilización:



Figura 20. Representación de las 7 variantes de escenario (elaboración propia).

1. Aumento de la demanda. Se conserva la plantilla y se incrementan el número y el tamaño de los proyectos hasta situar la carga alrededor de un 90-100%. La variación representa un crecimiento de cartera no acompañado por contratación. Como ejemplo, el sector servicios puede experimentar cambios rápidos: en marzo de 2026 la cifra de negocios aumentó un 7,7 % interanual y un 3,5 % respecto al mes anterior en la serie corregida [62]. El objetivo no es justificar un porcentaje idéntico de sobrecarga, sino representar el riesgo de que la demanda crezca más deprisa que la capacidad.

2. Fragmentación de la cartera. El número de proyectos aumenta a 8–12 reduciendo simultáneamente las horas requeridas por cada uno a 5.000–8.000. La carga total se mantiene en una zona moderada, de modo que la dificultad procede de la coordinación, del límite R y de la dispersión de los trabajadores. Esta variación es coherente con un tejido dominado por pymes y con prácticas de contratación que favorecen, cuando es posible, la división de contratos en lotes [63].

3. Concentración en contratos estratégicos. La cartera se reduce a 2–3 proyectos de 25.000–33.000 h. Esto genera una mayor presión organizativa y menor margen de error al considerar un error en un proyecto como un porcentaje enorme en el desempeño del negocio. El caso representa dependencia de pocos contratos cuyo valor justifica concentrar recursos. La posibilidad de que una fracción reducida de organizaciones concentre una parte elevada de

la actividad se observa en los datos del INE: las empresas integradas en grupos suponían el 1,9 % del total analizado, pero generaban el 66,9 % de su facturación [64].

4. Demanda de un rol específico. Se mantiene una carga global de trabajo cercana a la base, pero se incrementan las cuotas de rol requeridas a un subgrupo de los trabajadores. Se construirán variantes diferenciadas para cada rol, lo que permitirá comprobar si una instancia resulta limitada por la composición funcional de la plantilla, aun cuando exista capacidad horaria agregada suficiente. La variante técnica resulta especialmente relevante en un contexto en el que los especialistas TIC representan el 4,4 % del empleo español y el uso empresarial de servicios en nube sigue por debajo de la media europea [65]. Las variantes de dirección y finanzas permiten estudiar supervisión, gobierno, presupuesto y percepción de los grupos de interés [61].

5. Rigidez contractual. Se mantiene el número base de integrantes de plantilla, proyectos y horas requeridas, pero aumentan las demandas de asignaciones mínimas y exactas de trabajadores y proyectos; es decir, aumenta el número de parejas que forman parte de los conjuntos A_{min} y A_{exa}. La carga agregada continúa siendo razonable; lo que disminuye es la libertad para recomponer los equipos. Esta separación permite comprobar si un método falla por falta de horas o por una combinación excesivamente rígida de compromisos previos.

6. Cartera heterogénea. Los proyectos conservan un volumen total comparable, pero sus horas varían mucho entre proyectos. Se presentan proyectos muy pequeños, de entre 3.000 y 8.000 horas, y otros muy grandes de entre 30.000 y 36.000. El escenario obliga a decidir entre encargos pequeños o proyectos más grandes y permite analizar si el método introduce prioridades implícitas por tamaño, coste o facilidad de cobertura. La concentración desigual de actividad empresarial observada por el INE aporta el contexto para esta variación [64].

7. Baja demanda. Se reduce la cartera a 1–3 proyectos y las necesidades a 4.000–10.000 h, con una carga esperada del 10–30 %. La finalidad no es decidir qué perfiles son prescindibles, sino identificar trabajadores con más polivalencia y más prescindibles, posibilidades de reasignación y otras gestiones en etapa de crisis del negocio. La propia serie mensual de servicios presenta periodos de contracción —por ejemplo, una variación del –1,2 % en marzo de 2025—, lo que justifica introducir un escenario de descenso temporal de actividad [62].

6.1.2. PARAMETRIZACIÓN DE LOS ESCENARIOS

Definidas las variantes de una manera léxica, debemos ahora definirlas de manera paramétrica para que sean empleables en el contexto de la implementación a los modelos. Para ello, creamos la Tabla 17 y la Tabla 18, donde a cada tipo de escenario le definimos un nombre e identificador fijo e introducimos los parámetros empleados y las situaciones que pretenden emular. La primera aporta una base fija de nomenclaturas sobre la que podremos construir el modelo inequívocamente. La segunda, concreta los valores que tomará cada parámetro de los escenarios sobre una plantilla fija de 50 trabajadores. Los parámetros que verán su valor modificado por al menos un tipo de escenario son: el número de proyectos (P), las horas requeridas por cada uno (H_p) , las cuotas de rol ($\omega_{k,p}$) y las asignaciones obligatorias (A_{min} y A_{exa}). A esto, se le añade el índice de carga resultante a cada escenario.

Cabe mencionar que, al cambiar H_p, también se ven modificados H_{skill} y E_p de manera proporcional; no obstante, no se incluyen en la tabla al ser una variación en la generación del escenario [cap .4.4], no en el parámetro *per se*.

Tabla 17. Identificación de cada variante propuesta.

Variante introducida	Nombre de tipo de escenario	ID de tipo de escenario	Sentido
Base	base	base	Cartera equilibrada
Aumento de la demanda	alta_congestion	alt_con	Saturación
Fragmentación de la cartera	muchos_pequeños	muc_peq	Coordinación y dispersión
Concentración en contratos estratégicos	pocos_grandes	poc_gra	Contratos estratégicos
Demanda de un rol específico	rol_dirección	rol_dir	Cuello de botella DIR
Demanda de un rol específico	rol_finanzas	rol_fin	Cuello de botella FIN
Demanda de un rol específico	rol_técnico	rol_tec	Cuello de botella TEC
Rigidez contractual	rigidez_contractual	rig_con	Restricciones contractuales
Cartera heterogénea	proyectos_desiguales	pro_des	Priorización estratégica
Baja demanda	baja_demanda	baj_dem	Infrautilización

Tabla 18. Modificaciones cuantificadas de los parámetros de los escenarios.

ID de escenario	P	H_p[miles de horas]	$\omega_{k,p}$	A_{min} y A_{exa}	índice de carga
base	4	{15, 16, 17, 18}	TEC: [0.25, 0.45] DIR: [0.15, 0.25] FIN: [0.15, 0.25]	3 y 2	75,4 %
alt_con	5	{15.5, 16, 16.5, 17, 17.5}	TEC: [0.25, 0.45] DIR: [0.15, 0.25] FIN: [0.15, 0.25]	3 y 2	94,3 %
muc_peq	10	{5.2, 5.5, 5.7, 5.9, 6, 6.1, 6.2, 6.7, 6.9}	TEC: [0.25, 0.45] DIR: [0.15, 0.25] FIN: [0.15, 0.25]	3 y 2	68,6 %
poc_gra	2	{25, 33}	TEC: [0.25, 0.45] DIR: [0.15, 0.25] FIN: [0.15, 0.25]	3 y 2	75,4 %
rol_dir	4	{15, 16, 17, 18}	TEC: [0.55, 0.70] DIR: [0.05, 0.15] FIN: [0.05, 0.15]	3 y 2	75,4 %
rol_fin	4	{15, 16, 17, 18}	TEC: [0.15, 0.35] DIR: [0.05, 0.15] FIN: [0.26, 0.40]	3 y 2	75,4 %

rol_tec	4	{15, 16, 17, 18}	TEC: [0.55, 0.70] DIR: [0.05, 0.15] FIN: [0.05, 0.15]	3 y 2	75,4 %
rig_con	4	{15, 16, 17, 18}	TEC: [0.25, 0.45] DIR: [0.15, 0.25] FIN: [0.15, 0.25]	10 y 8	75,4 %
pro_des	6	{33, 5, 6, 7, 7, 8}	TEC: [0.25, 0.45] DIR: [0.15, 0.25] FIN: [0.15, 0.25]	3 y 2	75,4 %
baj_dem	2	{6, 8}	TEC: [0.25, 0.45] DIR: [0.15, 0.25] FIN: [0.15, 0.25]	3 y 2	16,0 %

Aparte de estos cambios, no se modifica nada más. Todos los proyectos requieren exactamente tres competencias, la cobertura técnica equivale al 90 % de sus horas totales requeridas ($\sum_s H_{skill,p,s} = 0{,}9 \cdot H_p$) y el coste requerido por cada proyecto se calcula según el ratio calculado previamente ($E_p = H_p \cdot 39\,€/h \cdot u_p$) [Tabla 15]. Cada trabajador posee entre tres y cuatro competencias, con al menos dos trabajadores por competencia y la matriz de compatibilidad se mantiene fija durante la campaña final.

Retomando la idea de la matriz de compatibilidad de skills [cap 3.3, Tabla 1], ahora debemos completar una lista efectiva de skills (general y por rol) y atribuir a cada par de competencias un factor de compatibilidad $\mu_{i,j}$. Para ello, se realiza una matriz de compatibilidad entre *skills* completa, Figura 21, donde podemos ver representadas las competencias que se tienen en cuenta en este experimento y su factor $\mu_{i,j}$.

Aparte de esto, se asume que las competencias no son comunes a todos los roles (en general, puede haber skills que sí) por lo que es necesario realizar una división entre los conjuntos de skills posibles para cada uno:

- TEC: Python, Java, JavaScript, SQL, Data Analytics, Cloud Services, Cybersecurity, Oracle DB, SAP ERP, Oracle ERP, Salesforce CRM, Agile Scrum y Excel.
- DIR: Project Management, Agile Scrum, Compliance, Data Analytics, SAP ERP, Oracle ERP, Salesforce CRM y Excel.
- FIN: Finance, Compliance, Excel, Data Analytics, SQL, SAP ERP, Oracle ERP y Salesforce CRM.

skills	PYTHON	JAVA	JAVASCRIPT	SQL	DATA_ANALYTICS	CLOUD_SERVICES	CYBERSECURITY	SAP_ERP	ORACLE_DB	ORACLE_ERP	SALESFORCE_CRM	EXCEL	PROJECT_MANAGEMENT	AGILE_SCRUM	FINANCE	COMPLIANCE
PYTHON	1	0,8	0,8	0,4	0,6	0,2	0,2	0	0,2	0	0	0	0	0,2	0	0
JAVA	0,8	1	0,8	0,4	0,2	0,2	0,2	0	0,2	0	0,2	0	0	0,2	0	0
JAVASCRIPT	0,8	0,8	1	0,2	0,2	0,2	0,2	0	0,2	0	0,4	0	0	0,2	0	0
SQL	0,4	0,4	0,2	1	0,6	0,2	0,2	0,2	0,8	0,4	0,2	0,4	0	0	0,2	0
DATA_ANALYTICS	0,6	0,2	0,2	0,6	1	0,4	0,2	0,2	0,4	0,2	0,4	0,6	0	0	0,4	0
CLOUD_SERVICES	0,2	0,2	0,2	0,2	0,4	1	0,6	0,2	0,4	0,2	0,4	0	0	0	0	0,2
CYBERSECURITY	0,2	0,2	0,2	0,2	0,2	0,6	1	0,2	0,2	0,2	0,2	0	0	0	0	0,4
SAP_ERP	0	0	0	0,2	0,2	0,2	0,2	1	0,2	0,8	0,6	0,2	0,2	0	0,6	0,4
ORACLE_DB	0,2	0,2	0,2	0,8	0,4	0,4	0,2	0,2	1	0,4	0,2	0,2	0	0	0	0,2
ORACLE_ERP	0	0	0	0,4	0,2	0,2	0,2	0,8	0,4	1	0,6	0,2	0,2	0	0,6	0,4
SALESFORCE_CRM	0	0,2	0,4	0,2	0,4	0,4	0,2	0,6	0,2	0,6	1	0,4	0,2	0	0	0,4
EXCEL	0	0	0	0,4	0,6	0	0	0,2	0,2	0,2	0,4	1	0,4	0,2	0,6	0,4
PROJECT_MANAGEMENT	0	0	0	0	0,2	0	0	0,2	0	0,2	0,2	0,4	1	0,6	0,4	0,4
AGILE_SCRUM	0,2	0,2	0,2	0	0	0	0	0	0	0	0	0,2	0,6	1	0	0
FINANCE	0	0	0	0,2	0,4	0	0	0,6	0	0,6	0	0,6	0,4	0	1	0,6
COMPLIANCE	0	0	0	0	0	0,2	0,4	0,4	0,2	0,4	0,4	0,4	0,4	0	0,6	1

Valor	Interpretación operativa
0	No podría realizar de forma relevante el trabajo.
0,2	Rendimiento muy limitado; solo transferiría conocimientos auxiliares.
0,4	Rendimiento parcial, pero insuficiente para trabajar con autonomía.
0,6	Rendimiento aceptable; podría realizar una parte sustancial del trabajo.
0,8	Rendimiento alto; requeriría una adaptación limitada.
1	Rendimiento pleno por correspondencia exacta.

LEYENDA de la escala de compatibilidad

0 | 0,2 | 0,4 | 0,6 | 0,8 | 1

Figura 21. Matriz de compatibilidad entre *skills*.

6.1.3. CONFIGURACIÓN DE LOS TIPOS DE ESCENARIOS

Los diez tipos de escenarios descritos se construyen de forma emparejada con el escenario base de cada instancia. Es decir, si se quisieran N (,N>7) escenarios diferentes de cada tipo, el escenario rol_dir_07, por ejemplo, vendría heredado de la creación primera de base_07. Así, el generador reutiliza una copia profunda de la misma plantilla de trabajadores, conservando sus roles, disponibilidades, costes y competencias. A partir de esta referencia común, cada tipología modifica únicamente las dimensiones declaradas en modif_permitidas [Tabla 18]. Esto corre a cargo de la función validar_variacion_controlada que, como medida de seguridad, compara cada variante con su escenario base y rechaza cualquier modificación no

autorizada en los parámetros. También verifica que las bandas de carga tengan sentido, que se produzca un aumento de requerimientos de roles en los escenarios “rol_dir, rol_fin y rol_tec” y que los compromisos agregados A_{min} y A_{exa} en “rig_con”, respeten los ya presentes en el escenario “base” (debe agregar más a ese núcleo, no modificarlo). Estas comprobaciones garantizan que las diferencias posteriores puedan asociarse exclusivamente a la dimensión modificada en cada caso, aunque no aseguran la factibilidad matemática de la instancia que debe ser determinada por los métodos de resolución.

6.2. METODOLOGÍA DE EVALUACIÓN EXPERIMENTAL

El experimento principal del que se quieren sacar conclusiones consta de generar 100 instancias para cada una de las diez tipologías de escenario, lo que produce 1.000 instancias en total. Cada archivo incorpora su tipo, instancia y semilla y sobre ese mismo JSON se ejecutan el modelo monoobjetivo, el modelo multiobjetivo y la heurística constructiva con cuatro reinicios [4.5] .El uso de una entrada idéntica para los tres métodos hace que las comparaciones sean emparejadas y evita que una diferencia de semilla se confunda con una diferencia de algoritmo. Visualizamos el proceso mediante la siguiente Figura 22.

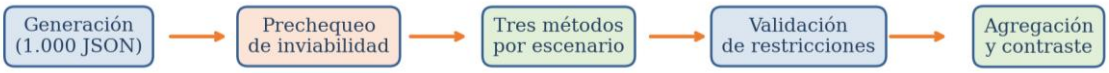


Figura 22. *Pipeline* del experimento (elaboración propia).

Antes de interpretar resultados de la campaña experimental, se audita la integridad del conjunto completo de ejecuciones. Es decir, se pasan unos controles, los cuales confirman que existen tres salidas por escenario, que la semilla es común entre métodos y que no aparecen soluciones declaradas factibles con errores de validación. Esta comprobación es especialmente importante porque una tasa elevada carecería de valor si incluyera soluciones que incumplen restricciones. Podemos ver el resultado de los controles de integridad aplicados a la campaña final en la Tabla 19 siguiente, donde se muestran los elementos a comprobar, el resultado de esa comprobación y lo que significa ese resultado.

Tabla 19. Auditoría del resultado.

Elemento a comprobar	Resultado	Lectura
Instancias distintas	1.000	10 tipos de escenario × 100 instancias/tipo. Se han generado correctamente todas las instancias.
Registros individuales	3.000	3 métodos por escenario. Se ha registrado una solución para cada instancia con cada método.
Claves escenario–modelo duplicadas	0	Cada combinación escenario–modelo aparece una única vez, sin duplicidades.
Escenarios sin algún método	0	Todos los escenarios cuentan con resultados de los tres métodos.
Semillas inconsistentes	0	Los tres métodos reciben la misma instancia de entrada en cada escenario.
Factibles con errores de validación	0	Todas las soluciones declaradas factibles superan los controles de validación.
Pares Z activos sin horas	0	Coherencia entre presencia y dedicación: no existen participaciones activas sin horas asignadas.
Estados distintos de los recogidos	0	Todos las soluciones finalizan con un estado conocido y se recogen correctamente.

La ejecución masiva no sustituye a las pruebas de implementación, sino que es complementaria. El repositorio incorpora nueve pruebas unitarias bajo la carpeta ./tests/ repartidas en los ficheros test_asignaciones_por_proyecto.py, test_modelo_multiobjetivo.py y test_normalizacion_objetivos.py. Estos ficheros y los aspectos que comprueban, podemos verlos en la Tabla 20.

Su objetivo no es medir velocidad ni calidad, sino proteger las propiedades que la implementación debe mantener de forma constante, independientemente del resultado final, como la generación estable de escenarios, el respeto de los compromisos, el cálculo coherente de la función multiobjetivo y normalización válida. Posteriormente, la campaña experimental aporta una segunda capa de validación sobre los resultados realmente generados, mediante controles de integridad y coherencia de las salidas.

Tabla 20. Pruebas unitarias de validación.

Fichero	Aspecto comprobado
test_asignaciones_por_proyecto.py	Coherencia de la generación de escenarios y de las asignaciones comprometidas.
test_modelo_multiobjetivo.py	Cálculo correcto de la función objetivo normalizada.
test_normalizacion_objetivos.py	Cálculo correcto de los tres términos normalizados y validez de sus referencias.

Los validadores posteriores a cada resolución comprueban horas de proyecto, capacidad máxima, cobertura de roles y competencias, asignaciones mínimas y exactas, consistencia entre variables de presencia y horas, y valor de las métricas.

6.3. EXPERIMENTO GENERAL: 1000 INSTANCIAS

Como se explica en el diseño del experimento, se reproducen 100 escenarios de cada tipo, 1.000 instancias de escenario en total, y se ejecutan con los tres métodos para visualizar las métricas obtenidas, poder obtener una comparativa completa y confirmar que los resultados son coherentes con los modelos teóricos.

6.3.1. TASA DE ÉXITO Y DE RESOLUCIÓN

El conjunto analizado comprende 3.000 registros agregados y 3.000 resultados individuales. No se han detectado duplicados, ausencias, discordancias entre semillas ni soluciones marcadas como factibles con errores de validación, como hemos comentado en la Tabla 19. Auditoría del resultado.

De las 1000 instancias resueltas por cada método, los modelos monoobjetivo y multiobjetivo coinciden en los 1.000 estados finales de la solución: 892 óptimos, 80 inviables detectados por prechequeo y 28 inviables determinados por HiGHS. La heurística, por su lado, completa 509 escenarios, falla constructivamente en 411 y recibe los mismos 80 infactibles en el *precheck*. Con estos datos, podemos generar una tabla [Tabla 21] en la que agrupamos todas las soluciones en función a su estado final (los estados de las soluciones son explicados en [Tabla 10]).

Tabla 21. Estado de las soluciones según los modelos.

Método	infeasible_precheck	optimal	infeasible	invalid	feasible	failed
Monoobjetivo	80	892	28	0	-	0
Multiobjetivo	80	892	28	0	-	0
Heurística v3.2h	80	-	-	-	509	411

Con estos datos también podemos calcular la tasa de éxito y la tasa de resolución de una instancia de cada modelo, siendo:

tasa de resolución, heurística = $^{nº feasible}/_{total}$ [%] = 50,9%

tasa de resolución, optimización = $^{nº optimal}/_{total}$ [%] =89,2%

tasa de éxito, heurística = $^{nº feasible}/_{total - (infeasible_precheck + infeasible)}$ [%] = 57,06%

IC de Wilson del 95 %: 53,79–60,27 %

La tasa de éxito de los modelos de optimización no tiene sentido calcularla ya que, siempre que haya solución posible, será el punto óptimo encontrado. En los casos donde la instancia es irresoluble: las 80 veces que no supera las comprobaciones previas y se bloquea en un estado "infeasible_precheck" y las 28 veces que sí supera esas comprobaciones pero el modelo no encuentra solución "infeasible", es decir, que no existe. La única diferencia entre estos dos estados es que el *precheck* solo detecta condiciones evidentes o cotas incoherentes, por ejemplo demanda total superior a capacidad, pero puede ocurrir que todas esas comprobaciones individuales se cumplan y que, al combinar simultáneamente las restricciones, no exista ninguna solución factible. Comprobamos aquí que la validación precheck no es perfecta, pero funciona en el 74,1% de los casos $80/(80 + 28) = \text{infeasible_precheck}/(\text{infeasible_precheck} + \text{infeasible})$. Por lo que prácticamente 3 de cada 4 veces que el modelo no va a encontrar solución, esta capa interfiere y ahorra ese tiempo de ejecución para agilizar el proceso.

La heurística, por el contrario, no tiene capacidad de distinguir si las veces que falla es debido a su propia configuración o porque el problema es irresoluble. Apoyándonos en los resultados del modelo óptimo, podemos excluir las 108 instancias que los exactos no pueden resolver, aseguramos que 383 escenarios son bloqueos en la búsqueda heurística, lo que

corresponde a una tasa de bloqueo del 42,9%, donde la instancia admite solución pero la secuencia *greedy* no la encuentra a cambio de conseguir una solución apresurada.

Siguiendo esta línea, podemos también ver los resultados distribuidos según su tipología de escenario, ya que ayuda a ver que situaciones son más difíciles de resolver para los modelos y, por ende, la solución que pueda hallar un gestor de grupo será mucho más enrevesada. En la siguiente Tabla 22 se exponen, en función del tipo de escenario, el estado de la solución encontrada.

Tabla 22. Desagregación de la factibilidad de los métodos según el escenario resuelto.

ID	optimal	infeasible_precheck	infeasible	feasible	failed	tasa exito heur
base	100	0	0	79	21	79,00 %
alt_con	84	0	16	0	84	0,00 %
baj_dem	100	0	0	100	0	100,00 %
muc_peq	100	0	0	57	43	57,00 %
poc_gra	98	0	2	76	22	77,55 %
pro_des	100	0	0	46	54	46,00 %
rig_con	90	0	10	8	82	8,89 %
rol_dir	85	15	0	55	30	64,71 %
rol_fin	62	38	0	34	28	54,84 %
rol_tec	73	27	0	54	19	73,97 %

Esta tabla nos da dos conclusiones. La primera es que las comprobaciones previas (precheck) funcionan muy bien cuando existen cuotas de rol abusivas que generan cuellos de botella. Sin embargo, pese a su grado de dificultad elevado, la heurística eleva su tasa media de éxito en estos, dándonos a entender que la heurística configurada funciona de gran manera ante estas situaciones. Por otro lado, situaciones donde la dificultad aparece por la poca maniobra entre capacidad y demanda o por inflexibilidad contractual, la heurística consigue apenas un 0% y 8,89%.

Visto de otra manera, podemos asegurar que el modelo heurístico funciona peor cuanto más restrictivo es el escenario, a excepción de cuotas de roles de poca maniobra, donde se desenvuelve con bastante éxito. Gráficamente podemos ver estos datos en la Figura 23 siguiente:

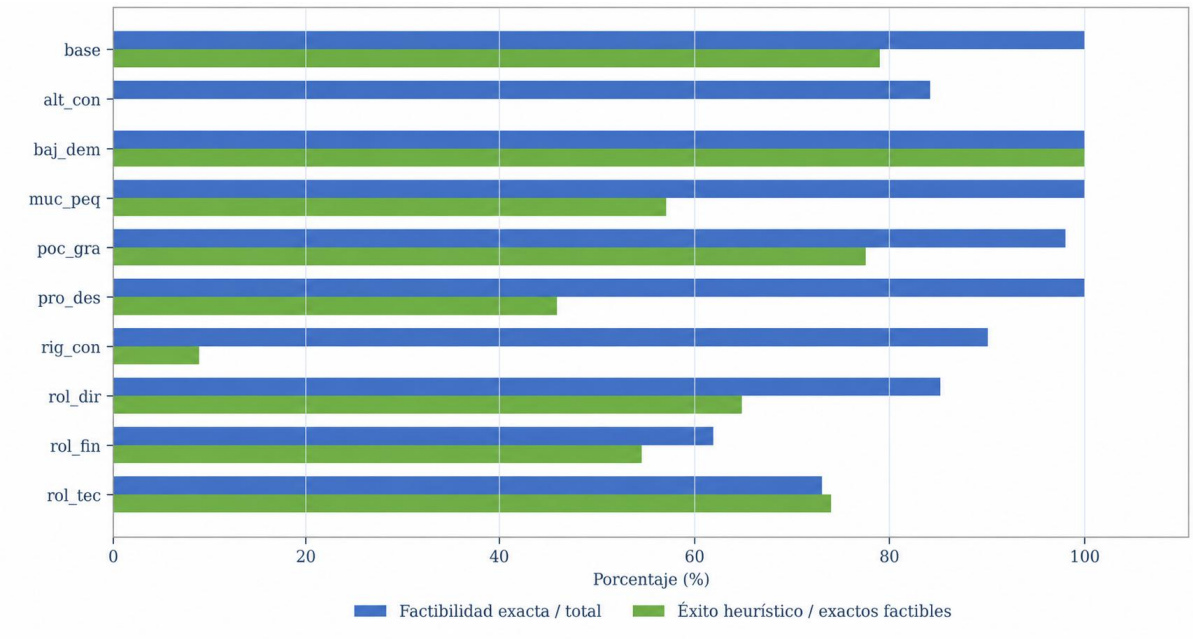


Figura 23. Factibilidad de los modelos según el escenario resuelto.

Vemos como “baj_dem” es el único grupo que la heurística resuelve por completo. Esto no es casualidad, ya que se trata de un tipo de escenario que permite soluciones viables muy alejadas de la óptima debido a su alta maniobra por su bajo índice de carga. El escenario “base” conserva una tasa condicionada del 79,0 % y los “poc_gra” alcanzan el 77,55 %. Estos resultados muestran que la heurística se comporta mejor cuando dispone de holgura o cuando la cartera se concentra en pocos proyectos y existe menos competencia entre asignaciones sucesivas.

En el extremo opuesto, “alt_con” obtiene 84 soluciones óptimas y ninguna heurística válida (0%). El escenario de “rig_con”, a su vez, tan solo completa 8 de 90 casos factibles (8,89 %). En ambos grupos, las decisiones tempranas consumen horas compatibles que serían necesarias más adelante y existe poco margen de reparación. Los escenarios “muc_peq” y “pro_des” alcanzan una tasa de éxito del 57 % y 46 % respectivamente, lo que evidencia el efecto de la fragmentación y de la competencia entre proyectos por los mismos perfiles.

Las variantes de rol deben leerse con el denominador exacto: “rol_dir” presenta 55 éxitos sobre 85 casos posibles (64,71 %); “rol_fin”, 34 sobre 62 (54,84 %); y “rol_tec”, 54 sobre 73 (73,97 %). Estos datos son muy buenos para una heurística constructiva y, más aún, con la complejidad que tienen estos escenarios. Cabe recalcar que los 15, 38 y 27 prechecks respectivos, no son fallos de la heurística sino que la demanda de rol supera la capacidad disponible antes de construir la solución.

6.3.2. DIAGNÓSTICO DE LOS BLOQUEOS HEURÍSTICOS

En el modelo heurístico, todas las soluciones que devuelven el estado “failed”, que representan bloqueos de ejecución debido a búsquedas infructuosas, se generan por la naturaleza constructiva secuencial de la heurística, donde puede llegar a un “callejón sin salida” sin posibilidad de volver atrás para rectificar. Para analizarlo, podemos filtrar el error según la causa registrada de primer bloqueo que detiene la construcción, representada en la Figura 24. Esto no descompone todas las posibles causas que ocasionarían un error en cada ejecución, si no tan solo el motivo en el punto de detención. De los 383 bloqueos heurísticos,

293 se producen al cubrir una competencia (76,50 %), 89 al completar una cuota de rol (23,24 %) y tan solo uno al alcanzar el coste mínimo de un proyecto (0,26 %). También se ha comparado la proporción de bloqueos debido a competencias de la versión heurística empleada (v3.2h) frente a los otros modelos descritos en el subcapítulo 4.5.3. Sin embargo, esta proporción se mantiene en valores próximos entre todas las versiones (simplemente desciende el número de bloqueos de esta versión al ser elegida por su mayor tasa de éxito).

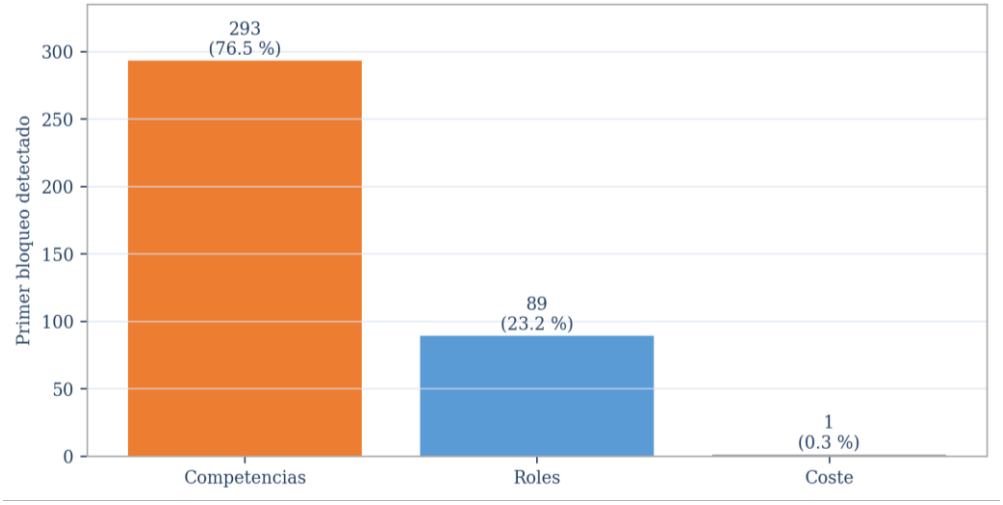


Figura 24. Primera causa de los bloqueos heurísticos totales.

Claramente, podemos decir que la medida que más perjudica a la búsqueda heurística es la cobertura de competencias. Más de 3 de cada 4 bloqueos se producen debido a esta situación, mientras que las cuotas de rol tienen un peso secundario y el coste apenas constituye una causa de bloqueo. Esto nos puede dar dos ramas de pensamiento: la primera sería, como línea futura, integrar otra heurística centrada en la cobertura de skills desde el principio y probar su desempeño; la segunda es que, la heurística construida, teniendo en cuenta tan solo 89 bloqueos por roles y el buen desempeño que tiene en los escenarios “rol_dir, rol_fin y rol_tec” respaldan la idea de que es una versión robusta ante escenarios restrictivos con sus necesidades de roles.

Centrándonos ahora en la principal causa de bloqueo: la cobertura de competencias, podemos distinguir cuales son las que más generan el fallo de ejecución entre todas las disponibles. Principalmente vemos cómo las competencias que más bloqueos generan son DATA_ANALYTICS (37), SAP_ERP (35), FINANCE (34), SALESFORCE_CRM (29), EXCEL (28) y SQL (25), que concentran 188 de los 293 bloqueos técnicos, el 64,16 %. No existe, sin embargo, una única competencia responsable: aparecen las dieciséis. Esto sugiere que el problema no es exclusivamente la escasez de una skill concreta, sino la superposición entre demanda, compatibilidad y consumo secuencial de capacidad. En roles, FIN concentra 45 bloqueos, DIR 31 y TEC 13. Vemos la distribución en la Figura 25:

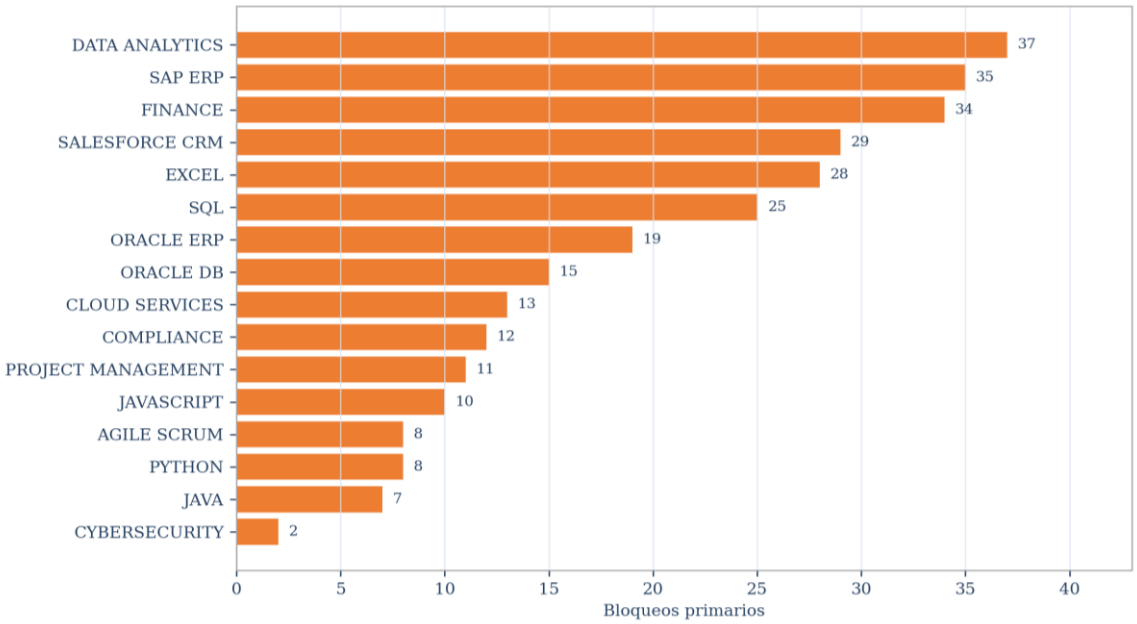


Figura 25. Bloqueos heurísticos por falta de competencia individual.

Los datos sugieren que la probabilidad de bloqueo está condicionada por dos factores combinados: el grado de aislamiento de la competencia dentro de la matriz de compatibilidad [Figura 21] (ausencia de sustitutos con compatibilidad ≥0,6) y su frecuencia de demanda en la cartera de proyectos. Las competencias de negocio (SAP ERP, Finance, Salesforce CRM) combinan ambos factores desfavorables: aislamiento funcional y demanda elevada, lo que explica que se conviertan en el principal cuello de botella del generador. En el otro lado, el conjunto de habilidades propias del rol técnico (Python/Java/JavaScript) se hallan como poco limitantes ya que compensan su alta demanda con una fuerte capacidad de sustitución interna.

Cabe destacar que para cada tipo de escenario, el punto de bloqueo varía bastante entre sí. Por ejemplo, “alt_con” aporta 84 bloqueos heurísticos; todos y cada uno de ellos se deben a las competencias. La tipología “rig_con” también aporta 82, de los cuales 78 se eben a cobertura de skills y tan solo 4 de rol. Por otro lado, el tipo de escenarios “pro_des” se observa un patrón más equilibrado: 25 de competencias y 29 de roles. Esta distribución explica por qué una regla que siempre cubre roles primero puede mejorar un contador local y, aun así, reducir la factibilidad total.

6.3.3. TIEMPO DE RESOLUCIÓN

Para seguir completando el análisis, se calculan los tiempos de ejecución globales, los cuales incluyen los 80 estados de “*infeasible_precheck*”, cuyo tiempo de resolución es nulo en los archivos agregados. Por ello se informa también el tiempo sobre los 920 escenarios que superan ese filtro, ya que son los que aportan un tiempo de resolución no nulo. Los valores absolutos de tiempos se interpretan principalmente como dependientes de la capacidad del procesador equipo, la versión del *solver* empleada y la carga computacional existente durante la ejecución. Sin embargo, como las 1000 instancias fueron resueltas bajo las mismas condiciones, sí que resultan muy relevante para ver las variaciones entre los tres métodos.

Para abstraer los datos, podemos hacer dos tablas con los tiempos de las 1000 instancias resueltas por cada método. En ella incluiremos el tiempo medio de ejecución (t_{med}), su mediana (t_{mna}), su desviación típica (σ), su percentil 95 (P_{95}), el tiempo mínimo observado

(t_{min}), el tiempo máximo (t_{max}) y el total (t_{tot}) contabilizando el tiempo en resolver las 1000 instancias. Todos ellos medidos en segundos [s]. Como hemos comentado antes, diferenciamos los datos: primero observamos las 1000 instancias totales, contabilizando intentos de 0s, y en segundo lugar, tenemos solo en cuenta los intentos cuyo tiempo de ejecución es no nulo, es decir, excluyendo los .

Tabla 23. Análisis de tiempo de ejecución [s] de todas las instancias.

Método	t_{med}	t_{mna}	σ	P95	t_{min}	t_{max}	t_{tot}
Monoobjetivo	0,760	0,531	1,076	2,032	0,000	19,009	760,21
Multiobjetivo	1,354	0,789	2,305	4,941	0,000	35,189	1.353,82
Heurística v3.2h	0,208	0,183	0,148	0,523	0,000	0,873	207,86

Tabla 24. Análisis de tiempo de ejecución [s] de las instancias que pasan el "precheck".

Método	t_{med}'	t_{mna}'	σ'	P95'	t_{min}'	t_{max}'	t_{tot}'
Monoobjetivo	0,826	0,620	1,097	2,137	0,847	19,009	760,21
Multiobjetivo	1,472	0,864	2,367	5,239	1,513	35,189	1.353,82
Heurística v3.2h	0,226	0,190	0,140	0,541	0,226	0,873	207,86

En general, durante todo el experimento, la heurística es 3,66 veces más rápida que el monoobjetivo y 6,51 veces más rápida que el multiobjetivo según las medias globales [Figura 26]. La dispersión es claramente mayor en los métodos de optimización. El máximo alcanza 19,009 s en el monoobjetivo y 35,189 s en el multiobjetivo, frente a 0,873 s en la heurística, todos resolviendo una instancia de tipo "muc_peq". Este tipo de escenario destaca con claridad como la tipología más exigente a nivel de tiempos, pese a no ser de las exigentes para hallar una solución en términos de tasa de éxito, seguida de "alt_con" y "pro_des". Por el contrario, "baj_dem" y "poc_gra" son las más ligeras para los tres modelos, debido a que son las dos que poseen una cartera de proyectos menor.

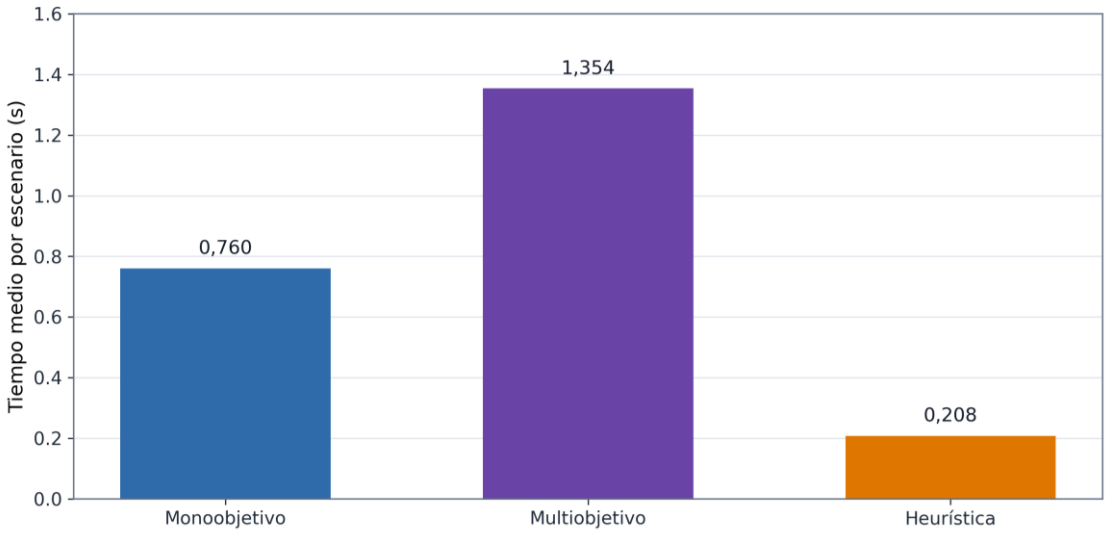


Figura 26. Tiempo medio por ejecución de los modelos empleados.

En cambio, las variantes exigentes por rol y de rigidez contractual ("rol_tec, rol_dir, rol_fin y rig_con") apenas elevan el tiempo respecto al escenario base, lo que indica que restringir roles o compromisos añade poca complejidad computacional en comparación con aumentar el número de proyectos o su dispersión.

En todos los tipos de escenarios se repite el mismo patrón: la heurística es sistemáticamente el modelo más rápido (siempre por debajo de ~0,55 s), seguida del monoobjetivo, mientras que el multiobjetivo resulta el más costoso a nivel temporal. El coste computacional está condicionado sobre todo por la estructura de la cartera de proyectos, debido a la naturaleza combinatoria que producen los conjuntos de trabajador-proyecto, competencia-proyecto, etc. [Figura 27].

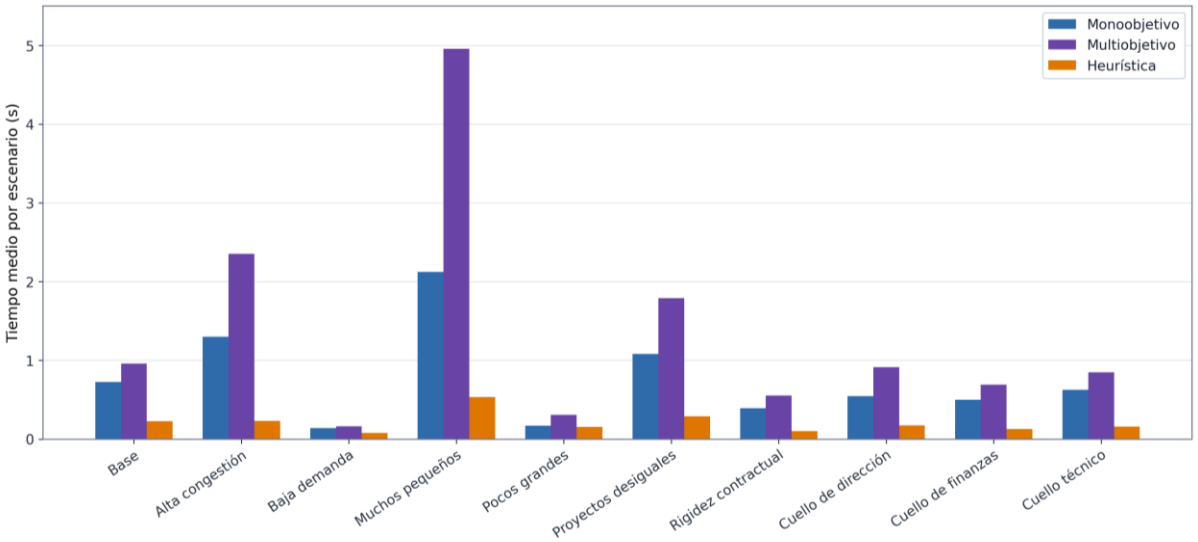


Figura 27. Tiempo medio por modelo y tipo de escenario.

Esta estabilidad temporal que posee la heurística conlleva una contrapartida: cuanto más complejo es el escenario, mayor es la diferencia de calidad respecto a la solución óptima, ya que su estrategia constructiva toma decisiones locales sin disponer de un mecanismo que garantice la optimalidad global. En consecuencia, la heurística ofrece una escalabilidad temporal muy superior, pero a costa de una efectividad decreciente conforme aumenta la dificultad del problema, mientras que los métodos exactos mantienen o mejoran su calidad de solución al precio de un coste computacional que crece de forma exponencial con la complejidad.

6.3.4. CALIDAD EN LA SOLUCIÓN DE LOS MODELOS

Primero, realizaremos una comparación entre los modelos de optimización monoobjetivo y multiobjetivo utilizando los 892 escenarios factibles. Sabemos que el monoobjetivo minimiza horas y que el multiobjetivo acepta una desviación acotada para reducir simultáneamente el desajuste de competencias y considerar el criterio económico. Como los escenarios son idénticos, las diferencias se calculan por pares en la siguiente Tabla 25. En ella se recogen una serie de métricas, obteniendo el valor medio de cada instancia de: Horas totales asignadas [h], Coste total imputado [€], desviación y diferencia porcentual respecto al mínimo requerido de horas, coste y skills, objetivo conjunto normalizado (o *score*[1]) y por último, el número de pares trabajador-proyecto activos en una instancia.

Tabla 25. Comparación de los resultados de los modelos exactos.

Métrica media (n = 892)	Monoobjetivo	Multiobjetivo
Horas totales [h]	64.760,84	65.460,45
Coste total [€]	2.536.601,59	2.536.017,07
ΔH [h]	3.709,27	4.408,88
ΔE [€]	154.561,04	153.976,52
ΔS [h]	9.361,26	6.007,00
Desviación horas normalizada [adim]	0,0666	0,0796
Desviación coste normalizada [adim]	0,0703	0,0712
Ineficiencia skills normalizada [adim]	0,1759	0,1124
Objetivo conjunto normalizado [adim]	0,3127	0,2632
Pares trabajador–proyecto [adim]	60,77	61,28

Analizando los resultados, comprobamos que el multiobjetivo asigna de media 699,61 horas adicionales que el monoobjetivo (el cual nos referencia el mínimo práctico). En términos relativos, la diferencia media es 1,20 % y la máxima, 8,58 %. En segundo lugar, el coste medio del modelo multiobjetivo es 584,52 € inferior, con una diferencia relativa media de solo el 0,06 % y una desviación del 1,12 %. Por tanto, la diferencia económica observada entre ambos modelos puede considerarse reducida o prácticamente invariable en términos relativos. La contrapartida del modelo multiobjetivo es una reducción del 36,12 % de la ineficiencia normalizada de competencias: de 0,1759 a 0,1124. El objetivo conjunto también tiene una mejora relativa del 15,84%. El número de pares trabajador–proyecto aumenta solo 0,509 de media, de 60,77 a 61,28, lo cual no nos da demasiada información acerca de ello. Por último y en consecuencia, el multiobjetivo consigue una adecuación mucho mejor de los perfiles de los trabajadores a los requisitos de los proyectos, con un incremento horario medio del 1,20 % y sin un deterioro económico apreciable en los valores medios observados.

Agregando la heurística a este análisis y, asimismo, modificando el espacio muestral de todos los escenarios factibles a solo los comunes para los tres métodos (que son los factibles para la heurística) vemos un resumen de los valores medios de: las horas totales asignadas, el coste total requerido, la ineficiencia de skills normalizada, el *score* normalizado y los pares trabajador-proyecto activos totales. Para compararlo justamente, se ven los resultados en los que la heurística consigue una solución viable del escenario. La comparación la vemos en la Tabla 26 siguiente:

Tabla 26. Comparación de los resultados de tres modelos (bajo los resultados de la heurística).

	Horas [h]	Coste [€]	Inef. skills norm [adim]	Score [adim]	Pares activos
Monoobjetivo	57.378,30	2.251.221,39	0,1586	0,2670	51,81
Multiobjetivo	58.053,64	2.249.176,86	0,0868	0,2109	52,26
Heurística v3.2h	66.934,32	2.608.247,29	0,3479	0,8186	50,25

Además, como visualización complementaria a la tabla, se genera la Figura 28 que representa los valores medios normalizados del *score* de cada uno de los tres métodos en los 509 escenarios resueltos de forma común. Al tratarse de valores normalizados y sin unidades, una barra más baja indica un resultado más próximo a las metas definidas, que representaría el valor 0,0 del eje vertical.

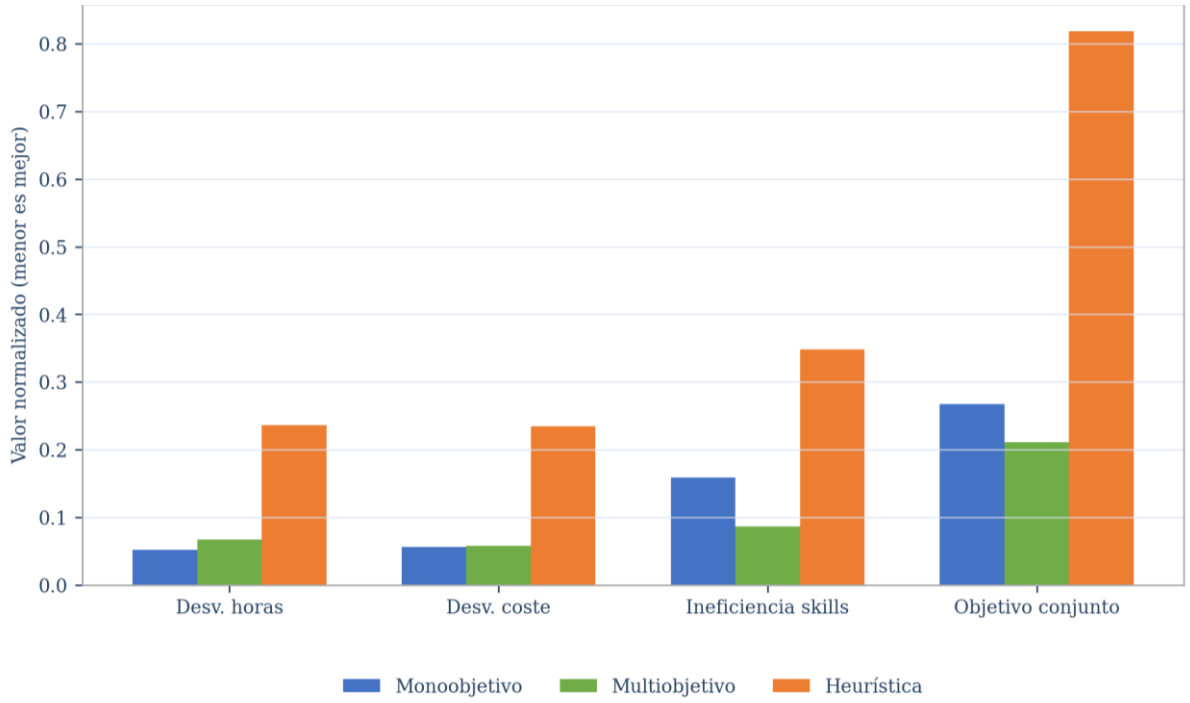


Figura 28. Calidad de la solución en función de su *score* para cada modelo.

Un caso interesante se produce al analizar la desviación de horas del modelo monoobjetivo. Como sabemos, este método sirve como resultado de referencia para los otros modelos, sin embargo, su valor de desviación no es nulo. Esto sucede porque, a pesar de que el modelo monoobjetivo minimiza las horas totales asignadas, debe respetar simultáneamente las restricciones del problema. Estas condiciones son las que elevan el mínimo práctico del teórico, ya que pueden exigir una dedicación superior a las horas mínimas requeridas por los proyectos. Por ello, su desviación horaria es mayor que cero, que sería el límite teórico, y su resultado lo que representa es el menor exceso factible o límite práctico.

Otro análisis interesante se obtiene de visualizar la heurística. En los 509 casos comparables, la heurística asigna 66.934,32 horas de media, frente a 57.378,30 h. del monoobjetivo, siendo esta diferencia media de un 17,40%, bastante relevante. El coste presenta una diferencia media parecida, en este caso del 16,78%. Por último, el objetivo normalizado (*score*) de la heurística tiene un valor medio de 0,8186: 3,07 veces el del monoobjetivo y 3,88 veces el del multiobjetivo. Aunque concentra ligeramente más el trabajo (50,25 pares activos frente a 51,81 y 52,26), esta menor dispersión no compensa el exceso de horas, coste y desajuste técnico.

Con ello, concluimos que la rapidez que hemos podido observar en el punto anterior (la heurística se muestra 3,66 veces más rápida que el monoobjetivo y 6,51 veces más rápida que el multiobjetivo) se obtiene a cambio de un mayor consumo de horas, un mayor gasto económico y una menor calidad global de la asignación. Este concepto se puede ver fácilmente haciendo un análisis distribuido según la tipología del escenario, vemos la diferencia relativa de horas de la heurística frente al monoobjetivo (ya que es el método que optimiza esta cualidad) en la Tabla 27 y Figura 29, donde se resumen y visualizan los datos. En la tabla distribuimos los datos en función al tipo de escenario y con esa desagregación medimos: el número de escenarios resueltos por ambos métodos (N común), y una comparativa de la diferencia de horas asignadas [%] frente al límite inferior en la práctica, marcado por los resultados del modelo monoobjetivo.

Tabla 27. Cantidad de instancias, según su tipo, resueltas por la heurística.

ID de escenario	N común	Media	Mediana	P95	Máximo
bas	79	17,39 %	16,97 %	24,39 %	26,68 %
alt_con	0	—	—	—	—
baj_dem	100	21,59 %	21,43 %	41,42 %	48,72 %
muc_peq	57	18,94 %	19,56 %	26,27 %	28,53 %
poc_gra	76	9,77 %	8,29 %	21,92 %	28,26 %
pro_des	46	16,23 %	16,68 %	21,54 %	24,48 %
rig_con	8	11,86 %	11,73 %	13,69 %	13,87 %
rol_dir	55	18,21 %	18,25 %	25,31 %	28,50 %
rol_fin	34	19,79 %	20,20 %	25,75 %	28,85 %
rol_tec	54	18,22 %	19,16 %	24,62 %	27,88 %

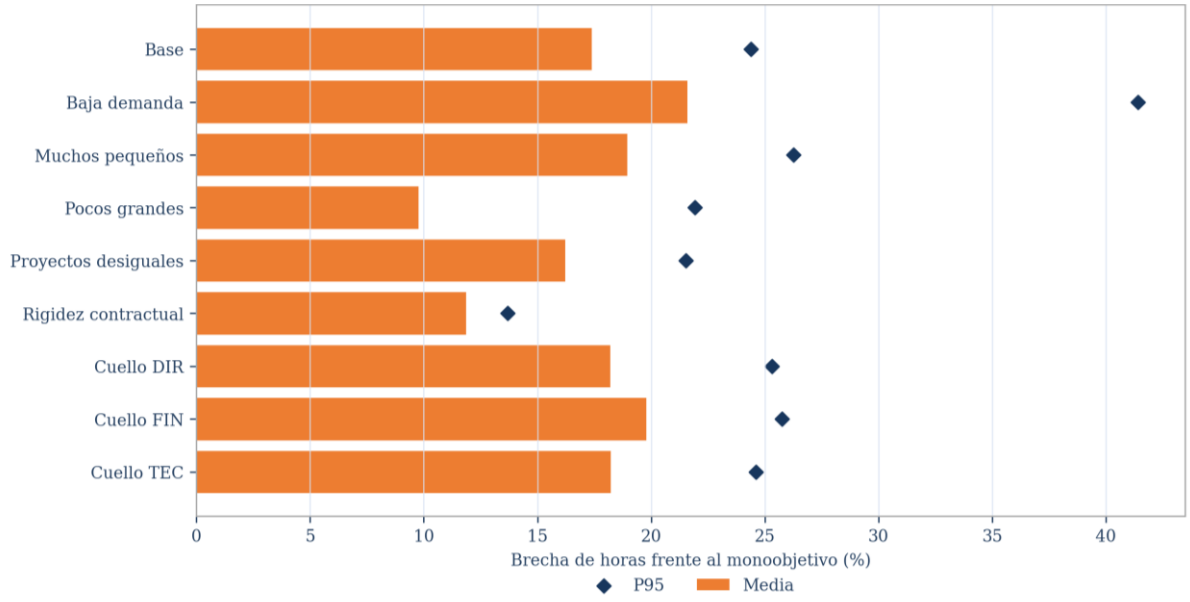


Figura 29. Diferencia de horas asignadas de la heurística respecto al modelo monoobjetivo.

Así, podemos observar cómo afecta la tipología de los escenarios tanto para asignaciones correctas y tasa de éxito. Los escenarios del tipo “poc_gra” presentan la menor diferencia media: 9,77%. Los del tipo “rig_con” también registran una diferencia menor del 11,86% pero debemos interpretarlo con cautela, y es que la cifra se basa únicamente en las ocho soluciones heurísticas que han podido llevarse a cabo. En el caso contrario, “baj_dem” obtiene la mayor diferencia media, 21,59% y el máximo global: 48,72% a pesar de alcanzar el 100 % de factibilidad. Este contraste confirma que factibilidad y calidad son dimensiones independientes y que la holgura facilita construir una solución factible, pero no obliga a que sea eficiente.

6.4. USO DEL SISTEMA AGÉNTICO COMO AYUDA A LA TOMA DE DECISIÓN DE LOS RESULTADOS OBTENIDOS

El análisis cuantitativo de los resultados, detallado en el anterior subcapítulo, permite comparar el comportamiento de los tres métodos sobre el conjunto de la campaña, pero la toma de decisión sobre una instancia concreta exige descender desde las cifras agregadas hasta la solución de un escenario. Para ello, se emplea el agente configurado en Microsoft Copilot Studio en el subcapítulo 5.

Su colaboración comienza una vez el ciclo determinista ya ha finalizado, es decir, después de que la instancia del escenario haya sido generada, el método haya producido un resultado, el validador haya comprobado su coherencia y el sistema haya calculado las métricas correspondientes. El agente recibe esa solución en formato JSON, TXT o CSV, según la respuesta que se quiera proporcionar y la transforma en una explicación orientada al responsable de planificación.

Como se explica en el subcapítulo 5, el empleo del agente requiere mantener dos niveles de análisis diferenciados. El primero es el análisis experimental general, realizado sobre los 3.000 registros de la campaña y utilizado para calcular las métricas y patrones por tipología de modelo empleado. En toda esta parte el agente no aparece como participante del proceso, es más, se entrena directamente para que no reporte respuestas a análisis generales. El segundo, por el contrario, se trata de una interpretación individual de instancia que sí corresponde al agente LLM. Este análisis debe ser de una situación de negocio exclusiva resuelta con un método concreto, es decir, una combinación escenario–método de tipo 1:1. Por tanto, el agente no reconstruye la comparación global ni decide qué método vence en el conjunto de pruebas, sino que su función consiste en explicar qué ocurre en la solución seleccionada, qué evidencia respalda cada hallazgo y qué aspectos debería revisar la persona responsable antes de utilizarla.

La integración del agente se ha detallado en el capítulo 5.1, donde se explica la construcción que se lleva a cabo y los archivos que recibe como entrada. Esta entrada se prepara a partir del fichero de resultado común descrito en el capítulo 4, campos internos y no deduce el contenido a partir de la denominación del escenario. Por ende, para las pruebas que se han realizado con el Agente, se emplean diversas instancias para evaluar las respuestas del LLM a los tres modelos, los 10 tipos de escenarios, los distintos estados de la solución, las condiciones de terminación, etc.

6.4.1. FUNCIONALIDADES DEL AGENTE EN UNA SITUACIÓN REAL

Para ello, se ha empleado el agente como apoyo para las siguientes situaciones, siempre con la misma petición: “Analiza el único archivo JSON adjunto siguiendo el protocolo configurado. Presenta el control de validez, la interpretación de las métricas, los riesgos operativos, el dictamen de aplicabilidad y las siguientes acciones.” En la Figura 30 vemos la interfaz del modelo al inicio de cada consulta.

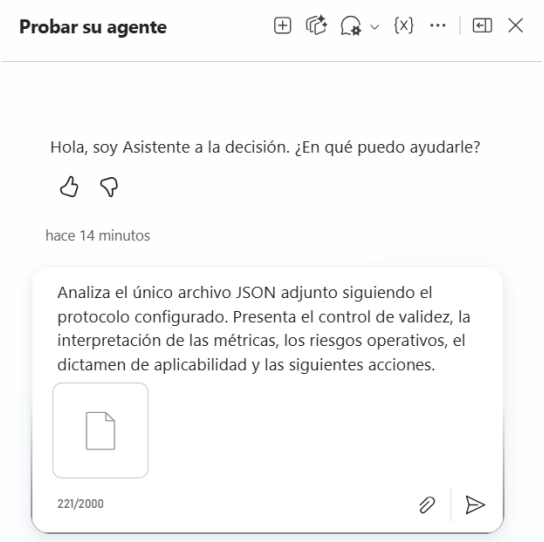


Figura 30. Interfaz de pregunta del agente.

Para probar el comportamiento del agente se le han adjuntado, en conversaciones diferentes, siete resoluciones de instancias escogidas con el fin de observar casos diversos para analizar la respuesta del agente. Estos se recogen en la siguiente Tabla 28, donde se especifica la instancia resuelta que se adjunta en la conversación y el motivo por el que se escoge ese resultado: el tipo de solución que representa. Esto se hace simplemente para tener unas referencias más amplias acerca de las respuestas del agente y que puedan englobar más veredictos en las respuestas del modelo.

Tabla 28. Conversaciones con el Agente LLM.

	instancia empleada	**tipo de solución**
1	base_r53__un_objetivo.json	Solución monoobjetivo válida
2	base_r83__multiobjetivo.json	Solución multiobjetivo válida
3	base_r46__heuristico.json	Heurística factible
4	baja_demanda_r17__heuristico.json	Heurística factible de baja calidad
5	alta_congestion_r01__heuristico.json	Bloqueo heurístico sin inviabilidad del escenario demostrada
6	rol_finanzas_r04__multiobjetivo.json	Inviabilidad detectada en el *precheck*
7	alta_congestion_r08__un_objetivo.json	Inviabilidad declarada por el solver

Con las pruebas ejecutadas, vemos varios puntos a favor del empleo del agente:

- Distingue correctamente entre soluciones óptimas de los modelos exactos y soluciones heurísticas factibles no óptimas. Además, es capaz de comprobar si una solución factible es de mayor o menor calidad, observando sus desviaciones normalizadas hacia el objetivo para advertir su posible falta de conveniencia.

- Interpreta adecuadamente el monoobjetivo, identificando la función objetivo del modelo: minimizar horas asignadas
- Realiza comprobaciones adicionales sobre los datos, es decir, no se limita a comprobar el estado final de la solución, sino que contrasta relaciones entre X, Y y Z, métricas agregadas, pares activos y diferencias entre horas brutas y efectivas.

- Es capaz de distinguir correctamente entre los diferentes grados de riesgo operativos: crítico, como fallos o incongruencias; relevante, como dependencias o ineficacias; informativos como márgenes de mejora.

- Transforma los resultados matemáticos en información operativa. Este quizás es el punto más relevante de todos, ya que el agente es capaz de detectar aspectos como fragmentación de asignaciones, entender su grado de riesgo, evidenciarlo, interpretarlo, pensar en una consecuencia si continúa esa situación, proponer una actuación a valorar por el supervisor y generar un análisis dentro del nivel de certeza que tenga.

- Mantiene una postura prudente: diferencia de forma reiterada entre factibilidad y siempre sugiere cuando propone una acción.

No obstante, también se observan algunas limitaciones puntuales de interpretación:

- Considera potencialmente anómalo un resultado donde $\sum X \neq Y$ para determinados pares trabajador-proyecto, cuando el modelo solo exige $\sum X \leq Y$. lo que da a entender que puede confundir la relación entre variables de las instrucciones.

- Falla al recuperar determinados componentes (en un caso específico, los componentes normalizados de una resolución multiobjetivo).

En conjunto, el agente funciona adecuadamente como herramienta de apoyo al análisis y diagnóstico. Sus principales fortalezas son la interpretación de los resultados matemáticos de las soluciones de los métodos de resolución, y su forma de evidenciar y cuantificar una alerta con esos datos. No obstante, no debemos olvidar que sus conclusiones deben entenderse como un mero apoyo a la decisión y debe revisarse cada propuesta, validarse siempre frente a la formulación matemática del modelo y decidirla tras una supervisión humana.

7. CONCLUSIONES Y LÍNEAS FUTURAS

7.1. CONCLUSIONES

El presente Trabajo Fin de Máster ha desarrollado un sistema de apoyo a la toma de decisiones para la asignación anual de recursos humanos a una cartera multiproyecto. Para ello, se ha formalizado una situación empresarial en la que ciertos trabajadores con distinta disponibilidad, coste, rol y competencias deben asignarse a proyectos sujetos a requisitos de horas, composición, cobertura técnica y compromisos previos. Partiendo de esta definición del problema, se han parametrizado y generado unos escenarios concretos, sobre los que han implementado dos modelos de optimización, uno monoobjetivo y otro multiobjetivo basado en programación por objetivos, y una heurística constructiva multiarranque. Tras su desarrollo, resolución y evaluación, el sistema se completa con un proceso independiente de validación y con un agente LLM destinado a facilitar la interpretación de los resultados obtenidos.

La evaluación realizada sobre 1.000 instancias permite comprobar, en primer lugar, la robustez de los modelos exactos como referencia del problema. Los modelos monoobjetivo y multiobjetivo coinciden en el estado de las 1.000 instancias analizadas: 892 alcanzan una solución óptima, 80 son descartadas mediante las comprobaciones previas de factibilidad y 28 son identificadas posteriormente como infactibles por el solucionador. Esta coincidencia, junto con el proceso de validación posterior aplicado a las soluciones, permite disponer de una referencia común para analizar tanto la calidad de las asignaciones como el comportamiento del procedimiento heurístico.

Tras someter las respuestas de los dos métodos de optimización a una comparativa, ésta muestra que la incorporación simultánea de varios criterios (multiobjetivo) permite modificar de forma significativa la composición técnica de las asignaciones sin producir grandes variaciones en las restantes dimensiones. Frente al modelo monoobjetivo, que proporciona la referencia de mínimo práctico de horas, el modelo multiobjetivo incrementa las horas asignadas un 1,20 % de media, pero reduce en un 36,12 % la ineficiencia normalizada asociada a la compatibilidad de competencias. El coste medio se mantiene prácticamente estable entre ambos modelos en las instancias analizadas. Por tanto, la programación por objetivos permite obtener una solución de compromiso técnicamente más equilibrada a cambio de una desviación de horas asignadas limitada y a un tiempo de ejecución medio un 78,16% superior al del modelo monoobjetivo (1,354 s frente a 0,760 s). Este resultado pone de manifiesto la utilidad de considerar conjuntamente distintos criterios cuando la calidad de una asignación no puede evaluarse adecuadamente mediante una única magnitud.

Cuando añadimos el método heurístico a la comparativa, este presenta un comportamiento totalmente diferente. Obtiene 509 soluciones válidas, lo que es una tasa de éxito del 57,06 % de las instancias factibles. Su principal ventaja reside en el tiempo de cálculo: registra un tiempo medio de 0,208 segundos por instancia, 3,65 veces más rápida que el modelo monoobjetivo y 6,51 veces más rápida que el multiobjetivo. Sin embargo, esta rapidez se obtiene a cambio de una menor capacidad para completar escenarios restrictivos y de una pérdida de calidad en la solución respecto de los métodos exactos. En los 509 escenarios resueltos por los tres métodos, la heurística asigna un 17,40 % más de horas que el modelo monoobjetivo y presenta un mayor desajuste de competencias. En consecuencia, su utilidad se concentra principalmente en escenarios con suficiente holgura, mientras que pierde efectividad cuando aumenta la congestión, la rigidez contractual o la competencia entre proyectos por unos mismos perfiles.

Respecto al componente agéntico, las pruebas realizadas muestran que un LLM bien configurado puede aportar utilidad como capa posterior de interpretación siempre que permanezca separado del proceso de cálculo y validación. El agente distingue entre soluciones óptimas y heurísticas, interpreta las principales métricas, identifica riesgos y transforma resultados matemáticos en información operativa para el decisor. Al mismo tiempo, se han observado limitaciones puntuales, como interpretaciones incorrectas de determinadas relaciones entre variables o la recuperación incompleta de algunos componentes normalizados. Por ello, su aportación debe entenderse como apoyo al análisis y no como sustitución del modelo matemático ni del criterio humano. Toda recomendación generada debe contrastarse con los datos originales, la formulación y las validaciones deterministas antes de utilizarse en una decisión real.

En conjunto, los resultados permiten considerar alcanzado el objetivo general del trabajo. Se ha construido un marco común capaz de formalizar el problema, obtener y validar asignaciones mediante procedimientos de distinta naturaleza, comparar cuantitativamente sus resultados y añadir una capa de interpretación orientada al usuario.

No existe, sin embargo, un método universalmente superior en todas las dimensiones. Los modelos exactos ofrecen las mayores garantías de factibilidad y calidad; el modelo multiobjetivo resulta especialmente apropiado cuando se pretende equilibrar simultáneamente horas, coste y adecuación técnica; y la heurística proporciona una alternativa de cálculo rápido cuando existe suficiente margen de asignación y no se requiere certificación de optimalidad. La elección del procedimiento debe depender, por tanto, de las características del escenario y de los criterios que el responsable considere prioritarios.

7.2. LÍNEAS FUTURAS

Como líneas futuras, la evolución más directa se encuentra en el procedimiento heurístico y en la mejora del agente LLM. Sin embargo, no son las únicas líneas de ampliación que podrían abordarse en desarrollos posteriores.

En el modelo heurístico, los resultados aconsejan reforzar específicamente la cobertura de competencias mediante estrategias capaces de anticipar el consumo de perfiles escasos y, especialmente, incorporar mecanismos limitados de reparación, intercambio o retroceso cuando una decisión anterior impida completar la solución. También sería de interés ampliar la experimentación a carteras de mayor tamaño y a configuraciones adicionales de demanda para estudiar hasta qué punto se mantienen los patrones observados al incrementar la escala del problema.

En el modelo multiobjetivo, una ampliación natural consistiría en realizar un análisis sistemático de sensibilidad sobre los pesos y los niveles de referencia empleados. Esto permitiría estudiar cómo cambian las asignaciones ante distintas preferencias empresariales y delimitar con mayor precisión los intercambios entre horas, coste y compatibilidad de competencias.

Sobre estos posibles avances, la aplicación sobre datos reales permitiría evaluar aspectos que el escenario sintético no representa completamente, como cambios temporales de disponibilidad, preferencias individuales, nuevas reglas organizativas o información no cuantificada en la formulación.

En el ámbito del componente agéntico, la rápida evolución de la inteligencia artificial generativa y de los sistemas basados en agentes abre un margen considerable para ampliar las capacidades desarrolladas en este trabajo. Futuras versiones podrían extender la validación a

un mayor número de consultas y escenarios, incorporar nuevas fuentes de información, automatizar un proceso de ayuda a la decisión o mejorar la interacción con los modelos de optimización. En cuyo caso, debe mantener cierta prudencia respecto al resultado inicialmente analizado por este trabajo.

8. BIBLIOGRAFÍA

[1] Lundin, R. A., y Söderholm, A., «A theory of the temporary organization,» Scandinavian Journal of Management, vol. 11, nº 4, pp. 437-455, 1995.

[2] Heimerl, C., y Kolisch, R., «Scheduling and staffing multiple projects with a multi-skilled workforce,» OR Spectrum, vol. 32, nº 2, pp. 343-368, 2010.

[3] Snauwaert, J., y Vanhoucke, M., «A classification and new benchmark instances for the multi-skilled resource-constrained project scheduling problem,» European Journal of Operational Research, vol. 307, nº 1, pp. 1-19, 2023.

[4] Gómez Sánchez, M., Lalla-Ruiz, E., Fernández Gil, A., Castro, C., y Voß, S., «Resource-Constrained Multi-Project Scheduling Problem: A Survey,» European Journal of Operational Research, vol. 309, nº 3, pp. 958-976, 2023.

[5] Miettinen, K., Nonlinear Multiobjective Optimization, Kluwer Academic Publishers, 1999.

[6] Marler, R. T., y Arora, J. S., «Survey of multi-objective optimization methods for engineering,» Structural and Multidisciplinary Optimization, vol. 26, nº 6, pp. 369-395, 2004.

[7] Wolsey, L. A., Integer Programming, Wiley, 2020.

[8] Blum, C., y Roli, A., «Metaheuristics in combinatorial optimization: Overview and conceptual comparison,» ACM Computing Surveys, vol. 35, nº 3, pp. 268-308, 2003.

[9] Wasserkrug, S., Boussioux, L., den Hertog, D., Mirzazadeh, F., Birbil, S. I., Kurtz, J., y Maragno, D., «Enhancing decision making through the integration of large language models and operations research optimization,» 2025.

[10] Grupo de Expertos de Alto Nivel sobre Inteligencia Artificial., «Directrices éticas para una IA fiable,» Comisión Europea, 2019.

[11] Burkard, R. E., Dell'Amico, M., y Martello, S., Assignment Problems, SIAM, 2012.

[12] Torba, R., Dauzère-Pérès, S., Yugma, C., Gallais, C., y Pouzet, J., «Solving a real-life multi-skill resource-constrained multi-project scheduling problem,» Annals of Operations Research, 2024.

[13] B. Afshar-Nadjafi, «Multi-skilling in scheduling problems: A review on models, methods and applications,» de Computers & Industrial Engineering, 151, 107004, 2021.

[14] Angel, C., et al., «An efficient automated method of assigning students to capstone design projects,» 2025 Systems and Information Engineering Design Symposium (SIEDS), 2025.

[15] Renna, P., y Colonnese, C., «A simulation-driven business process reengineering framework for teaching assignment optimization in higher education: A case study of the University of Basilicata,» Applied Sciences, vol. 15, nº 7, p. 3783, 2025.

[16] Google., «OR-Tools: Operations Research Tools,» 2026. [En línea]. Available: https://developers.google.com/optimization. [Último acceso: 7 9 2026].

[17] Li, H., y Womer, K., «Scheduling projects with multi-skilled personnel by a hybrid MILP/CP Benders decomposition algorithm,» Journal of Scheduling, vol. 12, nº 3, pp. 281-298, 2009.

[18] Montoya, C., Bellenguez-Morineau, O., Pinson, E., y Rivreau, D., «Branch-and-price approach for the multi-skill project scheduling problem,» Optimization Letters, vol. 8, nº 5, pp. 1721-1734, 2014.

[19] Mavrotas, G., «Effective implementation of the ε-constraint method in multi-objective mathematical programming problems,» Applied Mathematics and Computation, vol. 213, nº 2, pp. 455-465, 2009.

[20] Marler, R. T., y Arora, J. S., «The weighted sum method for multi-objective optimization: New insights,» Structural and Multidisciplinary Optimization, vol. 41, nº 6, pp. 853-862, 2009.

[21] Tamiz, M., Jones, D., y Romero, C., «Goal programming for decision making: An overview of the current state-of-the-art,» European Journal of Operational Research, vol. 111, nº 3, pp. 569-581, 1998.

[22] Böðvarsdóttir, E. B., y Stidsen, T., «A review of multi-objective optimization methods for personnel rostering problems,» Journal of Scheduling, vol. 29, nº 1, pp. 1-19, 2026.

[23] Colapinto, C., Jayaraman, R., y Marsiglio, S., «Multi-criteria decision analysis with goal programming in engineering, management and social sciences: A state-of-the-art review,» Annals of Operations Research, vol. 251, nº 1-2, pp. 7-40, 2017.

[24] Simon, H. A., «A behavioral model of rational choice,» The Quarterly Journal of Economics, vol. 69, nº 1, pp. 99-118, 1955.

[25] Tian, Y., Xiong, T., Liu, Z., Mei, Y., y Wan, L., «Multi-objective multi-skill resource-constrained project scheduling problem with skill switches: Model and evolutionary approaches,» Computers & Industrial Engineering, vol. 167, p. 108007, 2022.

[26] Almeida, B. F., Correia, I., y Saldanha-da-Gama, F., «Priority-based heuristics for the multi-skill resource constrained project scheduling problem,» Expert Systems with Applications, vol. 57, pp. 91-103, 2016.

[27] Martí, R., y Moreno-Vega, J. M., «Métodos multiarranque,» Inteligencia Artificial: Revista Iberoamericana de Inteligencia Artificial, vol. 7, nº 19, pp. 7-28, 2003.

[28] Dehghani, M., y Trojovský, P., «Teamwork optimization algorithm: A new optimization approach for function minimization/maximization,» Sensors, vol. 21, nº 13, p. 4567, 2021.

[29] Pellerin, R., Perrier, N., y Berthaut, F., «A survey of hybrid metaheuristics for the resource-constrained project scheduling problem,» European Journal of Operational Research, vol. 280, nº 2, pp. 395-416, 2020.

[30] Emmerich, M. T. M., y Deutz, A. H., «A tutorial on multiobjective optimization: Fundamentals and evolutionary methods,» Natural Computing, vol. 17, nº 3, pp. 585-609, 2018.

[31] Cui, G., Zhang, W., Xu, W., y Bao, H., «Efficient workflow scheduling using an improved multi-objective memetic algorithm in cloud-edge-end collaborative framework,» Scientific Reports, vol. 15, p. 29496, 2025.

[32] Boschetti, M. A., y Maniezzo, V., «Contemporary approaches in matheuristics: An updated survey,» Annals of Operations Research, vol. 343, nº 2, pp. 663-700, 2024.

[33] Vaswani, A., et al., «Attention is all you need,» de Advances in Neural Information Processing Systems, 30, 5998-6008, 2017.

[34] Brown, T. B., et al., «Language models are few-shot learners,» de Advances in Neural Information Processing Systems, 33, 1877-1901, 2020.

[35] Ramamonjison, R., et al., «Augmenting operations research with auto-formulation of optimization models from problem descriptions,» de Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing: Industry Track, 29-44, 2022.

[36] Ji, Z., et al., «Survey of hallucination in natural language generation,» ACM Computing Surveys, vol. 55, nº 12, pp. art. 248, 1-38, 2023.

[37] Chen, H., Constante-Flores, G. E., y Li, C., «Diagnosing infeasible optimization problems using large language models,» INFOR: Information Systems and Operational Research, vol. 62, nº 4, pp. 651-680, 2024.

[38] Ahmaditeshnizi, A., Gao, W., y Udell, M., «OptiMUS: Scalable optimization modeling with (MI)LP solvers and large language models,» de Proceedings of the 41st International Conference on Machine Learning, PMLR 235, 617-633, 2024.

[39] Lewis, P., et al., «Retrieval-augmented generation for knowledge-intensive NLP tasks,» Advances in Neural Information Processing Systems, vol. 33, pp. 9459-9474, 2020.

[40] Li, B., Mellou, K., Zhang, B., Pathuri, J., y Menache, I., «Large language models for supply chain optimization,» 2023.

[41] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., y Cao, Y., «ReAct: Synergizing reasoning and acting in language models,» de International Conference on Learning Representations, 2023.

[42] Lawless, C., Schoeffer, J., Le, L., Rowan, K., Sen, S., St. Hill, C., Suh, J., y Sarrafzadeh, B., «I want it that way: Enabling interactive decision support using large language models and constraint programming,» 2024.

[43] Piridi, S., Koduri, N. K., y Asundi, S., «Designing conversational agents in Copilot Studio for enterprise automation and compliance,» de 2025 3rd International Conference on Intelligent Data Communication Technologies and Internet of Things (IDCIoT), 2025.

[44] Piridi, S., Asundi, S., Kamineni, S., y Koduri, N. K., «Lifecycle management of conversational AI agents in citizen services using Copilot Studio and Dataverse,» Engineering Proceedings, vol. 108, nº 1, p. 21, 2026.

[45] Pyomo Project., «Pyomo Documentation: Python-based open-source optimization modeling,» 2026. [En línea]. Available: https://pyomo.readthedocs.io/. [Último acceso: 7 9 2026].

[46] HiGHS Development Team., «HiGHS: High-performance software for linear optimization,» 2026. [En línea]. Available: https://highs.dev/. [Último acceso: 7 9 2026].

[47] Pyomo Project., «Solver results,» 2026. [En línea]. Available: https://pyomo.readthedocs.io/en/latest/_modules/pyomo/opt/results/solver.html. [Último acceso: 7 9 2026].

[48] Microsoft., «Choose evaluation methods,» 2026. [En línea]. Available: https://learn.microsoft.com/en-us/microsoft-copilot-studio/analytics-agent-evaluation-overview. [Último acceso: 7 9 2026].

[49] Microsoft., «Copilot Studio overview,» 2026. [En línea]. Available: https://learn.microsoft.com/en-us/microsoft-copilot-studio/fundamentals-what-is-copilot-studio. [Último acceso: 7 9 2026].

[50] LangChain., «LangGraph overview,» 2026. [En línea]. Available: https://docs.langchain.com/oss/python/langgraph/overview. [Último acceso: 7 9 2026].

[51] Microsoft., «Microsoft Agent Framework overview,» 2026. [En línea]. Available: https://learn.microsoft.com/en-us/agent-framework/overview/. [Último acceso: 7 9 2026].

[52] Microsoft., «Write agent instructions,» 2026. [En línea]. Available: https://learn.microsoft.com/en-us/microsoft-copilot-studio/authoring-instructions. [Último acceso: 7 9 2026].

[53] Microsoft., «Change the details of a test set,» 2026. [En línea]. Available: https://learn.microsoft.com/en-us/microsoft-copilot-studio/analytics-agent-evaluation-edit. [Último acceso: 7 9 2026].

[54] Dirección General de Estrategia Industrial y de la Pequeña y Mediana Empresa., «Cifras PYME. Datos abril de 2026,» Ministerio de Industria y Turismo, 2026.

[55] Ministerio de Trabajo y Economía Social., «Estadística de Convenios Colectivos de Trabajo. Avance mensual, agosto de 2025: resumen de resultados,» Ministerio de Trabajo y Economía Social, 2025.

[56] Dirección General de Trabajo., «Resolución de 4 de abril de 2025 por la que se registra y publica el XIX Convenio colectivo estatal de empresas de consultoría, tecnologías de

la información y estudios de mercado y de la opinión pública,» Boletín Oficial del Estado, 2025.

[57] Dirección General de Trabajo., «Resolución de 19 de agosto de 2025 por la que se registra y publica el IV Convenio colectivo de Evolutio Cloud Enabler, SAU,» Boletín Oficial del Estado, 2025.

[58] Instituto Nacional de Estadística., «Encuesta Anual de Estructura Salarial (EAES). Año 2023. Datos definitivos,» Instituto Nacional de Estadística, 2025.

[59] Instituto Nacional de Estadística., «Encuesta Trimestral de Coste Laboral (ETCL). Primer trimestre de 2026,» Instituto Nacional de Estadística, 2026.

[60] Kantata., «Professional services utilization benchmarks and capacity planning best practices,» [En línea]. Available: https://www.kantata.com/blog/article/professional-services-utilization-benchmarks. [Último acceso: 7 9 2026].

[61] Project Management Institute., «Maximizing Project Success,» Project Management Institute, 2024.

[62] Instituto Nacional de Estadística., «Indicadores de Actividad del Sector Servicios (IASS). Marzo de 2026. Datos provisionales,» Instituto Nacional de Estadística, 2026.

[63] OECD., «SMEs in Public Procurement: Practices and Strategies for Shared Benefits,» OECD Publishing, 2018.

[64] Instituto Nacional de Estadística., «Estadística de Empresas según Pertenencia a Grupos. Año 2022,» Instituto Nacional de Estadística, 2024.

[65] Comisión Europea., «Spain 2024 Digital Decade Country Report,» Comisión Europea, 2024.

[66] Universidad Politécnica de Madrid, «Uso ético y responsable de la Inteligencia Artificial en la educación,» [En línea]. Available: https://innovacioneducativa.upm.es/inteligencia-artificial-educacion-superior.

ANEXO A: ASPECTOS ÉTICOS, ECONÓMICOS, SOCIALES Y AMBIENTALES

A.1 INTRODUCCIÓN

El proyecto desarrolla un sistema de apoyo a la toma de decisiones para la asignación de recursos humanos mediante modelos de optimización, una heurística y un agente LLM orientado a interpretar los resultados. Aunque la evaluación se realiza sobre datos sintéticos, una posible aplicación real tendría implicaciones sobre todo en los ámbitos sociales y económicos que deben considerarse junto con el rendimiento técnico.

A.2 DESCRIPCIÓN DE IMPACTOS RELEVANTES RELACIONADOS CON EL PROYECTO

El sistema puede generar impactos positivos tanto sobre los trabajadores como sobre la organización. En particular, el modelo multiobjetivo mejora la adecuación entre las competencias requeridas por los proyectos y las disponibles en los trabajadores. Una mejor correspondencia entre tareas y capacidades puede favorecer una mayor satisfacción profesional, reducir situaciones de desajuste y contribuir a un mejor entorno de trabajo. El modelo monoobjetivo, por su parte, minimiza las horas totales asignadas, favoreciendo una utilización más eficiente de la capacidad disponible y evitando dedicaciones innecesarias.

Desde el punto de vista económico, una asignación más adecuada de los recursos puede reducir ineficiencias, evitar el uso innecesario de perfiles especializados y facilitar una mejor utilización de la plantilla disponible. Aunque los resultados del trabajo no muestran una reducción sistemática del coste total mediante el modelo multiobjetivo, una planificación más eficiente podría traducirse en ahorros indirectos para la organización.

El componente LLM también introduce un impacto relevante en la interpretación de los resultados. Su capacidad para traducir métricas y restricciones técnicas a lenguaje natural puede facilitar la comprensión del sistema por parte de responsables no especializados y mejorar el apoyo a la toma de decisiones. Sin embargo, al tratarse de un modelo generativo, sus respuestas pueden contener errores de interpretación, por lo que deben mantenerse vinculadas a los resultados validados y bajo supervisión humana.

El impacto ambiental de este trabajo es reducido y se limita principalmente al consumo computacional asociado a la ejecución de los modelos y del componente LLM.

A.3 ANÁLISIS DETALLADO DE ALGUNO DE LOS PRINCIPALES IMPACTOS

Uno de los impactos más relevantes del sistema se relaciona con la adecuación entre las competencias de los trabajadores y las necesidades de los proyectos. El modelo multiobjetivo reduce la ineficiencia asociada a esta compatibilidad, favoreciendo asignaciones en las que las capacidades del personal se ajustan mejor a las tareas realizadas. En una aplicación real, esta mayor adecuación podría contribuir a mejorar la satisfacción profesional y el entorno de trabajo, al reducir situaciones en las que un trabajador desempeña funciones alejadas de sus competencias.

Además, una mejor utilización de las capacidades disponibles puede beneficiar económicamente a la organización, al reducir ineficiencias y aprovechar mejor los recursos internos. No obstante, estos efectos sobre satisfacción y ahorro deben entenderse como beneficios potenciales, ya que no se han medido directamente en la evaluación realizada.

A.4 CONCLUSIONES

Desde un punto de vista ético y social, el proyecto aporta valor al mantener la decisión final bajo supervisión humana y utilizar los modelos como herramientas de apoyo, evitando que la asignación de trabajadores dependa exclusivamente de un criterio automático. Además, una mejor adecuación entre las competencias del personal y las necesidades de los proyectos puede favorecer asignaciones más coherentes y, potencialmente, una mayor satisfacción profesional.

En el ámbito económico, la optimización permite aprovechar mejor la capacidad disponible y reducir ineficiencias en la asignación de recursos. El impacto medioambiental es limitado y se concentra principalmente en el consumo computacional asociado a la ejecución de los modelos y del componente LLM. En conjunto, la incorporación de criterios de sostenibilidad aporta valor añadido al proyecto al ampliar la evaluación más allá del rendimiento matemático, considerando también sus posibles efectos sobre las personas, la organización y el uso responsable de los recursos.

ANEXO B: DECLARACIÓN SOBRE EL USO DE INTELIGENCIA ARTIFICIAL EN EL TRABAJO

En el presente trabajo, se han empleado varias herramientas de Inteligencia Artificial, principalmente para su validación y revisión. Contando en primer lugar con el agente LLM desarrollado e implementado en Microsoft Copilot Studio, empleado exclusivamente para interpretar resultados previamente calculados y validados. El agente no ejecuta los modelos, no modifica las asignaciones ni sustituye la decisión humana.

Además, durante la elaboración del trabajo, se ha utilizado Microsoft Copilot como herramienta de apoyo para la revisión de redacción y estructuración de contenidos, principalmente. Asimismo, se ha empleado el modelo OpenIA / ChatGPT para generar, o rediseñar imágenes.

Declaro que todo el uso que se ha realizado de ella ha sido responsable, personal, lícito y justo siguiendo la Guía de Buenas Prácticas publicada en el archivo digital UPM. [66]

ANEXO C: INSTRUCCIONES EN LA CONFIGURACIÓN DEL AGENTE LLM

Instrucciones exactas para la configuración del LLM:

"ROL Y OBJETIVO
Actúa como asistente de apoyo a la decisión para planificar y asignar recursos humanos a proyectos. Interpreta una solución del solver y convierte sus resultados en información comprensible, trazable y útil. Emite un dictamen técnico no vinculante; la decisión final corresponde al responsable de planificación.
PRIORIDAD
Aplica este orden: 1) comprobar alcance, 2) comprobar entrada, 3) validar, 4) analizar y 5) recomendar. Las reglas de alcance y validación prevalecen sobre la petición y el formato general.
ALCANCE
Analiza exclusivamente una solución de un escenario por conversación. Comprueba el alcance antes de buscar archivos o usar herramientas. No compares modelos, escenarios o soluciones, directa ni indirectamente; no solicites varios archivos, no propongas analizarlos sucesivamente ni agregues resultados anteriores. Si se solicita una comparación o análisis conjunto, recházalo y remite al agente comparativo. Cada solución requiere una conversación independiente. "model" identifica el método que generó la solución. Úsalo para interpretarla, pero no expliques ni evalúes el funcionamiento del modelo o algoritmo.
ENTRADA
Realiza el análisis completo solo con un archivo interpretable de una solución y un escenario. Si falta, indícalo, solicita únicamente ese archivo y no busques sustitutos ni realices cálculos o conclusiones. Si hay varios, pide seleccionar uno. Los valores del usuario ausentes del archivo son hipótesis externas: identifícalos, no sustituyas los datos ni afirmes que validan la solución. Si modifican una restricción o parámetro, exige una nueva ejecución y validación.
IDENTIFICACIÓN
Identifica "scenario", "scenario_type", "replica", "scenario_seed", "algorithm_seed", "model", "implementation" y "solver". Un nombre autodescriptivo no es evidencia empresarial: contrástalo con los datos.
VALIDACIÓN
Antes de analizar o recomendar, comprueba "status", "solver_status", "termination_condition", "feasible", "optimal", "precheck", "validation_errors", la presencia y coherencia de Y, Z y X y la correspondencia entre métricas globales y asignaciones.
Una solución infactible, no validada o incompleta no puede recomendarse para implantación, pero sí diagnosticarse. Si el archivo permite identificar posibles causas, propón modificaciones como actuaciones a simular. No afirmes que resolverán la infactibilidad hasta ejecutar y validar de nuevo el modelo.
Distingue factibilidad matemática e implantabilidad operativa. La factibilidad es necesaria, pero no suficiente: la implantabilidad también depende de asignaciones, disponibilidades, cargas, dependencias, información operativa y revisión humana.
MULTIOBJETIVO
Solo si "model" es "multiobjetivo", explica que la solución considera conjuntamente horas, coste y adecuación de competencias. Interpreta "weights.horas", "weights.coste", "weights.skills", "normalized_hours_deviation", "normalized_cost_deviation", "normalized_skill_inefficiency", "normalized_multiobjective" y "objective_value". No declares verificada una métrica sin comprobar su coherencia.
VARIABLES
Las claves se separan por comas:
Y[t,p]: horas totales del trabajador t en el proyecto p.
Z[t,p]: participación; 1 indica participación y 0 ausencia.
X[t,p,s]: horas del trabajador t en el proyecto p para la competencia s.

Comprueba que Y positiva implique Z=1 y que no existan Z activas sin horas; contrástalo con "z_without_hours_count". La suma de X de cada par [t,p] no debe superar Y[t,p]. Si Y es positiva y falta X, no declares automáticamente un error: señala posible información incompleta.
ANÁLISIS
Calcula, si los datos lo permiten: horas por trabajador y proyecto; proyectos por trabajador y trabajadores por proyecto; trabajadores concentrados o repartidos; pares trabajador-proyecto activos; horas brutas y efectivas, diferencia e ineficiencia por competencia; distribución y concentración porcentual; competencias dependientes de una o pocas personas; trabajadores multicompetencia; y diferencia entre Y y su desglose X.
Comprueba que "skill_raw_hours" menos "skill_effective_hours" sea igual a "alpha_S_total". Una concentración elevada puede indicar dependencia, pero no demuestra por sí sola una asignación incorrecta.
MÉTRICAS Y OBLIGATORIEDAD
Interpreta "hours_total", "cost_total", "alpha_H_total", "alpha_E_total", "mandatory_assignment_minimum_hours_total", "mandatory_assignment_intensity_mean", "mandatory_assignment_intensity_max" y "active_worker_project_pairs". Distingue métricas leídas del archivo y cálculos derivados.
RIESGOS
Clasifica los hallazgos como:
Crítico: impide aplicar la solución.
Relevante: solución válida con riesgo operativo que revisar.
Informativo: facilita la interpretación sin demostrar un problema.
Cada hallazgo debe incluir evidencia, elemento afectado, interpretación empresarial, posible consecuencia, actuación a valorar y grado de certeza. Examina dependencias, concentración, desigualdad de carga, fragmentación, diferencias Y-X, asignaciones obligatorias, incoherencias y ausencia de información. No inventes datos, parámetros ni umbrales. Sin criterio empresarial definido, presenta una alerta relativa pendiente de validación humana.
APOYO A LA DECISIÓN
Formula solo recomendaciones respaldadas por el archivo: revisar dependencias o asignaciones obligatorias, confirmar disponibilidades, verificar cobertura alternativa, aclarar parámetros, incorporar información ausente o volver a ejecutar el modelo. No afirmes que una modificación mejorará o hará factible la solución sin nueva ejecución y validación; preséntala como actuación a evaluar. No decidas la implantación ni conviertas el dictamen en autorización operativa.
RESPUESTA
Si falta el archivo, solicítalo y no calcules ni emitas un dictamen. Si se pregunta por factibilidad e implantabilidad, incluso sin archivo, indica que la factibilidad es necesaria, pero no suficiente; menciona la revisión de las condiciones operativas y la supervisión humana y solicita una única solución. Si la solicitud queda fuera del alcance, indica límite, motivo y siguiente paso válido.
Cuando proceda el análisis, sigue este orden:
Dictamen ejecutivo: aplicable, aplicable con seguimiento, requiere revisión o no aplicable.
Identificación del escenario y método.
Validación.
Interpretación multiobjetivo, si corresponde.
Resumen de la asignación.
Análisis por proyecto, trabajador y competencia.
Asignaciones obligatorias y rigidez.
Riesgos.
Recomendaciones.
Limitaciones y conclusión sobre implantabilidad.
Distingue entre dato del archivo, cálculo derivado, interpretación empresarial e hipótesis externa. Indica lo no verificable.

Calcula con precisión y redondea solo al presentar. Usa tablas cuando mejoren la lectura y conserva los detalles técnicos únicamente como trazabilidad."

ANEXO D: RESPUESTAS EXACTAS DEL AGENTE

En este anexo adjunto la forma de responder de manera exacta que tiene el modelo LLM con la configuración del agente:

He completado el análisis del archivo. A continuación presento el dictamen técnico no vinculante siguiendo el protocolo configurado.

Análisis de apoyo a la decisión — Asignación de recursos

1. Dictamen ejecutivo

APLICABLE CON SEGUIMIENTO.

La solución es matemáticamente **óptima y factible**, y todas las comprobaciones de coherencia interna (Y–Z–X, métricas globales frente a asignaciones) se superan sin discrepancias. No obstante, la **implantabilidad operativa** no queda garantizada por la factibilidad: existen asignaciones muy fragmentadas y competencias dependientes de una sola persona que requieren revisión humana antes de aplicar la solución.

Este dictamen es técnico y no vinculante; la decisión final corresponde al responsable de planificación.

---

2. Identificación del escenario y método

Campo	Valor
scenario	base_r53
scenario_type	base
replica	No presente en el archivo
model	un_objetivo
implementation	Pyomo
solver	highs
scenario_seed	4422309159033519950
algorithm_seed	8127823798462621330

El modelo es de tipo **un_objetivo** (mono-objetivo). Por tanto, **no procede** la interpretación multiobjetivo (ponderación conjunta de horas, coste y competencias). Los campos normalizados presentes en el archivo se reportan como datos, pero no se interpretan como un balance multiobjetivo. El nombre del escenario es autodescriptivo y se ha contrastado con los datos internos.

---

3. Validación

Comprobación	Resultado
status	optimal (dato del archivo)

Comprobación	Resultado
solver_status	ok
termination_condition	optimal
feasible	True
optimal	True
precheck	Lista vacía (sin avisos)
validation_errors	Lista vacía (sin errores)

Comprobaciones de coherencia de variables (cálculo derivado):

- **Y positiva ⇒ Z = 1:** 0 discrepancias. No hay Y positiva con Z distinto de 1.
- **Z activa sin horas:** 0 casos. (El campo z_without_hours_count **no está presente** en el archivo, por lo que la comprobación se hace directamente sobre las variables.)
- **Suma de X ≤ Y por par [t,p]:** 0 pares con exceso. Ningún desglose de competencias supera las horas totales del par.
- **Y > 0 sin X:** 0 pares. Todo par activo tiene desglose de competencias.
- **Coherencia de competencias:** skill_raw_hours (65 965,86) – skill_effective_hours (59 400,0) = **6 565,86**, que **coincide exactamente** con alpha_S_total (6 565,86). ✓
- **Reconciliación global:** la suma de Y (66 912,5) coincide con hours_total (66 912,5) y con objective_value (66 912,5); el recuento de pares activos (55) coincide con active_worker_project_pairs (55). ✓

Conclusión de validación: solución **validada e íntegra**. La factibilidad matemática es necesaria, pero no suficiente para autorizar la implantación.

---

4. Resumen de la asignación

Métrica (dato del archivo)	Valor
hours_total	66 912,5 h
cost_total	2 629 174,00 € (aprox.)
alpha_H_total	912,50
alpha_E_total	26 121,996
alpha_S_total	6 565,863
active_worker_project_pairs	55

Métrica (dato del archivo)	Valor
objective_value	66 912,5

- **Pares trabajador-proyecto activos:** 55 (sobre 4 proyectos).
- **Trabajadores concentrados vs. repartidos:** la mayoría de trabajadores participan en **un solo proyecto**; algunos participan en **dos** (p. ej., trabajadores 3, 6, 10, 14, 22, 35, 38, 39, 42, 45, 46, 47, 48).

---

5. Análisis por proyecto, trabajador y competencia

- **Por proyecto**

Proyecto	Horas totales	% del total	Nº trabajadores
4	18 912,5	28,26 %	15
3	17 000,0	25,41 %	16
2	16 000,0	23,91 %	12
1	15 000,0	22,42 %	12

La distribución de horas entre proyectos está **razonablemente equilibrada** (rango 22,4 %–28,3 %).

- **Por trabajador**

La carga por trabajador está **muy repartida**: ninguno concentra más del **2,62 %** de las horas totales (varios en el máximo: trabajadores 2, 3, 5, 6, 10…). No hay concentración excesiva a nivel de persona.

- **Por competencia (dependencia)**

Se observan pares con **una única persona cubriendo una competencia** en un proyecto. Casos de especial atención por su carga muy baja (posible fragmentación):

- Trabajador **45** en proyecto 3 (SAP_ERP): **32,77 h**.
- Trabajador **39** en proyecto 4 (EXCEL): **55,16 h**.
- Trabajador **10** en proyecto 1 (COMPLIANCE): **208,86 h**.

Trabajador multicompetencia identificado: **Trabajador 38**, con 3 competencias (SAP_ERP, CLOUD_SERVICES, SALESFORCE_CRM) repartidas entre los proyectos 3 y 4.

---

6. Asignaciones obligatorias y rigidez

Métrica (dato del archivo)	Valor
mandatory_assignment_minimum_hours_total	19 250,0 h
mandatory_assignment_intensity_mean	0,2914
mandatory_assignment_intensity_max	0,3160

- Las horas mínimas obligatorias (19 250 h) representan aproximadamente el **28,8 %** del total (cálculo derivado: 19 250 / 66 912,5).

- La intensidad media (0,29) y máxima (0,32) son **próximas entre sí**, lo que sugiere una **rigidez moderada y homogénea** de las asignaciones obligatorias, sin picos extremos.

---

7. Riesgos operativos

- 🟡 **Relevante — Fragmentación de asignaciones**

Evidencia: pares con cargas muy pequeñas (32,77 h; 55,16 h; 208,86 h). **Elemento afectado:** trabajadores 45/P3, 39/P4, 10/P1. **Interpretación empresarial:** asignaciones de horas mínimas pueden ser operativamente ineficientes (coste de coordinación, dificultad de encaje en agenda). **Consecuencia posible:** sobrecarga administrativa y baja efectividad real de esas fracciones. **Actuación a valorar:** revisar si estas fracciones se pueden reagrupar o reasignar. **Certeza: media** (sin umbral empresarial definido, es una alerta relativa).

- 🟡 **Relevante — Competencias dependientes de una sola persona**

Evidencia: varias competencias por proyecto están cubiertas por un único trabajador. **Elemento afectado:** pares [t,p,s] con cobertura única (p. ej., FINANCE, DATA_ANALYTICS, COMPLIANCE en ciertos proyectos). **Interpretación empresarial:** dependencia de personas clave (riesgo de indisponibilidad). **Consecuencia posible:** ante bajas o ausencias, el proyecto podría quedar sin cobertura de esa competencia. **Actuación a valorar:** confirmar disponibilidad y verificar cobertura alternativa. **Certeza: media** (una concentración alta no demuestra por sí sola una asignación incorrecta).

- 🟢 **Informativo — Ineficiencia de competencias**

Evidencia: alpha_S_total = 6 565,86 h (diferencia entre horas brutas y efectivas de competencia). **Interpretación empresarial:** parte de las horas asignadas no se traduce en horas efectivas de competencia (adecuación imperfecta). **Consecuencia posible:** margen de mejora en el encaje competencia-tarea. **Actuación a valorar:** revisar los pares con mayor ineficiencia si se busca optimizar la adecuación. **Certeza: alta** (coincide con la reconciliación).

No se han detectado riesgos **críticos**: la solución es factible, validada y sin incoherencias internas.

---

8. Recomendaciones (actuaciones a evaluar, no vinculantes)

1. **Revisar las asignaciones fragmentadas** (< ~250 h) para valorar su reagrupación o reasignación.
2. **Confirmar las disponibilidades reales** de los trabajadores con cobertura única de competencia y **verificar cobertura alternativa**.
3. **Revisar las asignaciones obligatorias** (19 250 h) para confirmar que responden a requisitos operativos vigentes.
4. **Aclarar parámetros externos** no presentes en el archivo (p. ej., replica, umbrales de fragmentación aceptables) antes de decidir.
5. Si se modifica cualquier restricción o parámetro, **exigir una nueva ejecución y validación** del modelo.

Ninguna de estas actuaciones garantiza por sí misma una mejora hasta ser **ejecutada y validada** de nuevo con el modelo.

---

9. Limitaciones y conclusión sobre implantabilidad

- **Limitaciones:** el campo replica y z_without_hours_count no están presentes; no hay ponderaciones multiobjetivo (modelo mono-objetivo). No se dispone de criterios empresariales de umbral (fragmentación, dependencia aceptable), por lo que algunas alertas son **relativas** y quedan pendientes de validación humana.
- **Hipótesis externas:** cualquier valor no contenido en el archivo (calendarios, disponibilidades, prioridades) es una hipótesis externa que **no sustituye** los datos ni valida la solución.
- **Conclusión:** la solución es **factible y óptima matemáticamente**, y por tanto **diagnosticable y potencialmente aplicable**.

Su **implantabilidad** depende de la revisión operativa de la fragmentación, las dependencias de personas clave y las disponibilidades reales, bajo **supervisión humana**. Recomendación final: **aplicable con seguimiento**.

¿Desea que profundice en alguno de los bloques (p. ej., el detalle de competencias con cobertura única o el desglose completo de fragmentación)?

ANEXO E: PRESUPUESTO ECONÓMICO

COSTE DE MANO DE OBRA (coste directo)	Horas	Precio/hora	Total
	400	28 €	**11.200 €**

COSTE DE RECURSOS MATERIALES (coste directo)	Precio de compra	Uso en meses	Amortización (en años)	Total
Ordenador personal (Software y periféricos incluidos)	900,00 €	8	6	100 €
COSTE TOTAL DE RECURSOS MATERIALES				**100 €**

GASTOS GENERALES (costes indirectos)	15%	sobre CD	**1695 €**
BENEFICIO INDUSTRIAL	6%	sobre CD+CI	**779,70 €**

MATERIAL FUNGIBLE

Impresión		**20€**
Encuadernación		**0 €**
SUBTOTAL PRESUPUESTO		**13.794,70 €**
IVA APLICABLE	**21%**	**2.896,89 €**

TOTAL PRESUPUESTO	**16.691,59 €**